



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校

西安交通大学

参赛队号

24106980067

队员姓名

1. 余浩然

2. 陈扬

3. 陈新永

中国研究生创新实践系列大赛

“华为杯”第二十一届中国研究生

数学建模竞赛

题 目： 高速公路应急车道紧急启用模型

摘 要：

随着国民经济水平的不断提升和城市间交通运输需求的日益增长，高速公路的交通流量持续攀升，交通大流量已逐渐成为常态。许多高速公路的通行能力接近饱和状态，尤其在节假日、出行高峰期等特殊时段，车流量的剧增导致交通压力骤增，拥堵现象频繁发生。这不仅影响了车辆的通行效率，还增加了道路管理和维护的难度，给社会经济发展带来了潜在的挑战。面对这一问题，如何提升高速公路的通行能力和管理水平，缓解拥堵压力，成为交通管理部门亟待解决的重要课题。本文围绕高速公路应急车道的启用建模问题展开，并根据竞赛题目中的四个子问题进行建模与求解。

针对问题一第一问，本文采用 **YOLOv5** 对监控视频中的车辆进行精准目标检测，并引入**注意力机制 SE** 模块以显著提升模型的精确度。此外还借鉴了残差连接的思想，使用 **BottleneckCSP** 替代原 **C3** 模块有效解决了模型在训练过程中出现的过拟合问题。目标检测后，利用 **DeepSORT** 算法对同一 ID 的车辆进行高效跟踪，通过计算前后两帧目标中心点距离的差异来获取车辆速度，同时设置基准线以评估车辆通过基准线的速度和车流量。最终，本文检测出车流量、车流速度，并且根据交通工程基本流量公式再求出密度，四个观测点的交通流参数随时间变化的规律详见表 3.11。

针对问题一第二问，本文在基本拥堵指数计算公式的基础上推导出了有关交通流量、交通流速度和交通流密度的拥堵指数公式。并且为了消除交通拥堵异常值、方便划分交通拥堵等级，我们提出了 **Sigmoid-标准交通拥堵指数**，标准化了交通拥堵指数，去除了异常值，并且根据 **Sigmoid-标准交通拥堵指数** 对拥堵情况进行了**五个等级**的划分。同时未来的交通流量和交通流速度，本文选择了具有较高时序特性的双向门控循环单元（**Bi-GRU**），为了捕获全局特征性，获取更多时间序列信息，本文引入了 **SK-Net-Attention** 机制，并使用了**时间卷积网络(TCN)**以进行时序信息的捕获,提出了 **TCNAt-BiGRU** 模型对未来的交通流量和交通流速度进行预测，并且与原模型进行对比实验验证了我们提出模型的有效性。最后对预测结果进行了实时预警建模，采用**拥堵波**计算的方法，通过标准拥堵指数和拥堵波进行节点三和节点四之间路段的实时预警，为实时预警提供了依据。

针对问题一第三问，本文使用了监控点 1 和监控点 2 的第一段视频数据进行模型训练。训练后的模型随后应用于第二段监控视频进行预测，同时根据 **MSE、RMSE、MAE、MAPE** 等指标评估其在实际应用中的有效性和泛化能力。结果表明，模型能够在拥堵发生前十分钟左右发出**有效的提前预警**。

针对问题二，本文提出了明确的应急车道开启与关闭决策条件。当激波速度较大且趋势指向上游区域将受到影响时，应及时启用应急车道，以有效应对潜在的拥堵风险。而当车流拥堵指数 $\gamma_{norm}^p(t)$ 降到临界拥堵指数 $\gamma_{critical}$ 时、车速恢复正常且激波向下游传播并减缓时，应果断关闭应急车道，以确保交通管理的高效性与安全性。

针对问题三，我们利用标准交通拥堵指数 $\gamma_{norm}^p(t)$ 与推导出的拥堵波 w ，建立联合算法控制应急车道的开关，实现节点间路段的分段控制。同时为验证算法的有效性，采用Greenshields模型进行建模，分析不同流量、速度和密度之间的关系。我们量化了应急车道启用的影响，评估其对缓解交通拥堵的效果。最终结果表明，所设计的决策算法在实际应用中具有显著的有效性，为改善交通流提供了可行的解决方案。

针对问题四，针对如何科学布置监控点以控制成本并提高决策的准确性，文章通过对道路情况和应急车道启用前后的数据进行分析，提出了优化的监控点布局方案。其核心思路是增加第三点到第四点之间的监控点数量，细分监控区域，并在关键路段增加布点。同时利用无人机和移动摄像头补充固定监控的不足，并通过升级现有设备提升数据质量和处理能力。此外，该方案建立了数据整合与共享平台，确保数据的集中管理与实时传输，从而支持科学决策和智能化交通管理。该方案优化监控点布置，提高高速公路应急车道管理的实时性和精度，同时控制建设成本，显著提升了道路通行效率和安全性。

总的来说，文章通过深度学习算法、时序预测模型和交通流分析，提供了一套完整的高速公路应急车道启用策略，具有较高的实用价值和理论创新。

关键词：YOLOv5; BottleneckCSP; DeepSORT 算法; 交通拥堵指数; Sigmoid 标准化; TCNAt-BiGRU 模型; 实时预警; 拥堵波（激波）; 应急车道启用算法; Greenshields 模型

目录

一、 问题重述	5
1.1 问题背景	5
1.2 问题提出	5
二、 模型假设与符号说明	6
2.1 模型假设	6
2.2 符号说明	6
三、 问题一模型建立与求解分析	7
3.1 子问题（1）	7
3.1.1 问题分析	7
3.1.2 问题求解	8
3.1.3 数据集构建	9
3.1.4 目标检测 YOLO 模型构建	9
3.1.5 目标跟踪 DeepSORT 算法	16
3.1.6 基于 YOLOv5 和 DeepSort 的多目标跟踪	24
3.1.7 交通流速度与流量的计算	30
3.1.8 实验数据与结果分析	32
3.2 子问题（2）	34
3.2.1 问题分析	34
3.2.2 交通拥堵阈值的计算与确定	36
3.2.3 时序预测模型结构	38
3.2.4 预测系统框架	41
3.2.5 消融对比实验	43
3.2.6 车流量和车流速度预测与分析	43
3.2.7 预测结果实时预警建模	46
3.3 子问题（3）	49
3.3.1 问题分析	49
3.3.2 验证模型有效性	50
四、 问题二模型建立与求解分析	54
4.1 问题分析	54
4.2 开启应急车道的理论条件	55
4.3 关闭应急车道的决策条件	57
五、 问题三模型建立与求解分析	58
5.1 问题分析	58
5.2 建立规则启用应急车道	59
5.3 量化启用应急车道模型	60
5.3.1 Greenshields 模型建模	60
5.3.2 模拟分析	61
5.3.3 量化结果分析	62
六、 问题四模型建立与求解分析	64
6.1 问题分析	64
6.2 问题求解	64
七、 模型评价与改进	66

7.1 基于 YOLO 和 DeepSORT 的目标跟踪模型.....	66
7.2 基于 Bi-GRU 的时序预测模型.....	66
参考文献	68
附录 python 程序	69

一、 问题重述

1.1 问题背景

本次研究重点在于探索如何在现有高速公路条件下，最大限度地利用资源来缓解潜在的交通拥堵现象。交通拥堵的成因多样，其中部分路段的瓶颈效应是一个典型问题，常见于匝道出入口、桥梁入口等位置。此外，由于某些高速公路车道数量有限，通行能力不足，导致了交通流动受限，进而加剧了拥堵。尽管拓宽道路能够有效解决这一问题，但这种措施成本高昂且不在本研究的讨论范围内。相比之下，交通事故属于随机事件，难以预测和控制，也不在此次的研究范围。

动态应急车道作为调整路段交通流运行现状的重要管理手段，可以在短时间内提供道路供给，有效提升瓶颈路段的通行能力。早在 21 世纪初，欧美国家就已经通过开放应急车道缓解的方式来解决路段的交通拥堵^{[1][2]}。应急车道通常不能被普通车辆占用，但在特殊情况下，如果能够在拥堵发生前对其进行合理利用，可能有效缓解潜在的交通阻塞。然而，目前对于应急车道的使用大多依赖于管理人员的经验和视频监控的判断，缺乏科学的理论指导，难以系统性地评估其对缓解拥堵的实际效果。

因此，本题的研究具有重要意义：通过建立数学模型，明确高速公路特定路段即将发生拥堵的条件，利用交通拥堵指数^[3]科学评估应急车道的临时借用对缓解交通拥堵的作用。这样可以为高速公路管理部门提供科学依据，提升应急车道的利用效率，最大程度地缓解拥堵，提高道路通行能力，保障交通顺畅。这将为高速公路的管理和运行提供重要的理论支持和现实指导，具有极大的实际应用价值。

1.2 问题提出

题目给出了长深高速公路某段上四个点位交通监控视频，道路为两条行车道和一条应急车道，长度约 5000m，此阶段没有出入口直至最后一个监控点位后才是入口匝道。

问题一：针对题目提供的数据统计四个观测点的交通流参数随时间的变化规律；建立交通流拥堵模型，利用交通流在四个观测点的基本参数和道路情况给出第三点到第四点之间路段可能出现持续拥堵状态的实时预警和依据；并利用视频数据验证模型的有效性；

问题二：构建合理启用高速公路应急车道模型为巨册这提供临时启用应急车道的理论依据；

问题三：利用监控数据，设计合理的规则或算法，实时决策是否启用应急车道，并量化模型启用应急车道的作用；

问题四：为了提升第三个点到第四个点之间路段应急车道临时启用决策的科学性并同时控制成本，应如何布置视频监控点并给出原因。

二、模型假设与符号说明

2.1 模型假设

1. 假设题目提供的数据真实可靠；
2. 假设路段的道路情况只与车流量和速度有关，不受其他因素影响；
3. 假设所有的道路情况为随机挑选；
4. 假设不考虑拓宽路幅来解决拥堵；
5. 假设不考虑交通事故等偶然事件；

2.2 符号说明

符号	意义
q	流量，单位是车辆/时间（如辆/分钟、辆/小时等）
Δt	时间间隔，通常以秒、分钟或小时为单位
N	在时间段 Δt 内通过的车辆数
k	密度，通常以车辆/单位长度（如辆/公里）表示
v	平均速度，表示在单位时间内车辆的平均行驶速度
$frame/min$	表示在每分钟有多少帧的数据被捕获，与交通流量直接相关
$\gamma_{norm}^p(t)$	标准交通拥堵指数，范围在(0,1)之间
w	拥堵波(激波)，因车辆减速导致的反向传播的车流密集波动
$\gamma_{critical}$	临界交通拥堵指数
$\gamma^p(t)$	拥堵指数
$occ^p(t)$	道路占有率
$t_{arrival}$	拥堵波到达时间
w_{ij}	观测节点 i 和 j 之间的拥堵波
t_{ij}	拥堵波从观测点 j 到 i 的传播时间
n	车道数
q_{max}	每条车道的最大通行流量
Q_{max}	道路流量的总通行能力
v_f	表示每条道路在无干扰情况下的速度

三、 问题一模型建立与求解分析

3.1 子问题（1）

3.1.1 问题分析

针对问题一的第一小问，需要根据题目提供的监控点视频信息，系统地统计四个观测点的交通流参数随时间的变化规律。本文采用了 YOLO 目标检测算法对高速公路上车辆的信息进行实时检测。在完成目标检测后，为了有效地实现连续帧间的车辆追踪，本文引入了 DeepSORT 算法。该算法通过卡尔曼滤波技术来估算目标在下一帧中的运动位置，并运用匈牙利算法实现前后两帧中的车辆 ID 的精准关联，从而确保每辆车的跟踪过程是连续且可靠的。

在完成对同一 ID 的车辆进行跟踪后，需要对车辆计的速度进行计算，首先从连续的两帧图像中提取具有相同 ID 的车辆中心点坐标，利用欧氏距离公式计算车辆在像素坐标系下的移动距离。然后将像素距离转换为实际距离，结合时间信息即可实时地计算出车辆的速度。通上述方法即可从监控视频中提取流量、密度及车流速度等关键交通流参数。

算法流程图如下：

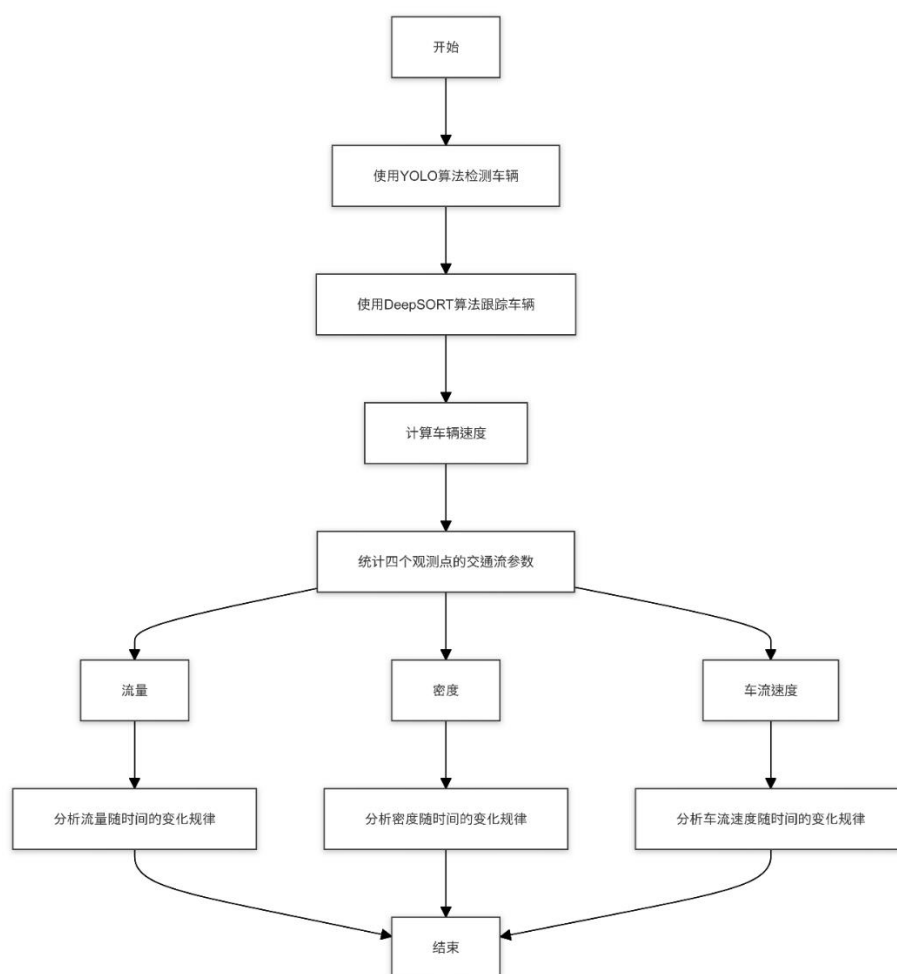


图 3.1 思路流程图

3.1.2 问题求解

1. 数据收集与预处理:

首先, 获取题目提供的监控数据, 这些数据包含以下内容:

- (1) 各个观测点在不同时刻的车流量(车辆数目)。
- (2) 各个观测点在不同时刻的车辆速度。
- (3) 各个观测点的观察时间间隔。

数据预处理:

本文在数据预处理中, 针对车辆拥堵状态的实时预警需求, 采用了一些特殊处理措施。首先, 考虑到道路上的实际限速情况及数据合理性, 同时由于建模主要关注的是低速拥堵的状况, 因此对于检测到的车速大于 120km/h 的车辆, 统一将其速度统计值设定为 120km/h, 以避免过高数据对模型造成的误差。对于由于车辆速度过快导致目标模糊, 进而无法进行准确检测而产生的异常速度数据, 也采取同样的处理方式, 即将其速度同样设定为 120km/h。对于车速在 120km/h 以下的车辆数据, 因其在正常范围内, 不作额外处理。

在本论文中我们对监控左上角显示的时间建模, 由于视频是经过抽帧处理的, 因此我们重新计算了帧数与左上角时间的对应关系, 计算得出每过 150 帧左上角经过一分钟, 即 **150 frame/min**, 在以下的所有计算中我们的时间都用的是视频中左上角显示的时间进行的建模。

2. 流量的计算:

根据公式:

$$q = \frac{N}{\Delta t} \quad (3-1)$$

其中 N 为单位时间内通过某观测点的车辆数, Δt 为时间间隔。

逐个观测点计算不同时段内的流量: 统计每分钟或每小时通过观测点的车辆数 N ; 选择适当的时间间隔(如一分钟或一小时), 计算各观测点的流量 q 。

结果呈现: 将随时间变化的流量结果绘制成图表, 展示出每个观测点的车流量随时间变化的趋势, 在本论文的建模中 q 的单位为 ($q: N/min$), 即每分中经过这个点有多少车辆。

3. 车速的计算:

对于每个观测点, 利用监控数据获取车辆的速度信息, 计算每个时段内车辆的平均速度 v 。

可以对每分钟或每小时的所有车辆速度取平均值, 公式如下:

$$\bar{v} = \frac{\sum_{i=1}^N v_i}{N} \quad (3-2)$$

其中, v_i 为第 i 辆车的速度, N 为该时间段内通过观测点的车辆总数。

结果呈现: 绘制出随时间变化的平均速度图表, 显示出各观测点车速的变化情况。在本论文的建模中 $\bar{v}$ 的单位为 ($\bar{v}: km/h$), 公式中的 N 为 1 分钟内经过车的数量。

4. 车密度的计算:

在我们得出流量和车速以后, 我们可以通过交通工程中的基本流量公式得出交通密度:

$$k = \frac{q}{v} \quad (3-3)$$

根据已经计算出的流量 q 和速度 v , 计算不同时段内的车密度 k 。逐个观测点和时间段计算车密度, 密度的单位通常为 ($k: \times 60 N/km$), 该单位是由车流量和车速量纲计算得出, 表示每千米有多少量车, 同时还要乘一个系数 60。

5. 绘制图表:

绘制车流量、速度和密度随时间变化的折线图，便于观察和分析每个观测点的交通流变化趋势。

3.1.3 数据集构建

采用 YOLO 目标检测算法对高速公路上的车辆进行识别和跟踪需要大量的车辆数据集对模型进行训练，为此，我们选择了 UA-DETRAC 车辆检测数据集。

UA-DETRAC 车辆检测数据集是一个专门用于车辆检测与跟踪的基准数据集。该数据集提供了丰富的交通场景和车辆类别的图像与视频数据，是自动驾驶和智能交通领域的重要基准。视频拍摄自中国不同城市的交通摄像头，场景包括城市道路、乡村道路、高速公路等，具有不同的光照条件和天气变化。数据集覆盖了不同的交通密度，包括稀疏和拥挤的交通场景，符合我们对数据集的要求。

最终我们选择了数据集中的 14034 张图片进行模型训练。

3.1.4 目标检测 YOLO 模型构建

1. YOLOv5 系列各模型对比：

YOLOv5 是一种备受欢迎的深度学习神经网络的版本，也是广泛应用的版本之一，主要用于计算机视觉领域的各种机器学习任务。由于其在复杂和嘈杂的数据环境中表现出的卓越性能、可用性以及与广泛使用的编程语言如 Python 相结合的易用性，YOLO 算法在数据科学界稳步获得认可。

YOLOv5 模型包括十个单独的体系结构^[4]，分别为 YOLOv5n、YOLOv5s、YOLOv5m、YOLOv5l、YOLOv5x、YOLOv5n6、YOLOv5s6、YOLOv5m6、YOLOv5l6、YOLOv5x6+TTA。然而，通常只考虑前五种情况进行研究。它们之间的主要差别在于特征提取模型和卷积核的数量。

YOLOv5 常用模型在 COCOval 2017 数据集上的对比如表 3-1 所示。

表 3-1 常用 YOLOv5 模型对比			
模型	mAP_0.5	mAP_0.5:0.95	训练持续时间 (s)
YOLOv5n	46.0	28.4	6.3
YOLOv5s	56.0	37.2	6.4
YOLOv5m	63.9	45.2	8.2
YOLOv5l	67.2	48.8	10.1
YOLOv5x	68.9	50.7	12.1

Horvat M 等人从实验数据集中^[4]，记录了 YOLOv5 各个版本的模型处理单个随机图像的时间，见表 3-2。在选择最佳网络架构时，需要在模型的质量和速度之间进行权衡。为了使更复杂的模型在速度方面具有竞争力，需要更强的硬件支持，然而，硬件资源并不总是易于获取和满足的。因此，在实际应用中，需要根据具体的应用场景和需求，综合考虑模型的质量和速度等因素，选择最适合的网络架构。

表 3-2 处理单个随机图像的时间对比				
模型	预处理时间	处理过程时间	非极大值抑制	总时间
YOLOv5n	0.2	5.1	2.5	7.8

YOLOv5s	0.5	5.0	2.0	7.5
YOLOv5m	0.3	11.2	1.8	13.3
YOLOv5l	0.6	17.8	2.0	20.4
YOLOv5x	0.8	32.7	1.8	35.3

YOLOv5 的不同变体针对不同的计算能力和实时性需求进行了优化。在选择合适的模型时，主要考虑设备的计算能力和任务对准确度的要求。鉴于本实验任务对模型的要求不高，且数据集的规模较小，本毕业设计选择采用 YOLOv5s 版本进行目标检测。该版本相对于 YOLOv5 的其他版本，具有更小的模型尺寸和更快的推理速度，能够更好地适应本实验的需求。

2. YOLOv5 目标检测算法：

YOLOv5 主要由输入端、Backbone、Neck 和 Prediction 四部分组成。其中，输入端负责接收并处理输入数据，Backbone 用于提取图像特征，Neck 用于特征融合和下采样，Prediction 则用于目标检测和定位。这种结构设计可以使模型更好地适应不同尺度和复杂度的目标，提高模型的检测精度和鲁棒性。

以下是 YOLOv5 在输入端采用的主要方法：

（1）Mosaic 数据增强。

Mosaic 数据增强^[5]是将经过随机裁剪、缩放等操作的四张图像拼接为一张新的图像，并将其输入网络进行训练，从而增加了训练数据的多样性，极大地丰富了检测物体的背景。

（2）自适应锚框计算。

对于不同的数据集通常需要不同的锚框尺寸来适应不同的目标物体大小和比例。在 YOLOv5 算法中，为了进一步提高模型的性能和鲁棒性，将锚框尺寸的计算嵌入到训练模型的过程中，使得模型能够自动计算最佳锚框，以便在不同数据集上训练时不必再单独设计锚框尺寸^[6]。

（3）自适应图片缩放。

在目标检测算法中，由于不同图片的长宽不相同，因此需要对输入图像进行预处理，以便更好地适应不同尺度的输入图像。自适应图片缩放是一种常用的预处理方法，它能够将图像缩放到一个预设尺寸，然后再进行处理。自适应图片缩放可以对原始图像填充最少的像素^[7]便可将其缩放至标准尺寸，从而减少冗余信息，有利于提高网络推理速度。

YOLOv5 主要网络结构图如图 3.2 所示：yolov5 整体流程框架图如图 3.3 所示。

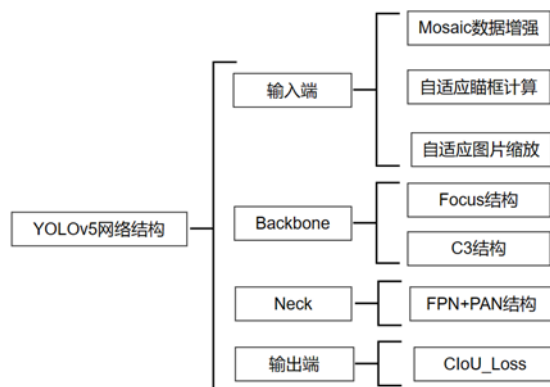


图 3.2 YOLOv5 网络结构图

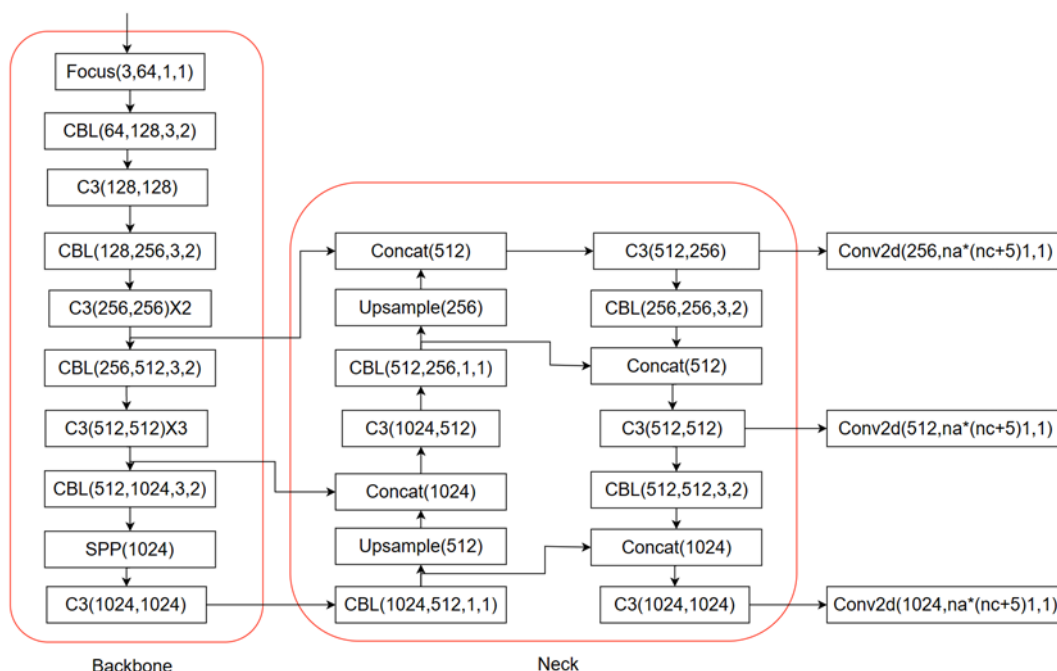


图 3.3 yolov5 整体流程框架图

在 yolov5 整体流程框架图中，对应参数分别代表：（In_channel,out_channel,kernal_size,stride）、(In_channel,out_channel)、(out_channel)。下面将对 YOLOv5 所涉及到的模型进行详细介绍。

（1）Focus

下采样就是一种图像缩小的手段，用来降低特征的维度并保留有效信息。可以将 Focus 模块视为一种特殊的下采样方式。Focus 的具体实现见图 3.4。该模块会在一张图片中每隔一个像素取一个值，从而得到四张子图像。YOLOv5 使用一个 Focus 模块替代了 YOLOv3 里的前三层，对模型起到了加速的效果。

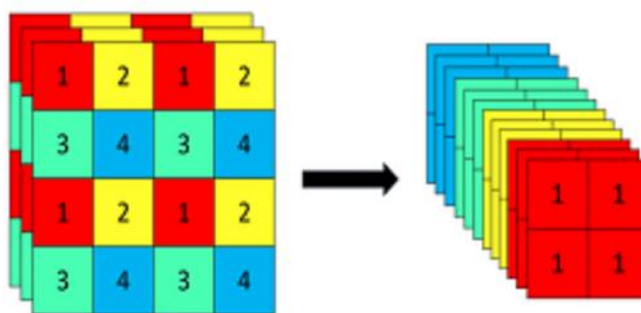


图 3.4 Focus 的切片操作

（2）CBL

YOLOv5 中的 CBL 即为 Conv+BN+active。此处的 Conv 是一个二维卷积。BN（Batch Normalization）也叫批量标准化，可以加快训练并维持其稳定，使得网络更快收敛。其主要原理在于对前一输出层的输出进行归一化，使得数据呈高斯分布。具体而言，该方法通过在 Batch 上减去该批量平均值再除以该批量标准差来实现归一化。CBL 使用的激活函数（Active）为 Leaky ReLU，公式如下：

$$\text{Leaky ReLU} = \begin{cases} x & x \geq 0 \\ p * x & x < 0 \end{cases} \quad (3-4)$$

其中，p 值一般取 0.1。

(3) C3

C3 模块由三个 Conv 模块和一个 Bottleneck 模块组成，C3 模块可以增加网络的深度和宽度，增加网络的非线性能力，以提高网络的检测精度。C3 模块结构图如图 3.5 所示。

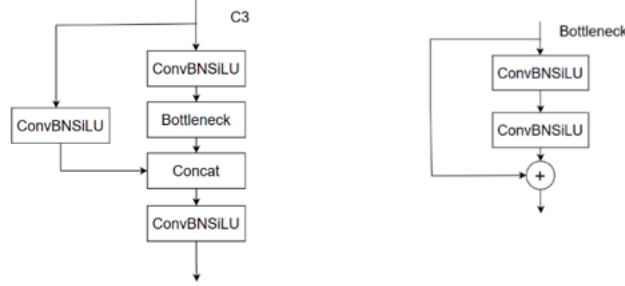


图 3.5 C3 模块结构图

ConvBNSiLU 模块即代表 Conv+ Batch Normalization+SiLU 的组合。其中，SiLU 激活函数如下式所示。

$$\text{SiLU} = x * \text{sigmoid}(x) = \frac{x}{1 + e^{-x}} \quad (3-5)$$

(4) SPP

SPP 结构又被称为空间金字塔池化，能将任意大小的特征图转成固定大小的特征向量，并获得不同尺度的特征融合信息^[8]。SPP 组成结构如图 3.6。其中 Maxpool 卷积核尺寸分别为 5×5 、 9×9 、 13×13 的规格。最后将四个感受野不同的图片拼接在一起。

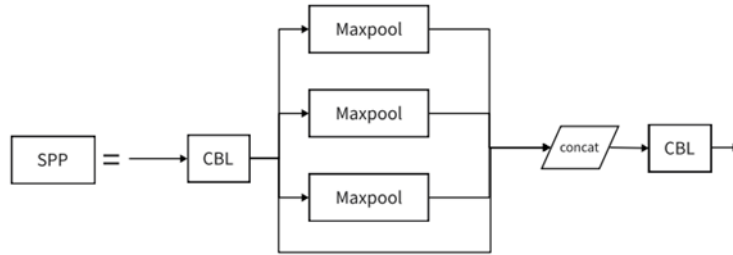


图 3.6 SPP 结构图

(5) FPN 和 PAN

在多目标跟踪的过程中，面临复杂环境中的多目标跟踪时，会面临许多外界因素的影响。同一帧中往往会出现不同大小的目标，如果对这些不同大小的目标使用相同的特征提取，较小的目标物体往往会失去更多的信息。为了解决这一问题，YOLOv5 采用 FPN（特征金字塔网络）和 PAN（路径聚合网络）的结构对特征进行多尺度融合。

FPN 结构通过从上到下进行上采样，将高层特征图的语义信息自上而下传递给底层特征图，PAN 结构通过从下到上进行下采样，将底层特征图的位置信息传递到高层特征图中。最后将不同特征图进行融合，保证了对不同尺度目标的检测，提高了模型的表达能力和目

标检测的性能。

FPN 与 PAN 结构如图 3.7 所示^[9], 为了便于理解, 将 YOLOv5 网络结构重画为图 3.8。

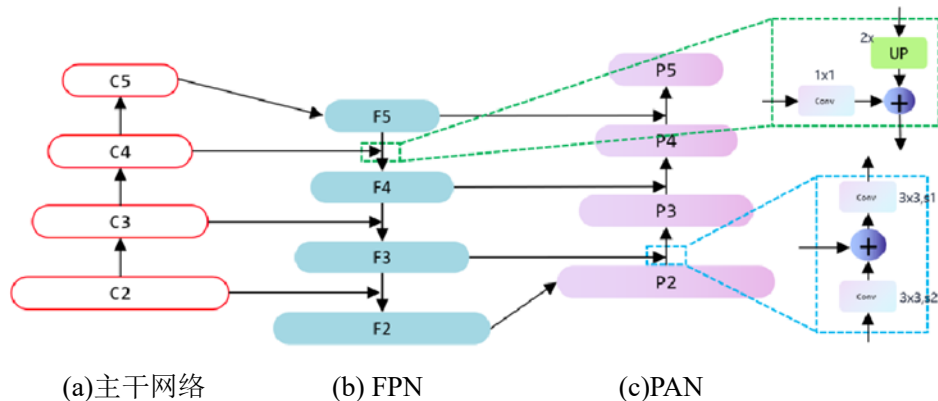


图 3.7 FPN 与 PAN 结构

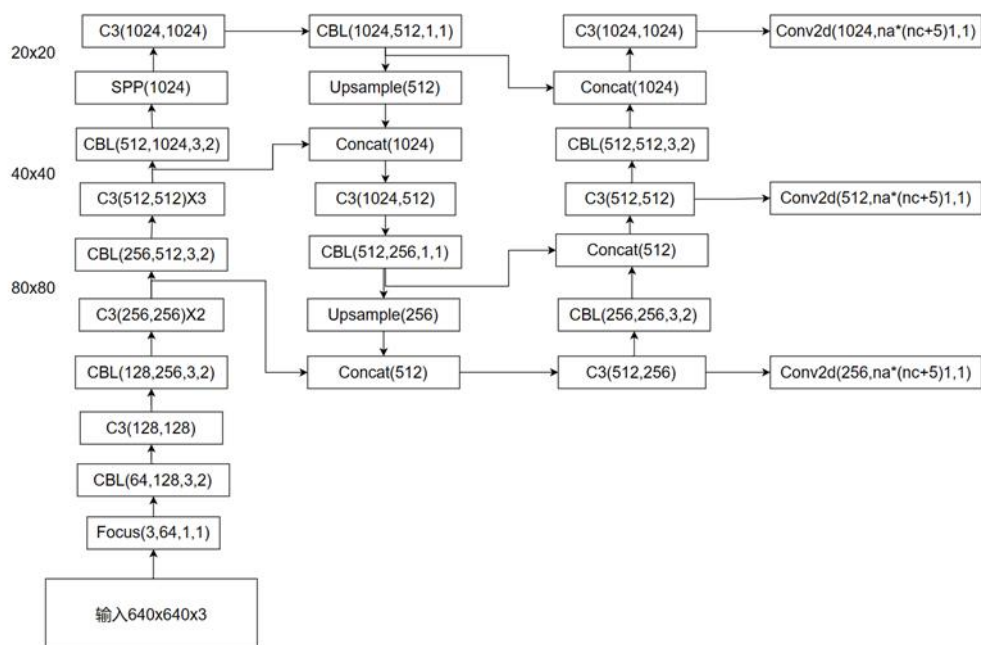


图 3.8 重画后的 YOLOv5 网络结构

可以很明显看到, 重新整理后的 YOLOv5 网络结构基本上与图 3.7 的结构是完全相同的。YOLOv5 正是采用了 PAN+FPN 结构, 保证了对不同大小目标的精确预测。

YOLOv5 的损失函数组成如图 3.9 所示。



图 3.9 YOLOv5 损失函数

其中，边界框损失涉及到预测框与目标框的重叠面积、中心点距离以及长宽比等信息，其使用 CIOU_Loss 进行计算，公式如下：

$$v = \frac{4}{\pi} \left(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h} \right)^2 \quad (3-6)$$

$$\alpha = \frac{v}{1 - IoU + v} \quad (3-7)$$

$$L_{CIOU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \quad (3-8)$$

相关参数见表 3-3。

表 3-3 边界框损失参数表

名称	代表意义
b	边界框 B 的中心点
b^{gt}	真实边界框 B^{gt} 的中心点
$\rho^2(b, b^{gt})$	b 与 b^{gt} 之间的欧氏距离
c	预测框与真实框最小外接矩形对角线的长度 ^[10]

对于置信度损失 L_{obj} 和分类损失 L_{cls} 的计算，涉及到 mask 掩码矩阵。以 $3*80*80$ 为例来解释 mask 矩阵的获得，首先将 $3*80*80$ 的 mask 的矩阵全部设置为 false。对于输入的每一张图像的每个目标框，进行以下判断，并根据判断结果将 mask 矩阵对应位置设置为 true：

- 将图像中所有目标框的中心坐标 (x, y) 、宽高 (w, h) 以及图像的宽高 (w_i, h_i) 解析出来，首先将 (x, y) 转换为 $640*640$ 的图像坐标 (x', y') ，并将 (w, h) 转换为 $640*640$ 图像中目标框的宽高 (w', h') 。
- 然后由 (x', y') 计算该目标框在 $80*80$ 网格中的坐标 (x_g, y_g) ，然后对 (x_g, y_g) 向下取整，得到网格 (x_0, y_0) 。为了加快收敛速度，对网格 (x_0, y_0) 的上下、左右再各取一个邻近的网格 (x_1, y_0) 和 (x_0, y_1) 。
- 认定目标框位于这三个格子附近，将 anchor0、anchor1、anchor2 分配给这三个格子，且这三个 anchor 框的宽和高分别为 (w_0, h_0) 、 (w_1, h_1) 、 (w_2, h_2) 。分别计算 (w', h') 与 (w_0, h_0) 、 (w_1, h_1) 、 (w_2, h_2) 的比例，剔除掉不满足要求的 anchor 框。将保留的 anchor 框标记为 true，剔除的 anchor 框标记为 false，则 anchor0、anchor1、anchor2 对应的标记为 (m_0, m_1, m_2) 。
- 根据 $80*80$ 网格中的 (x_0, y_0) 、 (x_1, y_0) 、 (x_0, y_1) 的位置，分别对 mask 矩阵赋值，即可得到该图像的 mask 矩阵。

$$\begin{cases} mask(0, x_0, y_0) = m_0 \\ mask(1, x_0, y_0) = m_1 \\ mask(2, x_0, y_0) = m_2 \end{cases} \quad (3-9)$$

$$\begin{cases} mask(0, x_1, y_0) = m0 \\ mask(1, x_1, y_0) = m1 \\ mask(2, x_1, y_0) = m2 \end{cases} \quad (3-10)$$

$$\begin{cases} mask(0, x_0, y_1) = m0 \\ mask(1, x_0, y_1) = m1 \\ mask(2, x_0, y_1) = m2 \end{cases} \quad (3-11)$$

对于置信度损失 L_{obj} 和分类损失 L_{cls} ，采用的是 BCEWithLogitsLoss 的计算方法，公式如下：

$$\sigma(a) = \frac{1}{1 + \exp(-a)} \quad (3-12)$$

$$Loss = -\frac{1}{n} \sum [y \cdot \log(\sigma(x)) + (1 - y) \log(1 - \sigma(x))] \quad (3-13)$$

YOLOv5 对 mask 为 true 的位置不直接赋值 1，而是计算对应预测框与目标框的 CIoU，使用 CIoU 作为预测框的置信度标签，而 CIoU 的值为-1.5~1，置信度标签的取值为 0~1，故而当 CIoU 的值小于 0 时，均取为 0。所以 80*80 网格的置信度损失值为：

$$I_{obj} = \frac{1}{num(mask = true)} \sum_{mask=true} Loss_{BCE}(z, x, y) \quad (3-14)$$

$$I_{noobj} = \frac{1}{num(mask = false)} \sum_{mask=false} Loss_{BCE}(z, x, y) \quad (3-15)$$

$$L_{obj} = a * I_{obj} + (1 - a)I_{noobj} \quad (3-16)$$

$$Loss_{BCE}(z, x, y) = -L(z, x, y) * \log P(z, x, y) \quad (3-17)$$

$$-(1 - L(z, x, y) * \log(1 - P(z, x, y))) \quad (3-18)$$

$$\text{其中, } \begin{cases} 0 \leq z < 3 \\ 0 \leq x < 80 \\ 0 \leq y < 80 \end{cases}$$

如果类别总数 t 为 80，那么分类概率矩阵为 3*80*80*80，则分类损失值为：

$$L_{cls} = \frac{1}{80 * num(mask = true)} \sum_{mask=true} Loss_{BCE}(z, x, y, t) \quad (3-19)$$

$$Loss_{BCE}(z, x, y, t) = -L(z, x, y, t) * \log P(z, x, y, t) \quad (3-20)$$

$$-(1 - L(z, x, y, t) * \log(1 - P(z, x, y, t))) \quad (3-21)$$

在目标检测算法中，由于同一目标的附近区域往往会产生多个重合度较高的预测框，为了减少计算量并筛选出高质量的检测结果，常常需要使用一些后处理方法。非极大值抑制^[11]是一种常用的后处理方法，其大致流程见图 3.10。它能够有效地剔除掉冗余的预测框，以减少后续计算量。

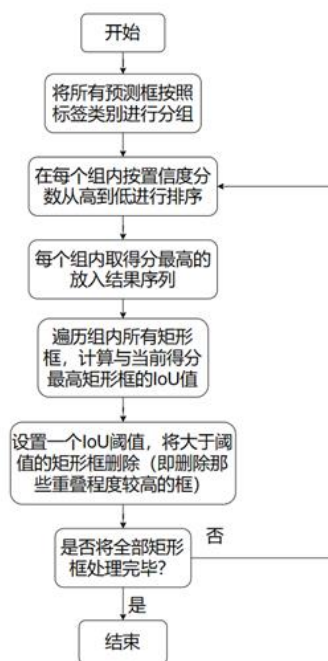


图 3.10 NMS 流程图

3.1.5 目标跟踪 DeepSORT 算法

目标跟踪是计算机视觉领域中备受关注的研究方向之一。目标跟踪技术的发展对于提高计算机视觉系统的智能化水平和实现更加精准的目标识别和定位具有重要的意义。目标跟踪最常用到的两个算法就是卡尔曼滤波和匈牙利匹配。在本章节将对卡尔曼滤波、匈牙利匹配、IoU 匹配以及 SORT 算法的流程进行详细介绍，本文后续所使用的 DeepSORT 跟踪算法就是在这些方法的基础上建立起来的。

1. 卡尔曼滤波：

卡尔曼滤波是一种常用的状态估计方法^[12]，在实际应用中，由于各种因素的影响，观测数据往往存在不确定性和噪声，这会导致对系统状态的估计产生误差。卡尔曼滤波通过对系统状态和观测数据进行建模，对系统状态进行递推估计和更新，从而实现对系统状态的精确估计和预测。在目标跟踪、通讯、制导系统等领域都有它的身影。

卡尔曼滤波的系统状态方程如下所示：

$$x_k = Ax_{k-1} + Bu_k + w_k \quad (3-22)$$

$$z_k = Hx_k + v_k \quad (3-23)$$

其中，系统状态方程各参数代表意义表 3.4 所示。

表 3.4 系统状态方程个参数意义	
名称	代表意义
x_k	系统在 k 时刻的状态矩阵
x_{k-1}	系统在 k-1 时刻的状态矩阵
z_k	k 时刻的测量值
A	状态转移矩阵

B	控制矩阵
H	状态观测矩阵
u_k	k 时刻的输入
w_k	满足高斯分布的过程噪声
v_k	满足高斯分布的测量噪声

其中， w_k 、 v_k 的均值为 0，其高斯分布满足如下关系：

$$w_k \sim N(0, Q) \quad (3-24)$$

$$v_k \sim N(0, R) \quad (3-25)$$

卡尔曼滤波的迭代过程主要分为两个部分，即预测和更新。

(1) 预测

一般状态估计方程如下所示：

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1} + B_k u_k \quad (3-26)$$

B_k 是作用在控制器向量 u_k 上的输入控制模型，一般运动学中没有这一项，因为对于检测的目标是无法测量其内部的控制量，所以可以简化为 0，如式 3.6 所示：

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1} \quad (3-27)$$

预测出 k 时刻的先验估计值 $\hat{x}_{k|k-1}$ 后，基于 $k-1$ 时刻的误差协方差矩阵 $P_{k-1|k-1}$ ，对 k 时刻的协方差 $P_{k|k-1}$ 进行预测，即为 $\hat{x}_{k|k-1}$ 向前推算的误差协方差矩阵。预测估计协方差的方程为

$$P_{k|k-1} = F P_{k-1|k-1} F^T + Q_k \quad (3-28)$$

预测过程各参数代表意义如下表所示。

表 3.5 预测过程各参数意义

名称	代表意义
$\hat{x}_{k k-1}$	k-1 时刻对 k 时刻的状态预测
$\hat{x}_{k-1 k-1}$	k-1 时刻的状态估计
A	状态转移矩阵
$P_{k-1 k-1}$	k-1 时刻的后验估计协方差矩阵
$P_{k k-1}$	k-1 时刻到 k 时刻的误差估计协方差矩阵
Q_k	过程噪声协方差矩阵

Q_k 用于描述系统模型中的噪声和不确定性。在卡尔曼滤波的状态估计过程中， Q_k 的大小直接影响到对当前状态估计的信任程度。当 Q_k 较小时，表示系统模型中的噪声和不确定性较小，当前状态估计的可靠性较高；反之，当 Q_k 较大时，表示系统模型中的噪声和不确定性较大，当前状态估计的可靠性较低。

(2) 更新

计算测量值 z_k 与先验状态 $\hat{x}_{k|k-1}$ 之间的测量残差 $\tilde{y}_k$ ，也称为新息(innovation)，其中， H 用于真实状态空间到观测空间的转换：

$$\tilde{y}_k = z_k - H\hat{x}_{k|k-1} \quad (3-29)$$

得到 $\tilde{y}_k$ 后，需要计算新息协方差(innovation covariance) S_k ，用于估计对当前测量值的信

任程度，越小则当前测量值越可靠；其中 R_k 为测量噪声协方差矩阵：

$$S_k = HP_{k|k-1}H^T + R_k \quad (3-30)$$

计算卡尔曼增益 K_k ，用于估计对新息 $\tilde{y}_k$ 的信任程度：

$$K_k = P_{k|k-1}H^TS_k^{-1} \quad (3-31)$$

更新 k 时刻状态的后验估计值 $\hat{x}_{k|k}$ ，用于下一次递归的计算：

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k\tilde{y}_k \quad (3-31)$$

更新 k 时刻后验状态 $\hat{x}_{k|k}$ 的误差协方差矩阵 $P_{k|k}$ ，用于下一次递归的计算：

$$P_{k|k} = (I - K_kH)P_{k|k-1} \quad (3-32)$$

卡尔曼滤波的大致流程如下图所示。

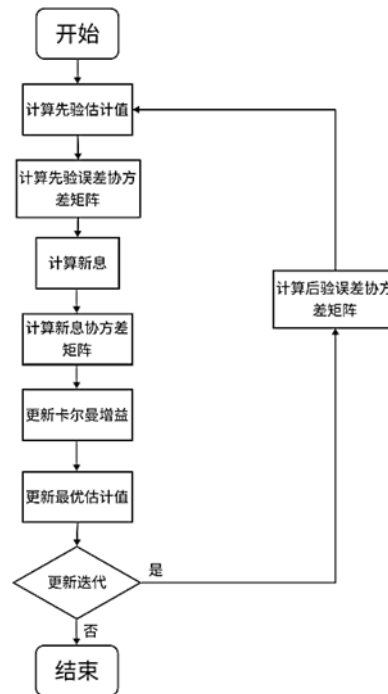


图 3.11 卡尔曼滤波的迭代流程图

在目标跟踪中，需要对目标的状态进行估计和预测。目标状态通常包括以下两个方面：

- 均值：包含被检测目标的位置以及速度信息，由向量 $(x, y, \gamma, h, \dot{x}, \dot{y}, \dot{\gamma}, \dot{h})$ 表示， (x, y) 为边框中心点的坐标， γ 为边框的宽高比，边框的高度为 h 。 $\dot{x}, \dot{y}, \dot{\gamma}, \dot{h}$ 表示 x, y, γ, h 的速度，其初始值为0。
- 协方差：描述了系统状态和观测数据之间的关系，它可以用来衡量系统状态的精度和稳定性。由 8×8 的对角矩阵表示，其中每个对角元素表示对应状态的方差，即该状态的估计值与真实值之间的差异程度。矩阵中的数字越大，则表明对应状态的不确定性越大。

卡尔曼滤波在目标跟踪过程中的步骤如下：

(1) 首先利用 $k-1$ 时刻的后验估计值, 通过状态转移矩阵 F 的变换, 得到当前时刻 k 的先验估计状态, 即

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1} \quad (3-33)$$

其中, 状态转移矩阵 F 为

$$\begin{pmatrix} x \\ y \\ \gamma \\ h \\ \dot{x} \\ \dot{y} \\ \dot{\gamma} \\ \dot{h} \end{pmatrix}_{\hat{x}_{k|k-1}} = \begin{pmatrix} 1 & 0 & 0 & 0 & dt & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & dt & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & dt & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & dt \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ \gamma \\ h \\ \dot{x} \\ \dot{y} \\ \dot{\gamma} \\ \dot{h} \end{pmatrix}_{\hat{x}_{k-1|k-1}} \quad (3-34)$$

F

(2) 使用 $k-1$ 时刻的后验估计协方差来计算当前 k 时刻的先验估计协方差

$$P_{k|k-1} = FP_{k-1|k-1}F^T + Q \quad (3-35)$$

(3) 利用先验估计方差矩阵 $P_{k|k-1}$ 和观测矩阵 H 以及测量噪声状态方差矩阵 R 计算出 k 时刻卡尔曼增益矩阵 K_k

$$K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1} \quad (3-36)$$

(4) 将卡尔曼滤波器的先验估计值 $\hat{x}_{k|k-1}$ 通过观测矩阵 H 投影到测量空间, 并计算出与测量值 z_k 的残差 $\tilde{y}_k$

$$\tilde{y}_k = z_k - H\hat{x}_{k|k-1} \quad (3-37)$$

$$\tilde{y}_k = \begin{pmatrix} x \\ y \\ \gamma \\ h \end{pmatrix} - \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ \gamma \\ h \\ \dot{x} \\ \dot{y} \\ \dot{\gamma} \\ \dot{h} \end{pmatrix} \quad (3-38)$$

(5) 将卡尔曼滤波器的测量值和预测值按照卡尔曼增益的比例相融合, 得到后验估计值 $\hat{x}_{k|k}$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k\tilde{y}_k \quad (3-39)$$

(6) 计算出卡尔曼滤波增益的后验估计协方差矩阵 $P_{k|k}$, 以便下一次递归的计算

$$P_{k|k} = (I - K_kH)P_{k|k-1} \quad (3-40)$$

2. 匈牙利算法:

多目标跟踪问题通常涉及到对多个目标在时间序列中的位置和运动状态进行跟踪和预测。为了实现对多个目标的跟踪和预测, 需要将目标与其对应的观测数据进行匹配。匈牙利算法可以通过对目标和观测数据之间的关系进行建模和分析, 实现对多目标跟踪问题的高效解决。匈牙利算法的数学理论为:

$$\min z = \sum_{i=1}^n \sum_{j=1}^m c_{ij} x_{ij} \quad (3-41)$$

$n > m$ 时:

$$s. t. \quad \sum_{i=1}^n x_{ij} = 1 (j = 1, 2, \dots, m) \quad (3-42)$$

$$\sum_{j=1}^m x_{ij} \leq 1 (i = 1, 2, \dots, n) \quad (3-43)$$

$$x_{ij} = 1 \text{ or } x_{ij} = 0 \quad (3-44)$$

$n < m$ 时:

$$s. t. \quad \sum_{i=1}^n x_{ij} \leq 1 (j = 1, 2, \dots, m) \quad (3-45)$$

$$\sum_{j=1}^m x_{ij} = 1 (i = 1, 2, \dots, n) \quad (3-46)$$

$$x_{ij} = 1 \text{ or } x_{ij} = 0 \quad (3-47)$$

其中，各参数的详细代表意义如表 3.6 所示。

表 3.6 公式各参数意义

名称	代表意义
x_{ij}	x_{ij} 为二值函数， $x_{ij} = 1$ 表示第 i 个目标和第 j 个轨迹完成匹配
c_{ij}	c_{ij} 组成了 $n \times m$ 权重矩阵 C ， c_{ij} 表示第 i 个目标和第 j 个轨迹之间的关联度量值
$\sum_{i=1}^n x_{ij} (j = 1, 2, \dots, m)$	$\sum_{i=1}^n x_{ij} = 1 (j = 1, 2, \dots, m)$ 表示每个目标都能与轨迹完成匹配， $\sum_{i=1}^n x_{ij} \leq 1 (j = 1, 2, \dots, m)$ 表示轨迹未必与目标相匹配
$\sum_{j=1}^m x_{ij} (i = 1, 2, \dots, n)$	$\sum_{j=1}^m x_{ij} \leq 1 (i = 1, 2, \dots, n)$ 表示目标未必匹配轨迹

匈牙利算法流程图如图 3.12 所示。

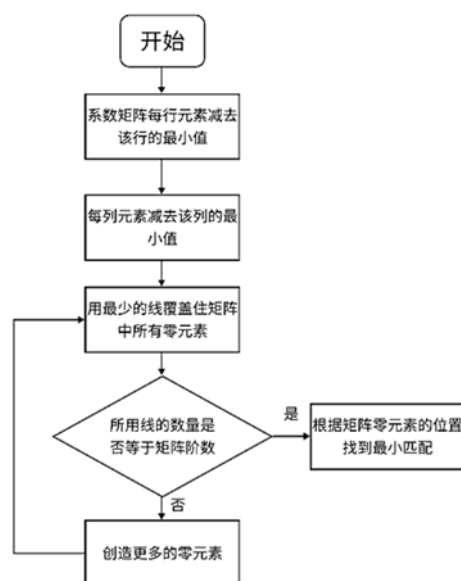


图 3.12 匈牙利算法流程图

使用匈牙利算法求解多目标跟踪的检测框与轨迹的匹配问题时，首先需要构建代价矩阵 $cost(i, j)$ ，代价矩阵描述了匹配两个集合中的某两个元素所需要花费的代价，其中 i 代表前一帧中的第 i 个目标边界框， j 代表当前帧的第 j 个目标边界框， $cost(i, j)$ 处的值代表两个边界框的 IoU。匈牙利算法的具体步骤如下所示：

(1) 将代价矩阵 $cost(i, j)$ （此处假设矩阵规格为 $n \times n$ ）的每一行，都减去该行中的最小值；其次，对于代价矩阵的每一列，都减去该列中的最小值。

(2) 用最少的水平线和垂直线覆盖掉矩阵所有的零元素。如果所使用的水平线和垂直线之和恰好为 n ，则说明找到了最优解，否则进入到第(3)步。

(3) 从第(2)步未被覆盖的元素中找到最小值，然后把剩余非零元素都减去该最小值，给水平线和垂直线交叉点处的0元素加上该最小值。重复步骤(2)(3)，直至划线数为 n 。

此处以 3×3 的网络来简要说明匈牙利算法的匹配过程。假设 obj_1、obj_2、obj_3 与 track_1、track_2、track_3 的代价矩阵如图 3.13 所示。

	track_1	track_2	track_3
obj_1	1	2	3
obj_2	2	4	6
obj_3	3	6	9

图 3.13 初始矩阵

执行匈牙利算法的第一步，每行的处理结果如图 3.14 (1)，每列的处理结果如图 3.14 (2)。

	track_1	track_2	track_3
obj_1	0	1	2
obj_2	0	2	4
obj_3	0	3	6

(1)

	track_1	track_2	track_3
obj_1	0	0	0
obj_2	0	1	2
obj_3	0	2	4

(2)

图 3.14 进行第一步操作后的结果

用最少的水平线和垂直线覆盖掉矩阵所有的零元素，结果如图 3.15 所示。

	track_1	track_2	track_3
obj_1	0	0	0
obj_2	0	1	2
obj_3	0	2	4

图 3.15 用水平线和垂直线覆盖零元素

此时仅仅使用两条直线就将矩阵中的所有零元素覆盖，小于 3。此时矩阵中除 0 以外的最小元素为 1，然后把剩余非 0 元素都减去 1，直线交叉点处加上 1。结果如图 3.16 所示。

	track_1	track_2	track_3
obj_1	1	0	0
obj_2	0	0	1
obj_3	0	1	3

图 3.16 最终结果

根据图 3.16 可以很明显看出，此时需要三条直线才能将矩阵中的所有零元素都覆盖掉，然后进行匹配。很明显，匹配的结果为：obj_3 与 track_1 进行匹配，obj_2 与 track_2 进行匹配，obj_1 与 track_3 进行匹配。将上述过程推广至多维矩阵，即为匈牙利匹配在多目标跟踪中的应用。

3. IOU 匹配:

IOU 匹配算法是目标跟踪中常用的一种匹配算法。该算法通过计算目标框之间的重叠度来判断是否为同一目标，从而实现目标的跟踪。对于每个检测到的目标框，算法都会来计算其与所有跟踪到的目标框之间的 IOU 值。

SORT 算法中，当某个检测框与某个已跟踪的目标框的 IOU 值超过一定阈值时，将它们匹配为同一目标。如果某个检测框与所有已跟踪的目标框的 IOU 值都小于阈值，则将其视为新进入跟踪场景的目标。IoU 匹配的流程如图 3.17 所示。

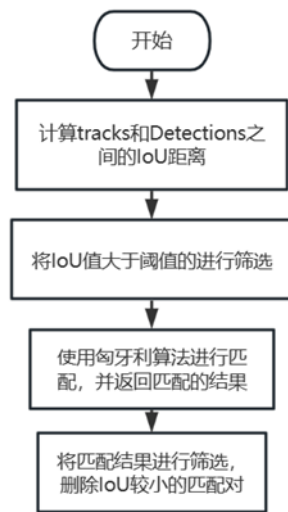


图 3.17 IoU 匹配流程图

4. SORT 算法流程:

SORT (Simple Online and Realtime Tracking) 是一个非常简单、高效的多目标跟踪算法^[13], 具有非常广泛的应用。SORT 算法的核心可以分为两个部分: 卡尔曼滤波和匈牙利算法。算法的大致流程如图 3.18 所示。

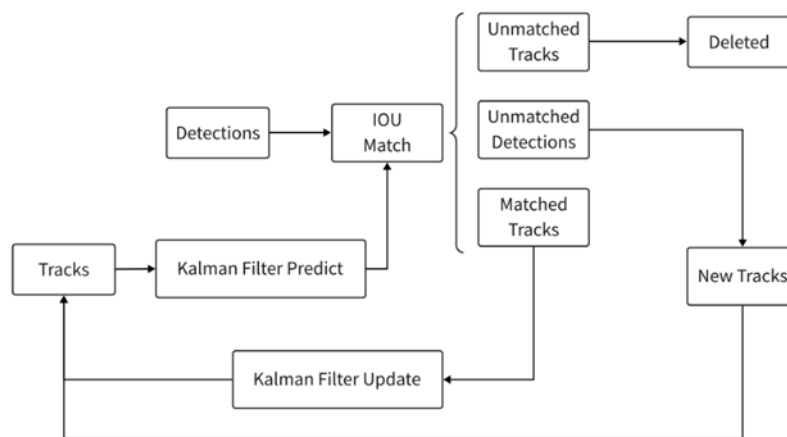


图 3.18 SORT 算法流程图

Detection 为通过目标检测器得到的检测框信息, Tracks 则是轨迹信息。在 IoU 匹配后的结果主要有三种状态:

- Unmatched Tracks, 这部分被认为是 Detection 和 Tracks 匹配失败, 如果连续 n 次都匹配不成功, 则将该目标 ID 轨迹删除。
- Unmatched Detections, 这部分说明没有任意一个 Tracks 可以与之进行匹配, 此 Detection 将被认为是新出现的目标, 为其分配一个新的 Track。
- Matched Track 说明 Detection 和 Tracks 匹配成功。

其大致工作流程为: 根据历史轨迹 Tracks 进行卡尔曼滤波预测其下一帧目标框的位置, 将预测出来的目标框与检测框进行 IoU 匹配, 匹配的结果有三种: Unmatched Tracks、

Unmatched Detections、Matched Track。此时要判断 Unmatched Tracks 是否已经超过最大的失配次数 $\max_age$ ，如果超过的话就将其删除，没有超过的话就将该轨迹的失配此处+1。对于 Unmatched Detections，将其视为新出现的目标，并为其分配一个新的 Track。对于 Matched Track，使用卡尔曼滤波器更新其参数。最后使用卡尔曼滤波器对 Track 进行更新，以实现对下一帧的目标预测，依此循环。

3.1.6 基于 YOLOv5 和 DeepSort 的多目标跟踪

在 SORT 算法的基础上，对其升级版——DeepSORT 算法进行深入介绍，将详细探讨 DeepSORT 算法中使用到的新算法和概念，例如马氏距离、余弦距离、级联匹配和重识别网络等。接着，在第二章所讲述的 YOLO 算法以及第三章讲述的卡尔曼滤波、匈牙利算法和 IoU 匹配的基础上，对目标检测算法 YOLOv5 与目标跟踪算法 DeepSORT 进行整合，并详细介绍和分析整个多目标跟踪流程。最后，本文将根据现有模型的缺点和不足，提出相应的改进方法，以期进一步提升多目标跟踪的性能。

1. 马氏距离、余弦距离：

马氏距离和余弦距离是常用的相似度度量方法，它们被广泛应用于计算相似度矩阵，以得到代价矩阵。马氏距离是一种用于衡量两个随机向量之间距离的指标，它考虑了向量之间的相关性和协方差矩阵，可以用来度量两个随机向量之间的相似度。其公式如下所示。

$$d^{(1)}(i, j) = (d_j - y_i)^T S_i^{-1} (d_j - y_i)$$

马氏距离的阈值函数为：

$$b_{i,j}^{(1)} = I(d^{(1)}(i, j) \leq t^{(1)})$$

其中，马氏距离各参数所代表的意义见表 3.7。

表 3.7 马氏距离各参数意义

参数	代表意义
d_j	第 j 个检测框的位置
y_i	第 i 个追踪器对目标的预测位置
S_i	d 和 y 之间的协方差矩阵
$b_{i,j}^{(1)}$	当马氏距离小于 $t^{(1)}$ 时，代表成功匹配，取值为 1；否则取 0
$t^{(1)}$	阈值，一般设置为 9.4877

马氏距离对于运动不确定性较低时表现出良好的性能，通常表现为物体运动的非常规律，且无遮挡环境。而实际场景中经常伴随目标的快速移动，将导致预测失误，同时马氏距离对于发生遮挡时表现较差，因此必须引入外观信息。外观信息通常由最小余弦距离表示，如下式所示：

$$d^{(2)}(i, j) = \min\{1 - r_j^T r_k^{(i)} | r_k^{(i)} \in R_i\} \quad (3-48)$$

余弦距离的阈值函数如下式所示：

$$b_{i,j}^{(2)} = I(d^{(2)}(i, j) \leq t^{(2)}) \quad (3-49)$$

其中，余弦距离各参数所代表的意义见表 3.8。

表 3.8 余弦距离各参数意义

参数	代表意义
r_j	第 j 个检测框的外观特征
$r_k^{(i)}$	第 k 条轨迹的第 i 个外观特征
$r_j^T r_k^{(i)}$	余弦相似度
$1 - r_j^T r_k^{(i)}$	余弦距离
$b_{i,j}^{(2)}$	同上，当余弦距离小于 $t^{(2)}$ 则代表匹配上

综上所述，DeepSORT 在数据关联的过程中有余弦距离和马氏距离两个代价。将马氏距离和余弦距离结合起来，建立综合模型用于衡量最终的匹配情况，将马氏距离和余弦距离加权如下：

$$c_{i,j} = \alpha d^{(1)}(i,j) + (1 - \alpha) d^{(2)}(i,j) \quad (3-50)$$

如果跟踪轨迹的预测框与真实的检测框满足以上两个的阈值条件，那么可以将其视为可接受的正确关联。

$$b_{i,j} = \prod_{m=1}^2 b_{i,j}^m \quad (3-51)$$

2. 级联匹配：

当目标经历长时间的遮挡，卡尔曼滤波器预测的位置由于失去了有效的更新导致位置预测不确定性大大提高。为了解决此类问题，引入了级联匹配。通过设置不同的优先级，使得真实目标的检测框强制性优先匹配那些更加频繁出现的轨迹^{[14][14]}，让遮挡时间短的轨迹优先与检测框进行匹配。其匹配流程如下：

(1) 在卡尔曼滤波的结果中，将确定态的轨迹集合记为 T，将目标检测的结果记为 D。

使用公式 $c_{i,j} = \alpha d^{(1)}(i,j) + (1 - \alpha) d^{(2)}(i,j)$ 计算成本矩阵 C。 $b_{i,j} = \prod_{m=1}^2 b_{i,j}^m$ 计算阈值矩阵 B。

(2) 将确定状态的轨迹与检测进行匹配，如果该轨迹与该检测框代价矩阵最小，并且在阈值范围内，表示二者匹配成功。

(3) 更新没有匹配的轨迹与检测，当达到最大循环次数，结束循环，返回没有匹配的轨迹与检测。

级联匹配的流程图如图 3.19 所示。

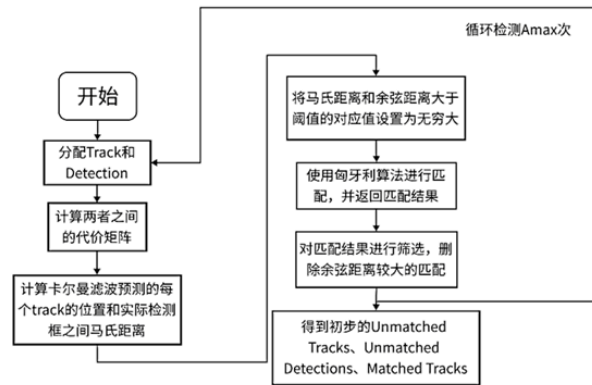


图 3.19 级联匹配的流程图

3. 重识别网络:

DeepSORT 算法之所以可以避免大量的 ID switch, 是因为 DeepSORT 算法中的特征提取网络可以将目标检测框内的特征提取出来并保存一定的帧数。在目标发生遮挡又重新出现后, 利用前后的特征对比就可以将遮挡后又重新出现的目标和遮挡之前的目标重新找到, 减少了在发生遮挡后丢失 ID 的可能。DeepSORT 中用于特征提取的 CNN 网络结构如下表。

表 3.9 CNN 网络结构

Name	Patch Size/Stride	Output Size
Conv1	3x3/1	32x128x64
Conv2	3x3/1	32x128x64
Max Pool 3	3x3/2	32x64x32
Residual 4	3x3/1	32x64x32
Residual 5	3x3/1	32x64x32
Residual 6	3x3/2	64x32x16
Residual 7	3x3/1	64x32x16
Residual 8	3x3/2	128x16x8
Residual 9	3x3/1	128x16x8
Dense 10		128
Batch and L ₂ normalization		128

Conv1、Conv2 是卷积层, 用来提取图像的特征。Patch Size 为卷积核的大小, Stride 为卷积步长。Max Pool 是最大值的池化操作, 其作用是下采样, 通过降低图像分辨率来剔除一些对于识别物体不必要的冗余信息。残差 (Residual) 可以防止网络层数过多造成的梯度消失和梯度爆炸。Dense 为全连接层, 通常被用于卷积神经网络(CNN)的尾部去重新拟合特征信息。全连接层能够无损的传递特征信息, 但是网络参数多, 计算量大, 训练较慢。Batch and L₂ normalization就是对每一批数据进行归一化, 在一定程度上缓解了深层网络中的梯度弥散问题。

4. 多目标跟踪模型建立:

以前文所述 YOLOv5、DeepSORT 为基础, 本文建立的多目标跟踪模型大致流程如图 3.20 所示。

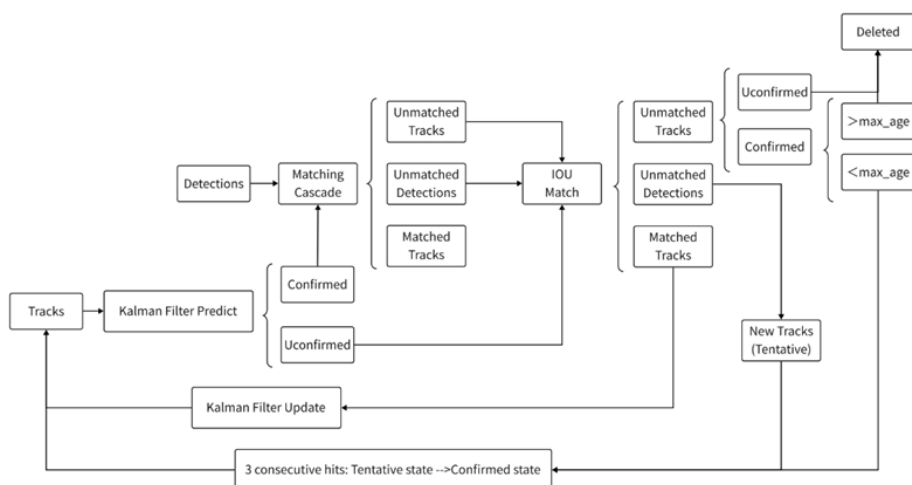


图 3.20 多目标跟踪流程图

整个算法的详细流程如下：

(1) 根据第一帧的检测结果创建对应的 Tracks。将卡尔曼滤波的运动变量初始化，通过卡尔曼滤波对下一帧的预测框进行预测。此时轨迹 Tracks 的状态一定是 Unconfirmed。

(2) 通过将第二帧得到的检测框与上一帧（此处为第一帧）通过卡尔曼滤波得到的预测框进行 IoU 匹配，并根据匹配的结果计算其代价矩阵，计算公式为 $1-\text{IoU}$ 。

(3) 将 (2) 中得到的代价矩阵作为匈牙利算法的输入，得到检测与轨迹匹配的结果。此时可以得到三种状态：Unmatched Tracks、Unmatched Detections 以及 Matched Tracks。对于 Unmatched Tracks，要判断此时 Tracks 的状态是 Confirmed 还是 Unconfirmed。每一条轨迹都会设置一个初始值为 0 的计时器，如果是 Confirmed 状态，则当 Tracks 没有与检测框成功匹配时，将其值加一，匹配成功时，将其值重置为 0。如果该计数值大于 max_age（最大保存次数），则将该轨迹删除；如果该 Tracks 的状态为 Unconfirmed，则直接将其删除。如果是 Unmatched Detections，则说明图像中检测到了新的目标，给它分配一条新的 Track，将其认定为 Unconfirmed。如果该 Unconfirmed 的轨迹连续三次都匹配成功，则将其状态转为 Confirmed。

(4) 反复循环 (2) (3) 步，直至出现 Confirmed 的轨迹。

(5) 通过卡尔曼滤波预测 Tracks 下一帧的边界框，将确认态的边界框与 YOLOv5 的检测结果进行级联匹配。只要 Tracks 和 Detections 匹配成功，就会默认保存前 100 帧的 Detections 的外观特征和运动信息。

(6) 进行级联匹配后有三种结果：Unmatched Tracks、Unmatched Detections、Matched Tracks。将 Unmatched Tracks、Unmatched Detections 再进行一次 IoU 匹配。通过 IoU 匹配的结果计算代价矩阵，计算公式为 $1-\text{IoU}$ 。

(7) 将 (6) 中得到的代价矩阵作为匈牙利算法的输入，得到检测与轨迹匹配的结果。根据不同的结果状态进行不同的操作。

(8) 循环 (5)、(6)、(7) 步骤，直到视频帧结束。

5. 模型的改进:

(1) 注意力机制 SE 模块的引入

本研究基于原有的 YOLO 网络进行训练，但发现精确率和召回率等评价指标表现不佳。为此，本文引入了注意力机制 SE 模块，以提高目标检测的准确性和效率。引入 SE 模块之后可以使模型更加关注对目标检测任务更为关键的信息^[15]，如目标的形状、大小、位

置等，从而减少模型对无关信息的关注，以提高检测的准确性。SE 模块的大致流程见图 3.21。具体流程可以大为以下三步：

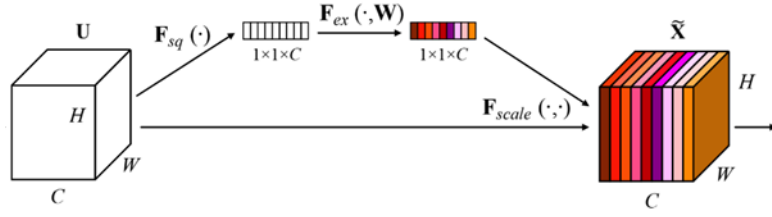


图 3.21 注意力机制 SE 模块

第一步：挤压（Squeeze）。首先进行全局平均池化操作，将输入特征图转为一个 $1 \times 1 \times C$ 的张量，这样每个通道就可以被一个数值所表示。这个数值表示该通道的重要性，即该通道对于目标分类的贡献程度。Squeeze 操作可以用以下公式表示：

$$z_i = F_{sq}(x_i) = \frac{1}{H \times W} \sum_{j=1}^H \sum_{k=1}^W x_{i,j,k} \quad (3-52)$$

第二步：激励（Excitation）。这一步通过两个全连接层完成，通过权重 w 得到所需要的权重信息，其中 w 是通过学习得到的。公式如下：

$$s_i = F_{ex}(z, W) = \sigma(g(z, W)) = \sigma(W_2 \delta(W_1 z)) \quad (3-53)$$

第三步：将生成的特征向量 $s(1 \times 1 \times C)$ 与特征图 $U(H \times W \times C)$ 对应通道相乘，得到最终的结果。

上述内容所涉及到的各参数所代表的意义见下表。

表 3.10 重识别各参数意义

参数	代表意义
$x_{i,j,k}$	输入特征图的第 i 个通道、第 j 行、第 k 列的值
H	特征图的高度
W	特征图的宽度
$F_{sq}(x_i)$	对 x_i 进行 Squeeze 操作
z_i	第 i 个通道的压缩结果
W_1, W_2	两个全连接层的权重矩阵，全连接层 W_1 的维度是 $C \times \frac{C}{\tau}$ ， W_2 的维度是 $\frac{C}{\tau} \times C$
	τ 表示一个压缩的比例
δ	ReLU 激活函数
σ	Sigmoid 激活函数
s_i	第 i 个通道的权重

(2) BottleneckCSP 模块的引入

本研究在原有的 YOLO 网络结构的基础上进行训练，但发现出现了过拟合的现象。具体表现为模型在训练集上的效果很好，但在验证集上的效果较差，甚至出现了随着 epoch 的迭代，损失函数上升的情况，产生了网络的退化问题。本文借鉴了残差连接的思想来解决这一问题。残差连接是一种用于解决深度神经网络中梯度消失、网络退化和过拟合等问题的技术。它的基本思想是将网络的输出表示为输入和输入的一个非线性变换之和，从而

使得网络可以更好地学习输入数据的特征。通过残差连接，网络可以更深更复杂，同时也可以更加稳定和可靠。

本研究将 YOLO 原 C3 模块替换为 BottleneckCSP 模块，该模块使用了残差连接的思想。BottleneckCSP 模块结构如图 3.22 所示。

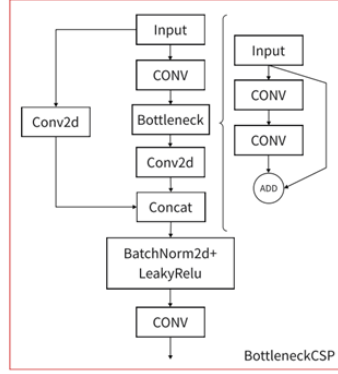


图 3.22 BottleneckCSP 模块结构图

其中 CONV 模块的结构如图 3.23 所示。

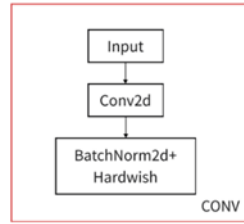


图 3.23 CONV 模块结构图

其中，hardwish 激活函数如下式所示。

$$\text{hardwish} = \begin{cases} 0 & x \leq -3 \\ x & x \geq 3 \\ \frac{x(x+3)}{6} & \text{其它} \end{cases} \quad (3-54)$$

（3）改进后的模型结构

改进后的模型结构如图 3.24 所示。在通道数较多的部分，添加了注意力机制 SE 模块，以便更好地发挥其作用。此外，将所有的 C3 模块替换为了 BottleneckCSP 模块，以进一步提高模型的性能。

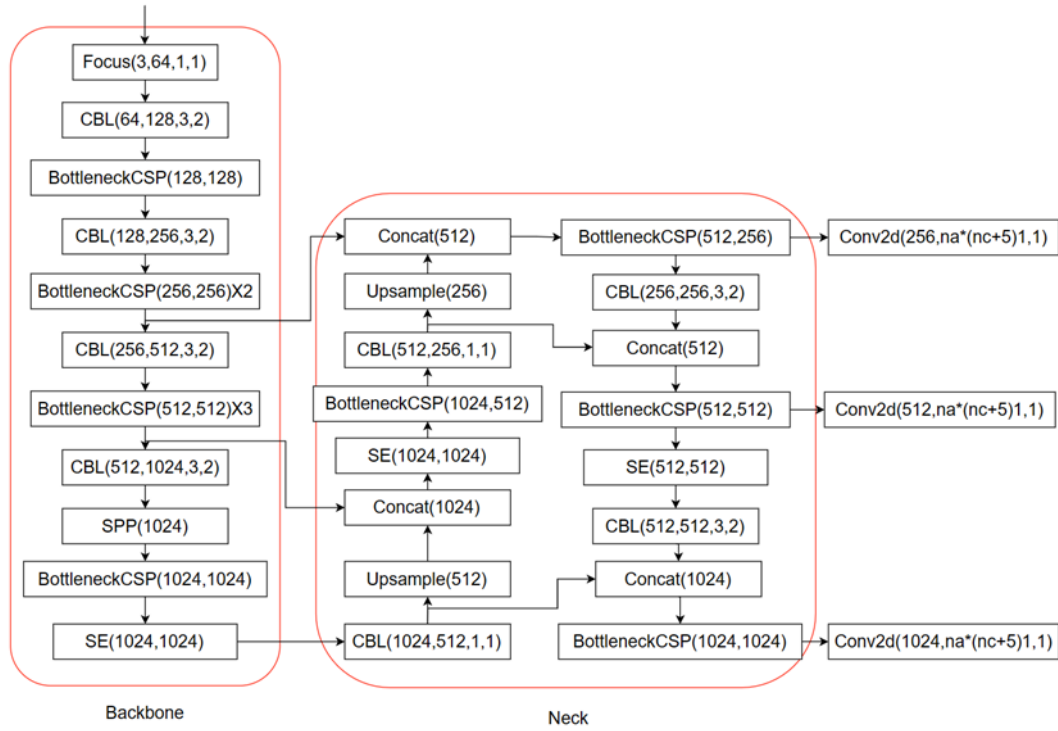


图 3.24 改进后的模型结构

3.1.7 交通流速度与流量的计算

1. 交通流速度计算方法:

本题的流量计算采用 YOLO-DeepSORT 进行目标检测后对每个车辆进行检测和跟踪，最后对每一个不同的车辆赋值一个独特的 ID，最后在每个时间节点统计 ID 的大小便可以检测车辆在规定时间内通过的数量即车流量，具体效果见下图所示：

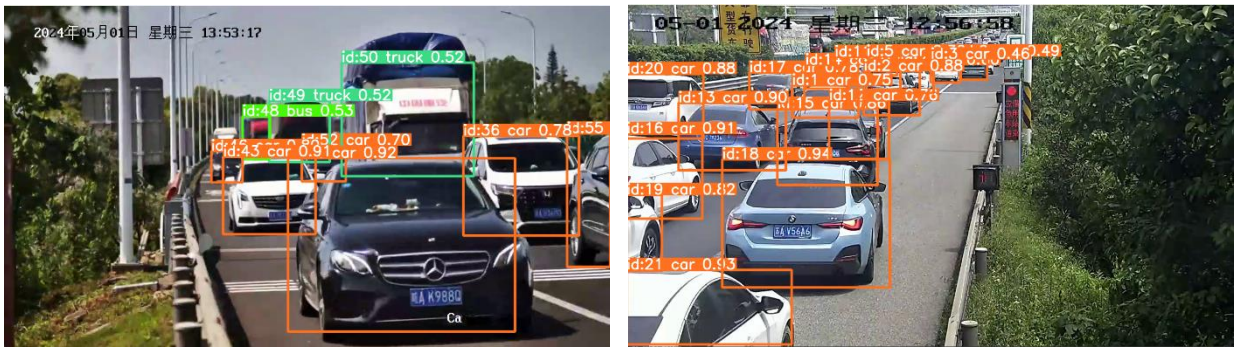


图 3.25 不同角度的车流量监控

2. 交通流速度计算方法:

在对车辆的实际速度进行计算时，利用 DeepSORT 算法对每一辆被跟踪的车辆在连续的两帧图像中提取具有相同 ID 的车辆中心点坐标，记为 (x_1, y_1) 和 (x_2, y_2) 。然后利用欧氏距离公式计算车辆在像素坐标系下的移动距离 d_p ：

$$d_p = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (3-55)$$

由于摄像机的视角、焦距和安装高度等因素，像素坐标系下的距离并不能直接反映车辆在实际物理空间中的移动距离。因此，为了将像素距离 d_p 转换为真实距离 d_r ，需要建立一个固定的映射关系，即确定像素距离与实际物理距离之间的比例系数 k 。

为了求解比例系数 k ，我们在监控区域内设置了一条基准线。当车辆经过这条基准线时，对其速度进行统计和分析。假设在基准线附近，实际距离与像素距离的比值为 k 。由于在基准线附近，距离摄像机相同的物体，其像素变化与实际距离变化之间的比例关系是稳定的，因此可以认为 k 在基准线附近是一个恒定值。

3. 确定比例系数 k

为了确定比例系数 k ，我们利用已知的最高车速进行反推。在高速公路上，车辆的最高限速通常为 120 km/h 。考虑到可能存在的超速情况，我们取 130 km/h 作为最高速度 $v_{\max}$ 。当车辆以最高速度经过基准线时，我们可以记录其在像素坐标系下越过基准线前后两帧的中心点坐标，计算得到像素速度 v_p ：

$$v_p = \frac{d_p}{\Delta t} \quad (3-56)$$

其中， Δt 为相邻两帧之间的时间间隔。根据监控视频的帧率和加速处理信息，我们计算出 $\Delta t = 0.4$ 秒。

利用实际最高速度 $v_{\max}$ 和像素速度 v_p ，我们可以求解比例系数 k ：

$$k = \frac{v_{\max}}{v_p} \quad (3-57)$$

4. 计算实际移动距离和速度

在获得比例系数 k 后，对于后续每一辆经过基准线的车辆，我们可以计算其实际移动距离 d_r ：

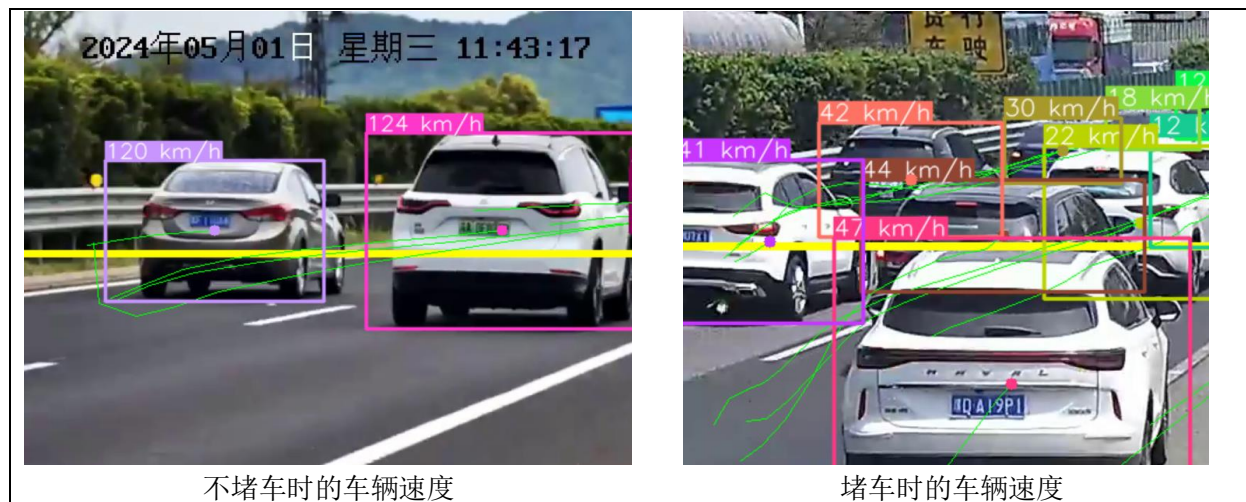
$$d_r = k \times d_p \quad (3-58)$$

然后，结合相邻两帧的时间间隔 Δt ，计算车辆的实际速度 v ：

$$v = \frac{d_r}{\Delta t} \quad (3-59)$$

我们在原目标跟踪算法的基础上引入了计算速度的功能，具体效果见下表所示：

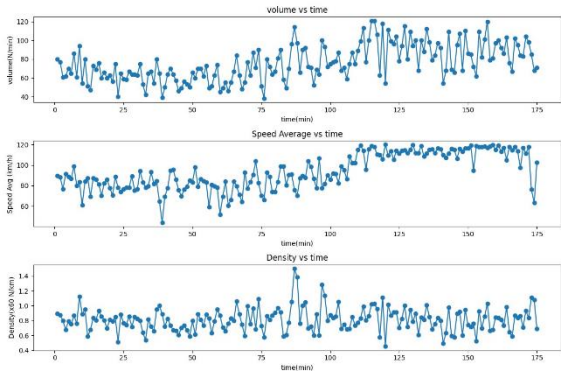
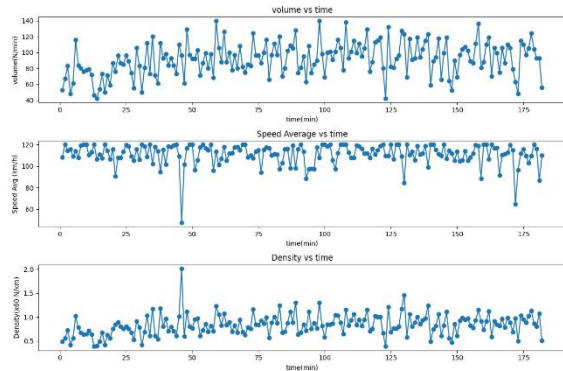
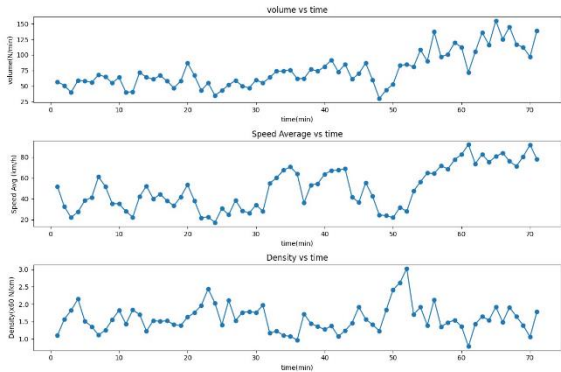
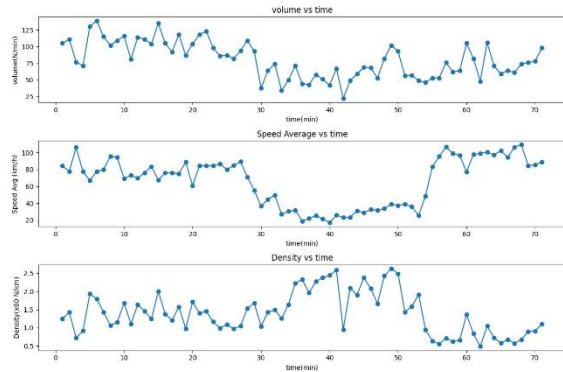
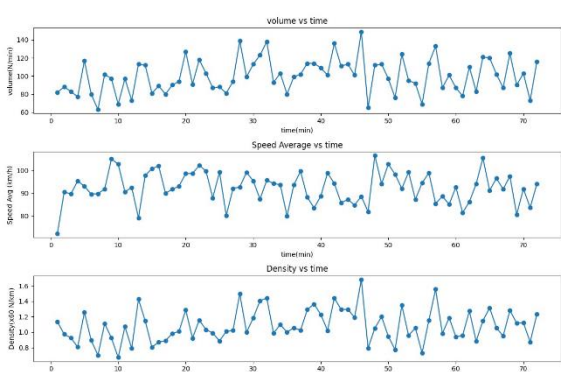
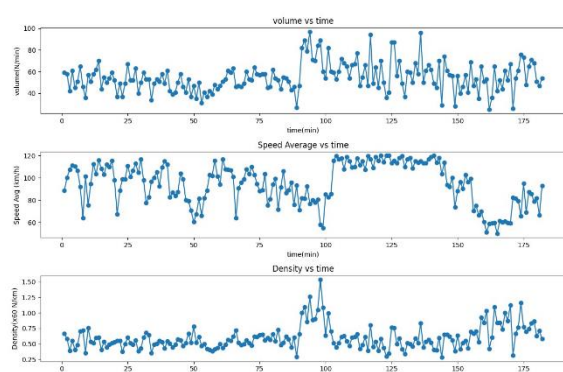
表 3.11 车辆速度检测示例

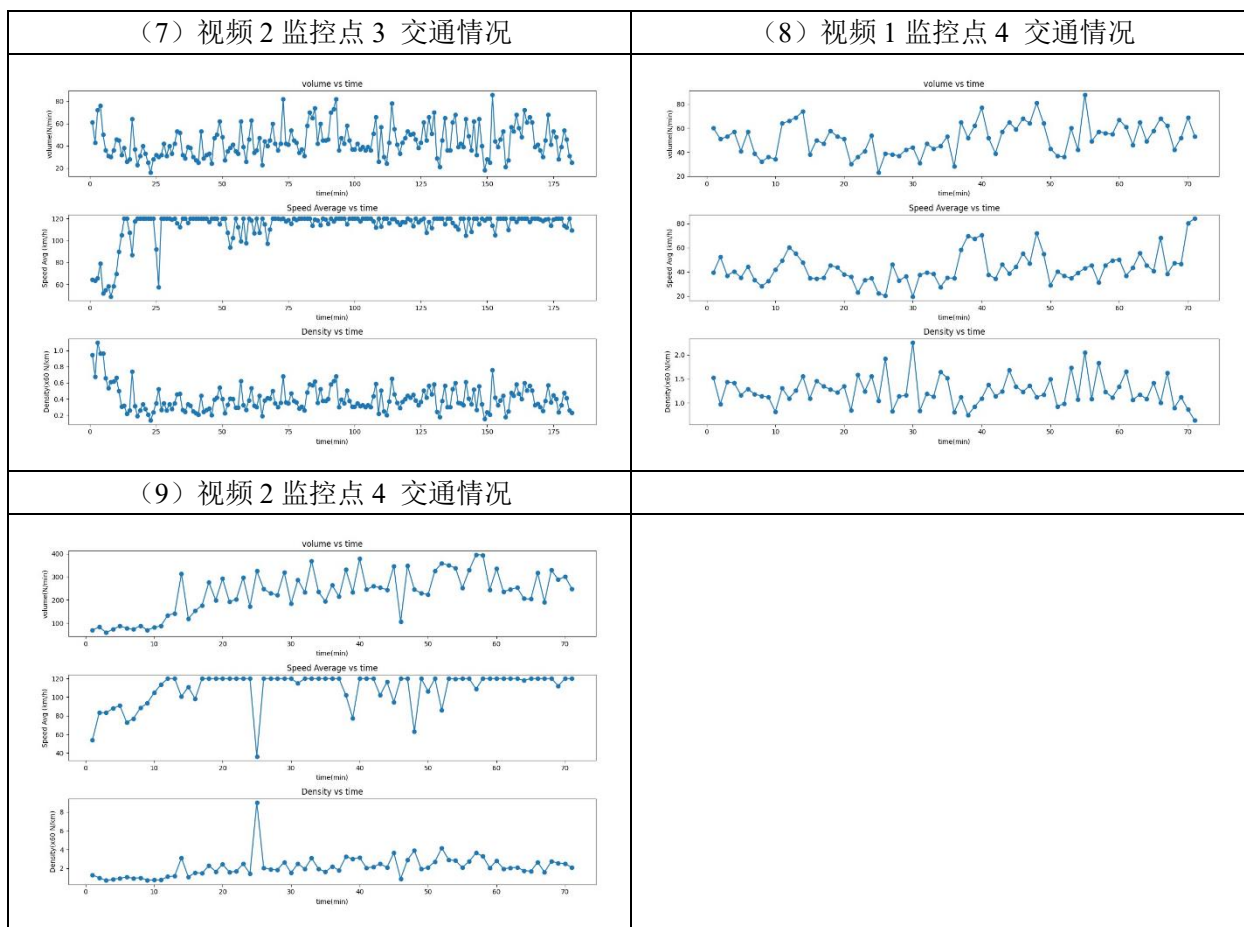


3.1.8 实验数据与结果分析

通过上述算法构建目标跟踪模型，对每个视频数据中的车辆进行跟踪测速，并对每分钟时间段的交通流量进行统计，最后计算出每分钟的交通密度 k 。绘制的交通量(N/min)、平均车速(km/h)和交通密度($\times 60$ N/km)随时间的变化情况如下表所示：

表 3.12 交通量、平均车速和交通密度随时间变化情况

(1) 视频 1 监控点 1 交通情况	(2) 视频 2 监控点 1 交通情况
 Figure 1: Traffic data for Video 1, Point 1. The top graph shows volume (N/min) fluctuating between 40 and 120 over 175 minutes. The middle graph shows speed (km/h) fluctuating between 60 and 120. The bottom graph shows density (x60 N/km) fluctuating between 0.4 and 1.4.	 Figure 2: Traffic data for Video 2, Point 1. The top graph shows volume (N/min) fluctuating between 40 and 140 over 175 minutes. The middle graph shows speed (km/h) fluctuating between 60 and 120. The bottom graph shows density (x60 N/km) fluctuating between 0.5 and 2.0.
(3) 视频 1 监控点 2 交通情况	(4) 视频 2 监控点 2 交通情况
 Figure 3: Traffic data for Video 1, Point 2. The top graph shows volume (N/min) fluctuating between 25 and 150 over 75 minutes. The middle graph shows speed (km/h) fluctuating between 20 and 80. The bottom graph shows density (x60 N/km) fluctuating between 1.0 and 3.0.	 Figure 4: Traffic data for Video 2, Point 2. The top graph shows volume (N/min) fluctuating between 25 and 125 over 75 minutes. The middle graph shows speed (km/h) fluctuating between 20 and 100. The bottom graph shows density (x60 N/km) fluctuating between 0.5 and 2.5.
(5) 视频 3 监控点 2 交通情况	(6) 视频 1 监控点 3 交通情况
 Figure 5: Traffic data for Video 3, Point 2. The top graph shows volume (N/min) fluctuating between 60 and 140 over 75 minutes. The middle graph shows speed (km/h) fluctuating between 40 and 100. The bottom graph shows density (x60 N/km) fluctuating between 0.8 and 1.6.	 Figure 6: Traffic data for Video 1, Point 3. The top graph shows volume (N/min) fluctuating between 40 and 100 over 175 minutes. The middle graph shows speed (km/h) fluctuating between 60 and 120. The bottom graph shows density (x60 N/km) fluctuating between 0.25 and 1.50.



从图中可以看出（以第一幅图为例）：

（1）Volume vs time：交通量与时间的关系

0-50 分钟：交通量波动较大，波动范围在 40 至 100 辆之间，总体上没有明显的增长或下降趋势。

50-100 分钟：交通量呈上升趋势，波动也相对较大，但车辆数量逐渐增加，从 60-80 辆增加到接近 120 辆。

100-175 分钟：在接近 100 分钟之后，交通量进入一个较为稳定的区间，波动较小，保持在 80 至 120 辆之间。

（2）Speed Average vs time 平均车速与时间的关系

0-50 分钟：车速波动较大，范围在 60 至 100 公里/小时之间，存在一些明显的速度低谷，最低点接近 60 公里/小时。

50-100 分钟：平均车速逐渐上升，尤其在 80 分钟左右达到峰值，接近 120 公里/小时。

100-175 分钟：车速维持在较高水平，约在 100 至 120 公里/小时，波动较小，直到最后几分钟车速出现较大下滑。

（3）Density vs time 交通密度与时间的关系

0-50 分钟：交通密度波动较大，在 0.4 至 1.4 之间剧烈变化，表明车辆分布在该时段有较大的波动，可能存在拥堵和通畅交替的现象。

50-100 分钟：密度有所下降，并趋于稳定，波动较小，范围在 0.6 至 1.0 之间，意味着车流变得较为均匀，拥堵状况有所缓解。

100-175 分钟：交通密度较低，持续在 0.8 左右小幅波动，但最后几分钟有较大波动，密度稍有上升。

整体来看：

交通流量：整体上，交通量从 50 分钟之后逐渐上升，并在 100 分钟左右趋于稳定，说明此时的车流量达到了一个较高水平，可能处于一个较为繁忙的时段。

平均车速：随着交通量的增加，车速在前半段有所波动，50 分钟之后平均车速逐渐提高，达到了接近 120 公里/小时的峰值，表明此时交通条件较好，车辆能够较快通行。

交通密度：密度在前期波动较大，表明交通流不稳定，拥堵和畅通交替出现。50 分钟后，密度趋于稳定，表明道路上的车流分布较为均匀，交通情况有所改善。

同时我们还对交通流量和交通流速度进行了离散点的绘图，其示意图如下所示：

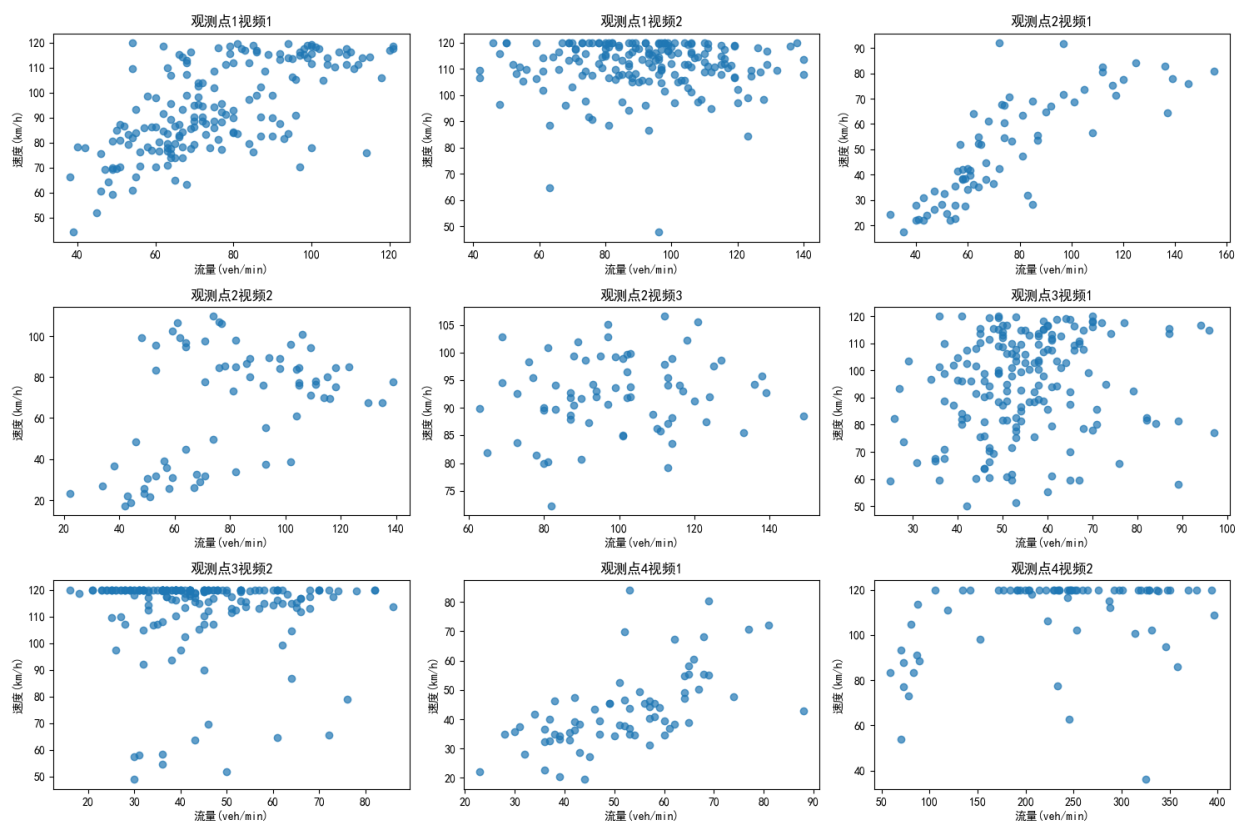


图 3.26 交通流速度与流量的散点图

对交通流速度与流量的散点图进行分析我们可以得知在不同地点和视频下，速度与流量的关系表现出多样性：正相关（流量增加，速度增加）通常表明道路通畅，拥堵较少；负相关（流量增加，速度下降）则是典型的交通拥堵现象；而无明显关系或分散数据则反映了交通流的波动性较大，表明该区域的交通情况较为复杂。整体来看，不同地点的交通状况可能受到多种因素的影响，如道路条件、交通管控等，需结合具体的交通模型进行深入分析。

3.2 子问题（2）

3.2.1 问题分析

本题需要建立交通流拥堵模型：基于四个观测点的交通流参数和道路条件，构建一个能够预测交通拥堵的模型，尤其要预测在第三个观测点到第四个观测点之间的路段可能出

现持续拥堵的情况。为此本体提出了如下的解决方案：

(1). 首先为了量化拥堵情况，本文在参考文献中的拥堵指数计算公式的基础上推导出了一个有关交通流量、交通流速度和交通流密度的公式。并且为了消除交通拥堵异常值、方便划分交通拥堵等级，我们提出了 Sigmoid-交通拥堵指数，将交通拥堵指数限制在 0 到 1 内，标准化交通拥堵指数。同时利用交通拥堵等级的对交通拥堵指数进行了划分。

(2). 在本题若要预测未来的拥堵状况，则需要对未来的交通流量和交通流速度，进行预测。由于注意到交通流量和交通流速度是时序信息，具有时序依赖性因此本体又建立了，因此本文选择了具有较高时序特性的双向门控循环单元（Bi-GRU），同时为了捕获全局特征性，获取更多时间序列信息，本文引入了 Attention 机制，并且将原本模型中的卷积神经网络(CNN)换成了时间卷积网络(TCN),构建了 TCNAt-Bi-GRU 模型对未来的交通流量和交通流速度进行预测。

(3). 最后我们对预测结果进行了实时预警建模，采用激波计算的方法，计算拥堵波从节点四传播到节点三的时间，通过标准拥堵指数和激波进行节点三和节点四之间路段的实时预警。最后通过与真实拥堵状况的比较验证了我们模型预警的有效性。

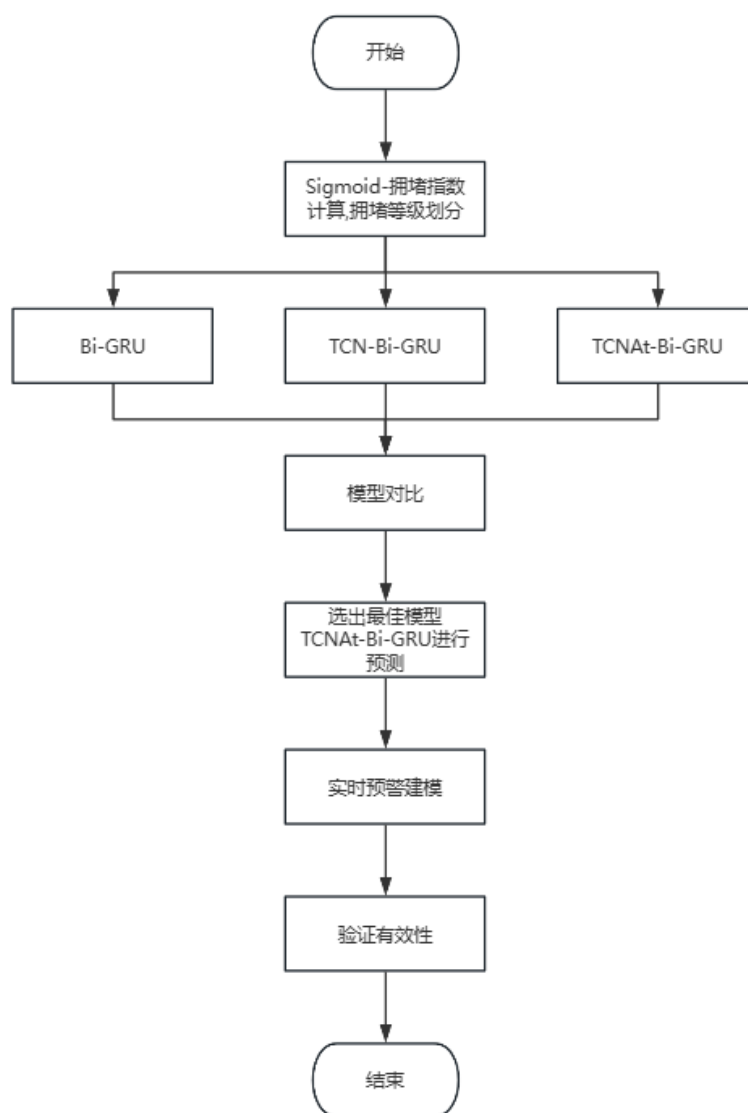


图 3.27 思路流程图

3.2.2 交通拥堵阈值的计算与确定

1. Sigmoid-交通拥堵指数:

为了充分利用流量、速度和密度与拥堵之间的关系,通过流量、速度和密度建模了一个拥堵指数全面衡量这三者与交通拥堵之间的关系。

由参考文献可以得知拥堵指数的一般公式为:

$$\gamma^p(t) = \frac{occ^p(t)^2 \times 1000}{Q^p(t) \times v^p(t) + 0.00001} \quad (3-60)$$

$occ^p(t)^2$ 代表道路的占有率,而在此次建模中我们没有用到占有率这一指标,而是用密度代替占有率这一指标,同时车流密度和占有率都反应了道路拥挤状况,典型的经验公式为:

$$occ = \frac{L_v \cdot k}{100} \propto k \quad (3-61)$$

其中 L_v 代表车辆的长度,因此我们在此处定义的交通拥堵指数的公式为:

$$\gamma^p(t) = \frac{k^p(t)^2 \times N}{Q^p(t) \times v^p(t) + 0.00001} \quad (3-62)$$

N 为常数。

在使用交通拥堵指数时,我们发现交通拥堵指数的值不能够很有效的限制在一定范围内(可能因为异常值而存在指数非常大的情况),为了更好的反映交通拥堵指数,并且将交通拥堵指数标准化在一定范围内我们引入了 **sigmoid 函数**,经验证, **sigmoid 函数**能够有效的将交通拥堵指数限定到 0-1 内,并且越趋近于 1 表示交通越拥堵, **sigmoid-交通拥堵指数**公式可以写为:

$$\gamma_{norm}^p(t) = \left(\frac{1}{1 + e^{-\gamma^p(t)}} \right) \times 2 - 1 \quad (3-63)$$

2. 常数 N 值的选定:

要将公式中的拥堵因子 **sigmoid 函数**归一化到 0-1 区间,并使数据在归一化后分布较为均匀,关键是确定上式中的常数 N ,使得拥堵因子在归一化后不会集中在 0 或 1。通常步骤如下:

(1) 确定公式中的变量

$Q^p(t)$ 对应流量; $v^p(t)$ 对应速度; $k^p(t)$ 对应交通密度; 常数项为 0.00001。

(2) 计算未归一化的拥堵因子 $\gamma^p(t)$

计算出文件中的每一行的 $\gamma^p(t)$,然后再用 **sigmoid 函数**进行归一化。**Sigmoid 函数**公式为:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3-64)$$

这里 x 是计算出的 $\gamma^p(t)$;

(3) 调整 N 值:

我们希望 N 的取值可以使得 $\gamma^p(t)$ 的值在应用 **sigmoid 函数**后分布均匀。为此,我们可以通过实验不同的 N 值,观察归一化后的结果的分布情况。

接下来计算公式中的拥堵因子 $\gamma^p(t)$ ，并使用不同的 N 进行归一化，最终选择一个合适的 N 。

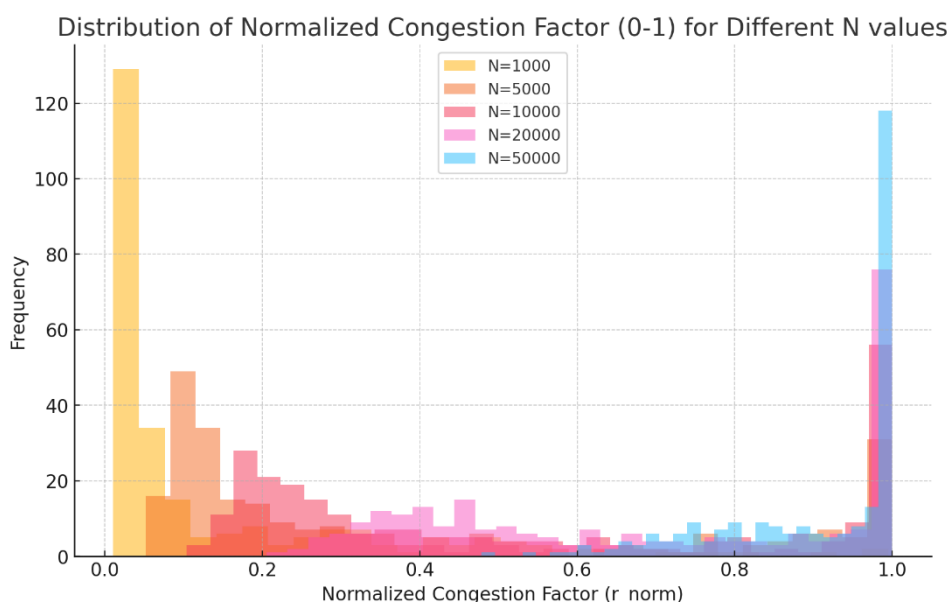


图 3.28 不同 N 值下归一化拥堵因子的分布

从上图可以看出，随着 N 的增加，拥堵因子 $\gamma^p(t)$ 归一化后的分布也随之变化：

当 N 较小时 (如 $N = 10$ 和 $N = 100$),归一化后的拥堵因子值大多集中在 0 附近，分布非常集中。

随着 N 增大(如 $N = 1000$ 到 $N = 50000$),拥堵因子的分布逐渐向两端 (0 和 1)扩展，分布范围更宽。为了避免归一化后的拥堵因子过于集中在 0 或 1，较合适的 N 值大致在 $N = 10000$ 到 $N = 20000$ 之间，这样的值可以让拥堵因子均匀分布在 0 和 1 之间，同时避免过度偏向某一端。

在计算拥堵指数时我们选择 $N = 10000$ 进行计算。

3. 交通拥堵等级的划分：

在我们已经得到归一化后的标准交通拥堵指数以后，我们将对标准交通拥堵指数进行分级，下表是分级后的标准交通拥堵指数：

表 3.13 拥堵等级的划分

拥堵指数	状态	行程时间	拥堵描述
0 ~ 0.2	畅通	T	基本无拥堵
0.2 ~ 0.4	基本畅通	$1.2T \sim 1.5T$	少数路段拥堵
0.4 ~ 0.6	轻度拥堵	$1.5T \sim 1.8T$	部分路段拥堵
0.6 ~ 0.8	中度拥堵	$1.8T \sim 2.1T$	大量路段拥堵
0.8 ~ 1	严重拥堵	$> 2.1T$	大部分道路拥堵

在确定交通拥堵指数分级以后，又根据子问题（1）求出的数据计算出了每个时刻的交通拥堵指数，每个视频中的交通拥堵指数随时间变化情况如下图所示：

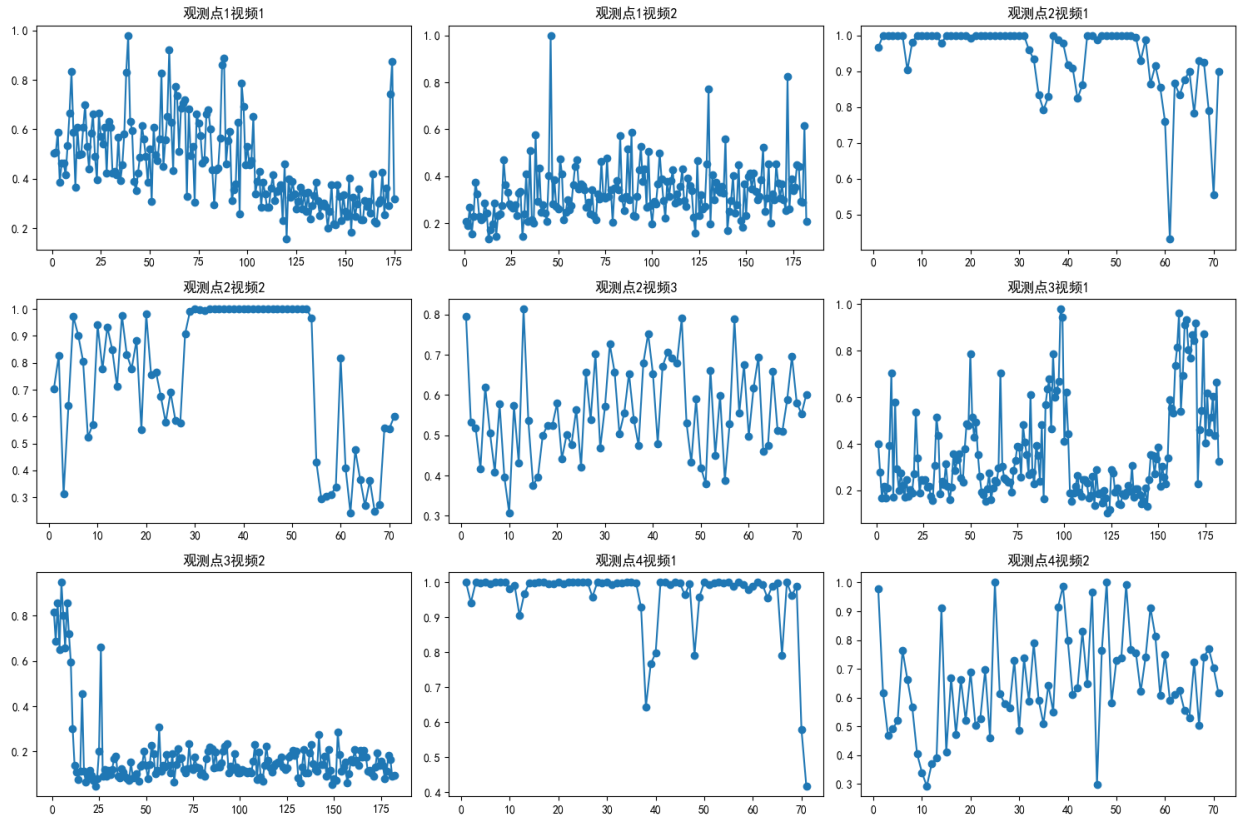


图 3.29 标准交通拥堵指数随时间变化情况

结果分析：

总体趋势：大多数视频显示标准交通拥堵指数随着时间推移有显著的波动，有些观察点在一定时段出现了严重的拥堵情况（如观察点 2 视频 1 和观察点 4 视频 1），而有些视频则显示出较多的变化和波动（如观察点 3 视频 3 和观察点 4 视频 2）。

不同观察点的特点：不同观察点的交通状况有较大差异，有些区域的交通长时间保持拥堵，而有些则表现出不稳定的波动。拥堵的起始时间和持续时间因观察点和视频而异。

3.2.3 时序预测模型结构

1. 注意力机制 SK-Net:

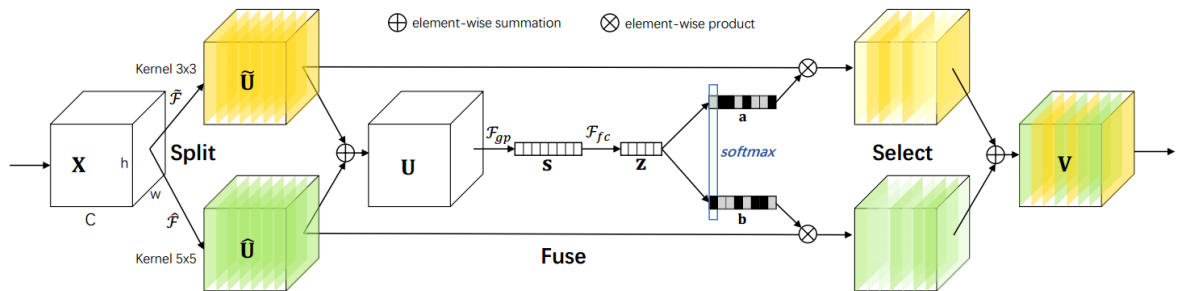


图 3.30 注意力机制 SK-Net 结构图

SK-Net 网络主要由三个部分组成：Split、Fuse、Select。其中，Split 部分将输入信息 X

分别输入不同的核大小；Fuse 部分进行特征融合；Select 部分根据计算得到的权重对对应的特征进行选择操作。

(1) Split 部分

在 Split 中，分别输入两个卷积层，两个卷积层分别为 3×3 和 5×5 。其中，每个卷积层都是由高效的分组/深度卷积、批处理归一化和 ReLU 函数依次组成的。最终得到的输出分别为 $\tilde{U}$ 、 $\hat{U}$ 。

(2) Fuse 部分

Fuse 部分的基本思想是使用门来控制来自多个分支的信息流，这些分支携带不同尺度的信息到下一层的神经元中。为实现这一目标，门需要整合来自所有分支的信息。首先通过 element-wise summation 融合来自多个分支的结果：

$$U = \tilde{U} + \hat{U} \quad (3-64)$$

F_{gp} 为全局平均池化操作， F_{fc} 为先降维再升维的两层全连接层。需要注意的是输出的两个矩阵 a 和 b ，其中矩阵 b 为冗余矩阵。然后通过简单地使用全局平均池化以生成 channel-wise 统计信息 $s \in R^C$ 来生成全局信息。具体来说， s 的第 C 个元素是通过空间尺寸 $h \times w$ 收缩 u 来计算的：

$$s_c = F_{gp}(U_c) = \frac{1}{h \times w} \sum_{i=1}^h \sum_{j=1}^w U_c(i, j) \quad (3-65)$$

此外，还创建了一个紧凑的特征 $z \in R^{d \times 1}$ ，以便为精确和自适应选择提供指导。这是通过一个简单的完全连接 (fc) 层实现的，降低了维度以提高效率：

$$z = F_{fc}(s) = \delta(B(Ws)) \quad (3-66)$$

(3) Select 部分

Select 操作使用 a 和 b 两个权重矩阵对 $\tilde{U}$ 和 $\hat{U}$ 进行加权操作，然后求和得到最终的输出向量 V 。

通过引入 SK-Net 这样的注意力机制，网络可以在处理复杂数据时更加高效地提取关键信息，从而提高模型的性能和泛化能力。此外，SK-Net 还可以有效应对输入数据中的噪声和干扰，提高模型的鲁棒性和稳定性。

2. BiGRU:

LSTM(长短期记忆网络) 是一种特殊的循环神经网络 (RNN)，能够学习长期依赖关系。标准的 LSTM 包含一个遗忘门、一个输入门和一个输出门。GRU(门控循环单元) 可以看作是 LSTM 的一个变体，它将 LSTM 中的三个门合并为两个门：更新门和重置门。

(1) GRU

GRU 网络内部包含两个门使用了更新门 (update gate) 与重置门 (reset gate)。重置门决定了如何将新的输入信息与前面的记忆相结合，更新门定义了前面记忆保存到当前时间步的量。这两个门控向量决定了哪些信息最终能作为门控循环单元的输出。这两个门控机制的特殊之处在于，它们能够保存长期序列中的信息，且不会随时间而清除或因为与预测不相关而移除。

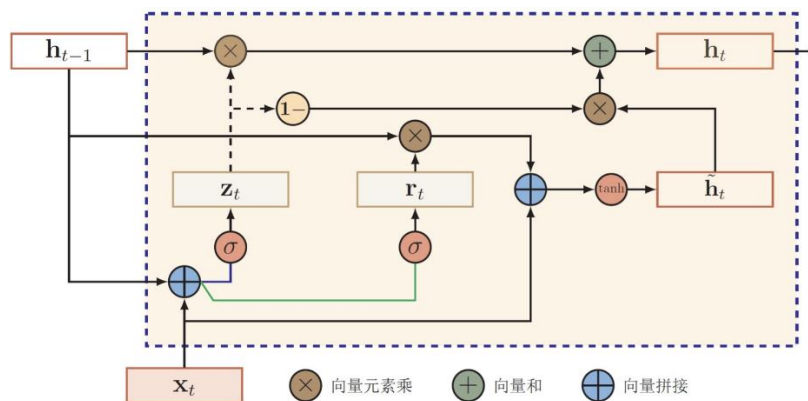


图 3.31 GRU 网络结构

(2) BiGRU

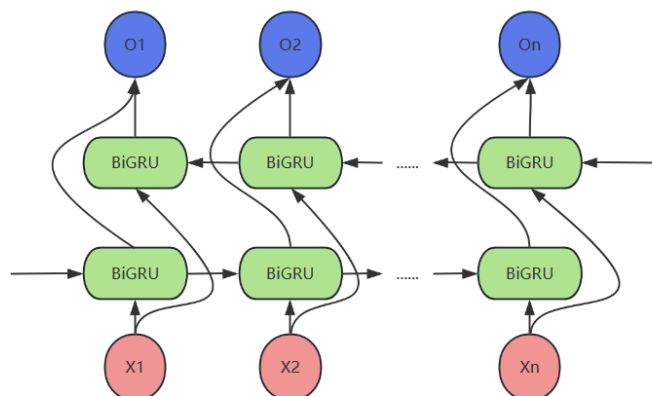


图 3.32 BiGRU 网络结构

在 BiGRU 中，对于每个时间点 t ，模型包括两个独立的 GRU：一个正向 GRU 用于处理序列从开始到当前时间点的信息，而另一个逆向 GRU 则处理从序列末尾到当前时间点的信息。

加料回潮设备的状态可能受到之前和之后操作环节的影响。BiGRU 通过学习时间序列数据的前向和后向依赖，可以更准确地预测设备状态，从而帮助监测和调节设备的工作性能。

3. TCN 卷积：

TCN 采用的是一种特殊的卷积，称为因果卷积，确保在进行预测时，模型不会“窥视”未来的信息。这是通过设计卷积核来实现的，其中输出在时间 t 上只依赖于时间 t 及其之前的输入点。膨胀卷积可以帮助网络覆盖更大的接收域，即模型可以访问的历史序列信息的广度。膨胀卷积通过定义一个膨胀因子来跳过输入数据中的某些点，这个因子随着网络深度指数般增长，可以使网络捕捉到更长期依赖关系，而不显著增加计算成本。

TCN 通常包含残差连接，它允许较浅层的输出直接与更深层次的层相加。这种设计有助于缓解梯度消失问题，使得更深的网络能够有效地训练，同时允许信息在网络中更快地流动。

TCN 模型的卷积操作采用了膨胀卷积（Dilated Convolution），膨胀卷积通过在卷积核之间插入零元素来扩大感受野，示意图如下所示。

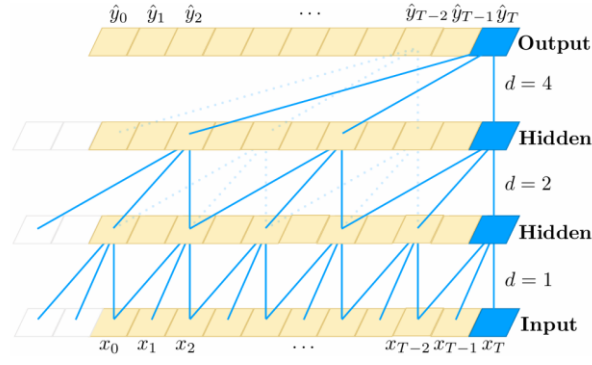


图 3.33 TCN 卷积模型网络结构

膨胀卷积的数学表达式为:

$$y[t] = \sum_{k=0}^{K-1} w[k] \cdot x[t - d \cdot k] \quad (3-67)$$

其中, $y[t]$ 是卷积操作的输出, $w[k]$ 是卷积核的权重, $x[t - d \cdot k]$ 是输入序列的元素, d 是膨胀率。

TCN 模型的残差块由两个卷积层和一个残差连接组成。残差块的计算过程为: 输入 x 经过一个膨胀卷积层, 得到输出 y , 将 y 与输入 x 相加, 得到残差块的输出。

残差块的数学表达式为:

$$\text{Output} = x + \text{Conv1D}(x) \quad (3-68)$$

残差-TCN 结构如下图所示:

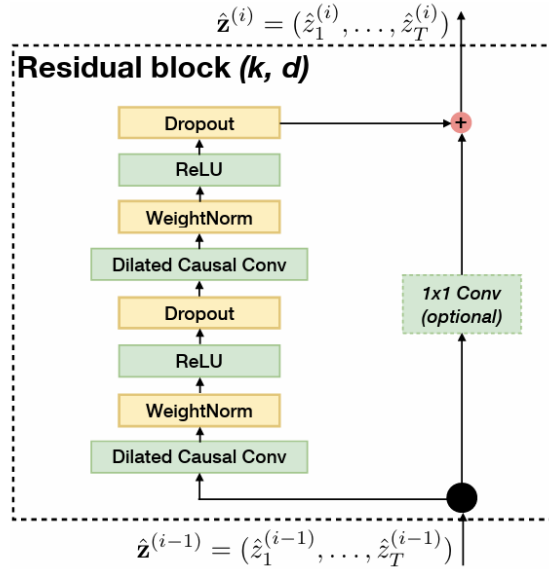


图 3.34 残差-TCN 结构

3.2.4 预测系统框架

本文选择 MSELoss 作为该模型的损失函数, 其计算公式如下。

$$MSELoss = \frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2 \quad (3-69)$$

模型的整体结构如图 3.34 所示，

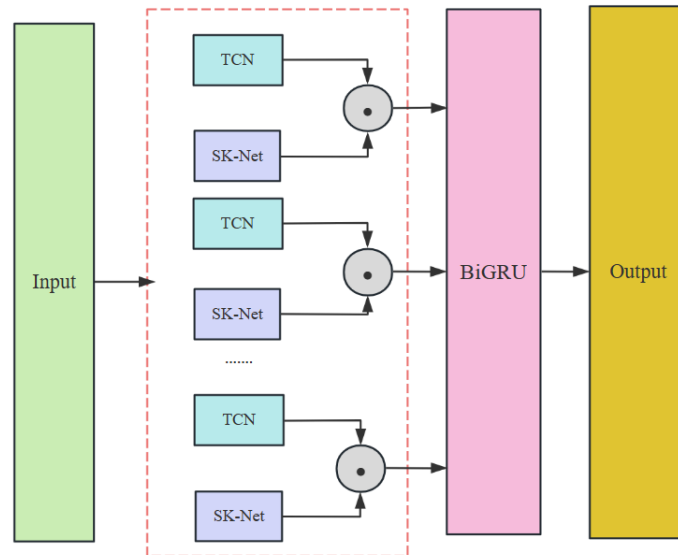


图 3.35 预测模型整体框架

预处理后的输入数据首先流经 SK-Net 注意力机制模块。在这个模块中，模型会关注不同范围内的特征，并识别出那些对于回潮设备状态预测最为重要的信息。同时，输入数据还会流到 TCN 模块。TCN 层将会通过一系列的因果卷积层，扩展卷积层以及残差连接来处理时间序列数据。TCN 可以捕捉长期的时间依赖性，并且能够保持较低的延迟。处理过后的数据保留了输入序列的时间特征；接下来，由 SK-Net 模块输出的多尺度空间特征和 TCN 模块输出的时间特征进行点乘操作。通过点乘操作融合后的结果被输入到 BiGRU 中。双向 GRU 利用它的两个 GRU，一个处理正向信息流，而另外一个处理反向信息流，这样可以同时捕捉序列的过去和未来的依赖关系。通过这种方式，BiGRU 应该可以提供一种更全面的时间特性表征。

整个模型的预测算法流程图如图 3.36 所示：

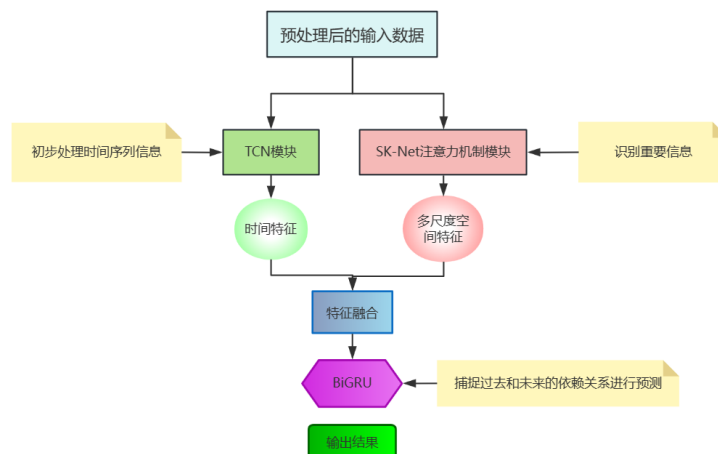


图 3.36 整个模型的预测算法流程图

3.2.5 消融对比实验

以预测车流量为例，通过进行消融实验，将模型中的 SK-Net 注意力机制去掉、TCN 替换为常规的 CNN 卷积层，并将 BiGRU 替换为单向 GRU 层，以验证模型各个组成部分的功能与效能。

表 3.14 消融对比试验

SK-Net	TCN	BiGRU	MSE	RMSE	MAE	MAPE	R ²
√	√	√	0.00002	0.00410	0.00329	0.01547	0.98703
	√	√	0.00012	0.01131	0.00890	0.04178	0.90125
√		√	0.00002	0.00428	0.00395	0.01857	0.98582
√	√		0.00002	0.00432	0.00383	0.01804	0.98558

从消融实验数据中，我们可以看到各个组件(SK-Net 注意力机制、TCN 层、以及 BiGRU 层)在预测模型中的贡献和影响。完整模型表现最好，拥有最低的误差指标和最高的 R² 值，这表明包含所有组件的模型在此预测任务上具有最高的性能；去掉 SK-Net 后，误差明显增加，R² 值显著下降，显示 SK-Net 的注意力机制对准确率有显著的提升作用；去掉 TCN 层，模型整体性能略有下降，但仍然保持较高的准确度。这表明尽管 TCN 对模型有所贡献，但 SK-Net 和 BiGRU 的存在能够在一定程度上补偿 TCN 层的缺失；移除 BiGRU 后，模型仍然表现出较高的预测准确性，这可能是因为 SK-Net 和 TCN 的组合依然有效地捕获了时间序列数据的关键特征。

3.2.6 车流量和车流速度预测与分析

1. 数据处理：

在进行实验之前，需要对数据进行标准化处理，以消除量纲的影响。如果对未标准化的数据直接进行模型的训练，可能会导致模型对数值大的变量学习过多，而对数值小的变量训练不够充分，往往模型效果会不好。常用的数据标准化方法有最大最小归一化、均值方差标准化、小数定标法、定量特征二值化等。

本实验采取最大最小归一化的方法，其利用数据列中的最大值和最小值进行标准化处理，标准化后的数值处于[0,1]之间，计算方式为数据与该列的最小值作差，再除以极差。

具体公式为：

$$x' = \frac{x - \min}{\max - \min} \quad (3-70)$$

其中， x' 表示单个数据的取值， $\min$ 是数据所在列的最小值， $\max$ 是数据所在列的最大值。

下列评价指标是用于预测模型和回归分析中常用的性能度量方法：

MSE (Mean Squared Error, 均方误差): MSE 是通过计算模型预测值与真实值之差的平方然后求平均得到的。它衡量的是预测误差的平方和的平均值。

RMSE (Root Mean Squared Error, 均方根误差): RMSE 是 MSE 的平方根。它的计算方式更符合实际误差的量度，因为它的单位与原数据保持一致。

MAE (Mean Absolute Error, 平均绝对误差): MAE 是通过计算模型预测值与真实值之差的绝对值然后求平均得到的。它衡量的是预测误差的绝对值的平均。

MAPE (Mean Absolute Percentage Error, 平均绝对百分比误差): MAPE 是通过计算模型预测值与真实值之间差的绝对值除以真实值, 然后对这些商值求平均得到的百分比。它衡量的是预测误差占真实值比例的平均。

R^2 (R-squared, 决定系数): R^2 衡量的是模型解释的变异量占总变异量的比例, 用于度量模型的拟合优度。 R^2 越接近于 1 表明模型解释的变异量越大, 拟合度越好。

2. 训练集测试集划分:

因为需要预测第三点到第四点之间路段可能出现持续拥堵状态的二点实时预警, 那么需要选择第三点和第四点对应的时间进行预测中间是否会有拥堵。本文在分析了视频时间后给出了如下训练集测试集的划分:

表 3.15 GRU 训练集测试集的划分

交通点	测试集	训练集
节点三	12:56~13:52	11:35~12:56, 13:52~16:09
节点四	12:56~13:52	13: 52~15:20

3. 模型训练过程:

表 3.16 时序预测模型训练效果图

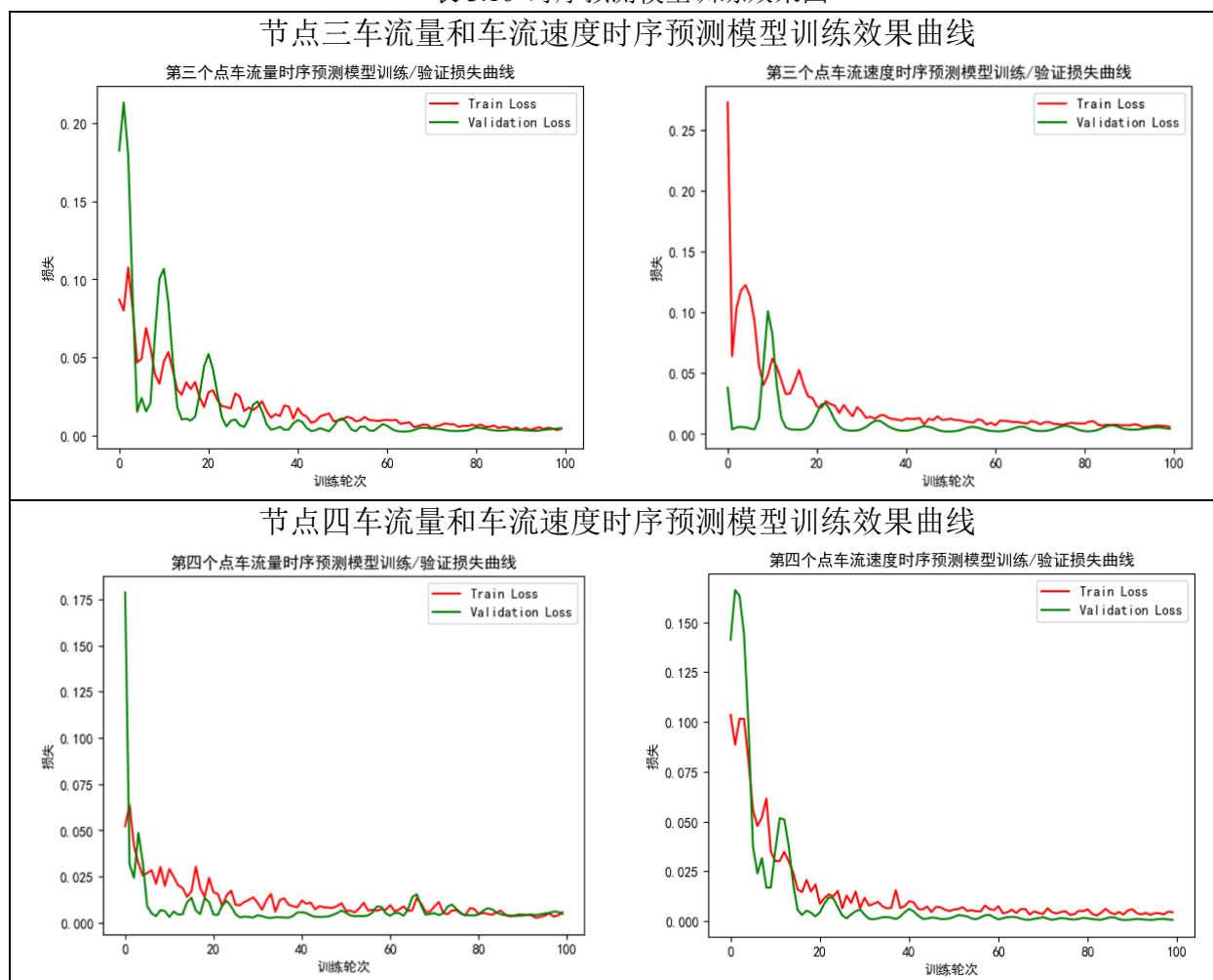
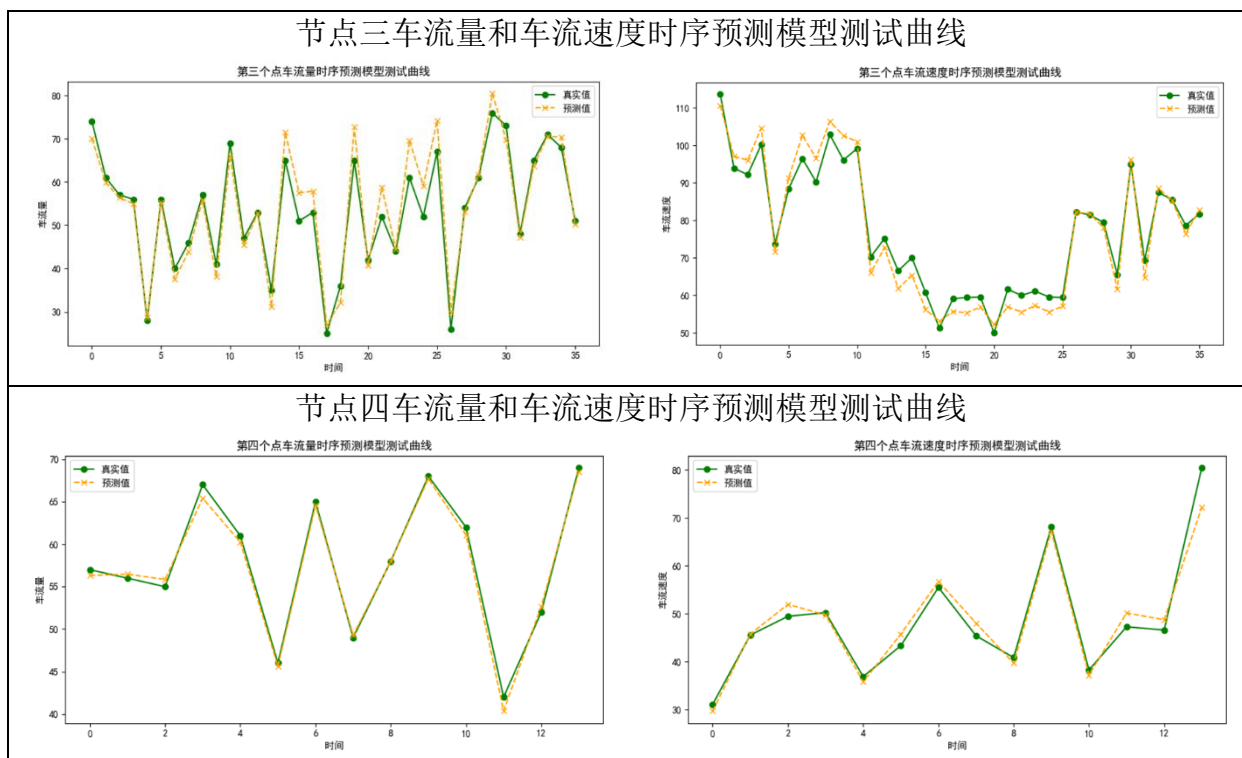


表 3.17 时序预测模型测试效果图



模型训练最终的评价指标如下表：

表 3.18 模型评价指标结果

	MSE	RMSE	MAE	MAPE	R ²
节点三（视频一）	14.82286	3.85004	2.96889	5.76406	0.91791
节点三（视频二）	12.73061	3.56799	3.15878	4.30141	0.95211
节点四（视频一）	0.64855	0.80533	0.67039	1.21023	0.98996
节点四（视频二）	7.72926	2.78015	2.02457	3.95321	0.94880

其中：

MSE 代表均方误差；RMSE 代表均方根误差；MAE 代表平均绝对误差；MAPE 代表平均绝对百分比误差；R² 代表决定系数。

结果分析：

训练和损失曲线：

训练损失和验证损失都在早期迅速下降，说明模型在训练初期对数据的拟合效果在快速提升。经过大约 20 轮次后，损失趋于平稳，训练损失和验证损失之间差距不大，说明模型的泛化能力较好，没有明显的过拟合现象。最终的损失值较低，显示模型在对训练数据和验证数据上都有良好的表现。

预测结果曲线：

预测曲线与实际曲线基本重合，特别是在大部分波动点上，两者拟合得较好。某些局部点上，预测值与实际值略有偏差，但整体趋势与实际情况高度一致。预测结果表明模型在时间序列上的表现较为准确，能够较好地捕捉数据的变化模式。

模型评价指标：

MSE、RMSE、MAE、MAPE 模型评价指标都较小，表示误差控制在较低水平；R² 接近 1，表示模型对数据的拟合度非常高，能够很好地解释数据的变异性。

3.2.7 预测结果实时预警建模

(1) 3、4 点同时段拥堵指数图与平均车速分析：

在得到预测结果车流量和车流速度后，需要对预测的车流量和车流速度进行建模，本文截取了第三个点和第四个点的同时段的视频进行相应的分析(12:56-13:52),并且画出了相应的拥堵指数图和平均车速图：

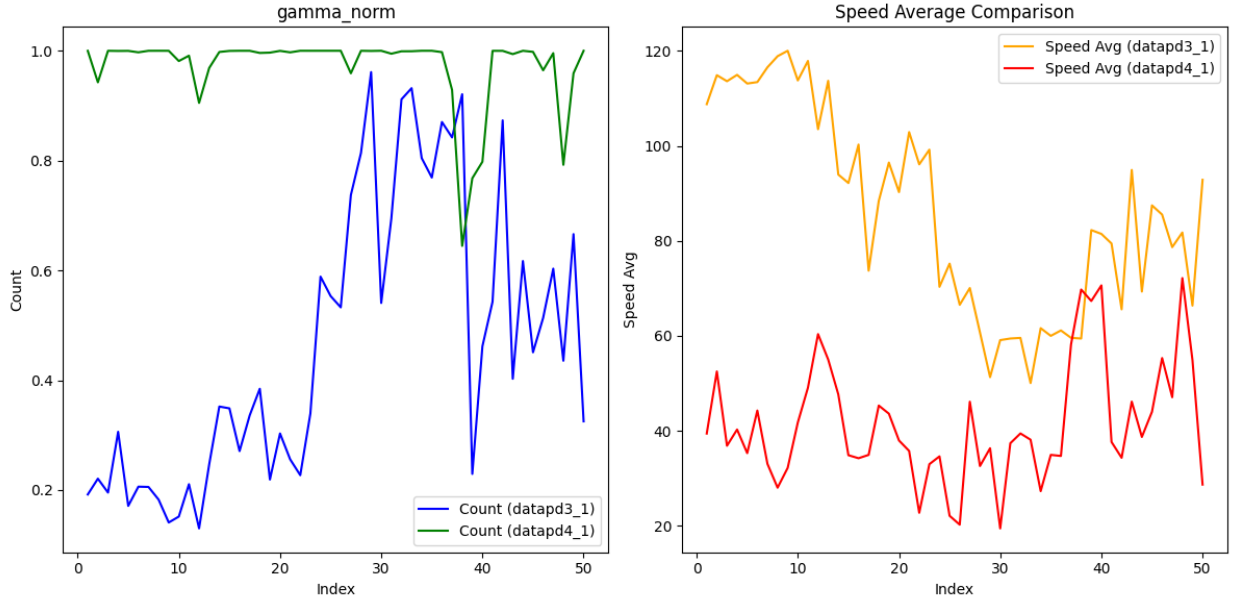


图 3.37 拥堵指数图与平均车速图

从图中提供的数据来看，左边的图表示拥堵指数，显示的是两个位置点（3 和 4）的拥堵程度随时间的变化。可以观察到，在时间范围内，点 4（绿色线）的拥堵指数通常先于点 3（蓝色线）上升，表明在交通流动中，如果发生拥堵，点 4 会先堵塞，而后通过拥堵波的传递，拥堵影响会传递到点 4，而经分析拥堵波的传递时间为 3 到 5 分钟。

右边的图表则是关于车辆的平均速度对比。这里显示车辆在点 3（橙色线）处的速度通常在点 4（红色线）之前上升，也就是说，车辆的加速在点 3 处先发生，随后影响传递到点 4 处。通过这两个图表的对比，我们可以推测交通流动中的滞后效应，即在交通流量较大的情况下，车辆在点 4 处会先出现拥堵，然后逐渐影响到点 3，而速度的变化是相反的，车辆先在点 3 处加速，再传递到点 4。

这种现象表明了交通流中某些点之间的影响，尤其是当拥堵出现时，堵塞会逐渐向后传递，影响后续交通流，而车辆的加速则是先从后向前传递的。根据这些信息，可以考虑在点 4 提前采取措施以缓解潜在的拥堵问题，确保交通更加顺畅。

同时我们还发现，点 4 的拥堵状况在大多数情况下是大于点三的，对于这一问题，我们分析可能是由于点 4 前面有服务区，经常有车辆进出而造成的。

(2) 激波计算拥堵波传递时间：

本文采用激波计算拥堵波从 4 点传递到 3 点的时间，当交通参数发生突变时，会产生激波（Shockwave），其传播速度为：

$$w = \frac{q_2 - q_1}{k_2 - k_1} \quad (3-71)$$

其中 q_1, k_1 ：激波前方（上游）的流量和密度， q_2, k_2 激波后方（下游）的流量和密度。

w 激波在空间上的传播速度。利用预测模型同时预测点 3 和点 4 的车流量和车流速度，并且计算进入和离开路段的流量差： $\Delta q = q_3 - q_4$ 。如果 $\Delta q > 0$ ，表示车辆在路段内积累，可能形成拥堵。

同理可以计算出密度差异： $\Delta k = k_4 - k_3$ 最后得出激波速度 $w = \frac{\Delta q}{\Delta k}$ 。

对 w 进行相应的情况分析可得知：

1: $w < 0$: 激波向上游（点 3 方向）传播，可能导致上游发生拥堵。

2: $w > 0$: 激波向下游（点 4 方向）传播，对上游影响较小。

同时拥堵波到达点 3 的时间为：

$$t_{\text{arrival}} = \frac{D}{|w|} \quad (3-72)$$

D 表示激波从点 4 到点 3 的距离，此处取 3km。

(3) 实时预警依据

实时预警和验证：利用模型提供对该路段的实时拥堵预警，并结合实际视频数据对模型的有效性进行验证，确保模型能够准确预警潜在的拥堵现象。

在预测出未来的车流量和车流速度以后，我们给出如下的预警流程图：

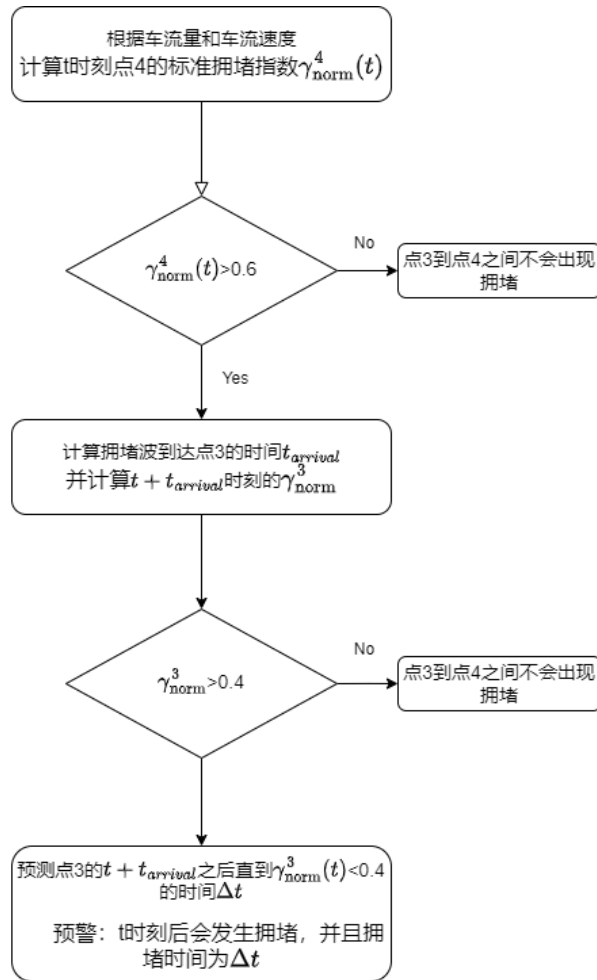


图 3.38 预警流程图

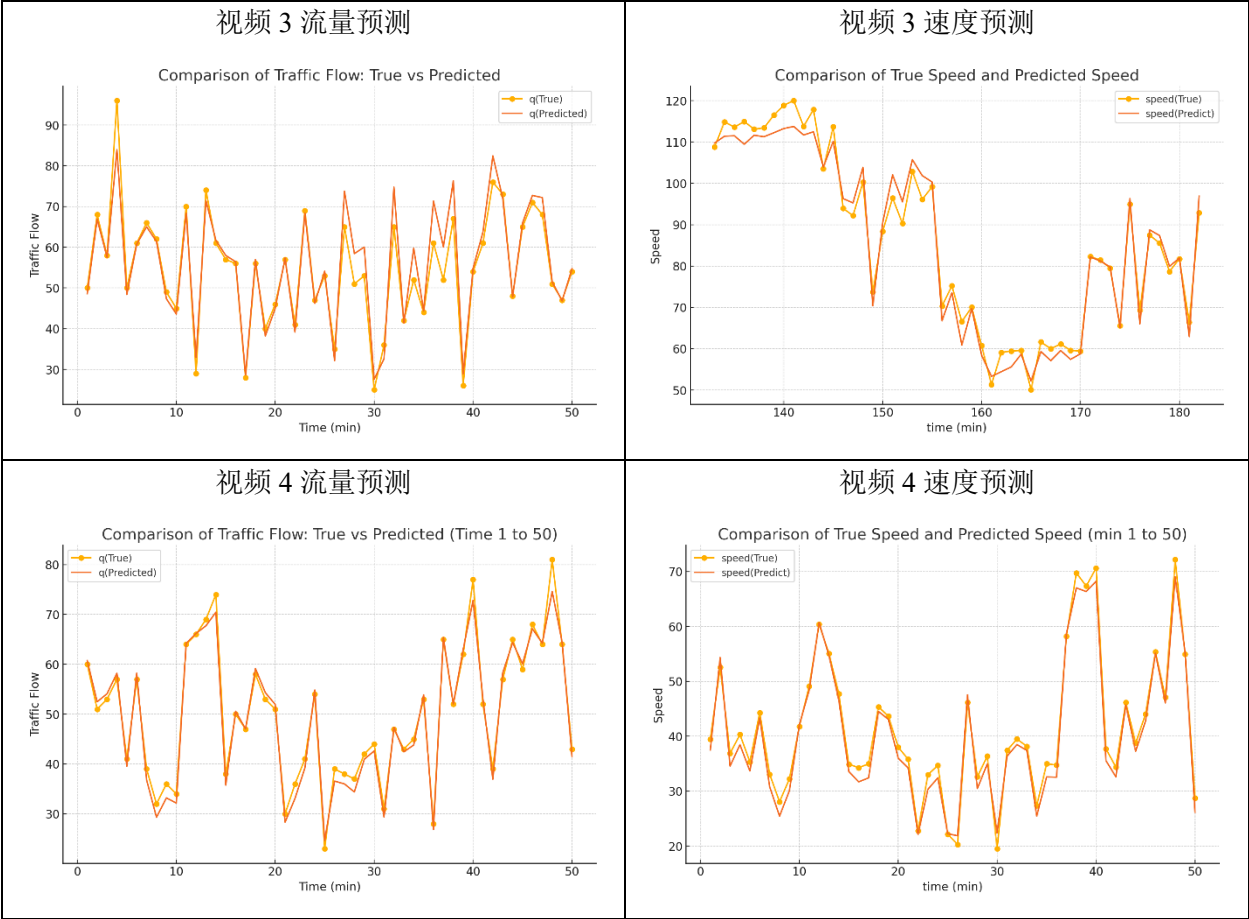
流程图说明： $\gamma_{norm}^4(t)$ 和 $\gamma_{norm}^3(t)$ 分别表示 4 点和 3 点在 t 时刻的标准拥堵指数。对于

4 点拥堵指数阈值超过 0.6（即中度拥堵及以上）则认为可能引发拥堵波，而对于 3 点拥堵指数阈值超过 0.4（即轻度拥堵及以上）则认为会受到拥堵波的影响。 $t_{arrival}$ 代表拥堵波从 4 点传播到 3 点所需的时间。 Δt 拥堵在 3 点到 4 点持续的时间。

首先预测指定时刻的交通参数，并计算 4 点的拥堵指数。如果 4 点的拥堵指数超过阈值，意味着可能会有拥堵波向上游传播，需要计算其到达 3 点的时间。在拥堵波可能到达 3 点的时刻，再次预测 3 点的交通参数，计算其拥堵指数。如果 3 点的拥堵指数也超过阈值，意味着拥堵波会影响 3 点，需要预测拥堵将持续多长时间，并进行预警。如果任何一步的拥堵指数未超过阈值，则认为 3 到 4 段不会出现拥堵。

最后在训练好模型后我们利用模型预测了 12:56~13:52 的交通流量和交通流速度，并且与真实值相比较，结果如下表所示：

表 3.19 交通流量和交通流速度训练效果



结果分析：

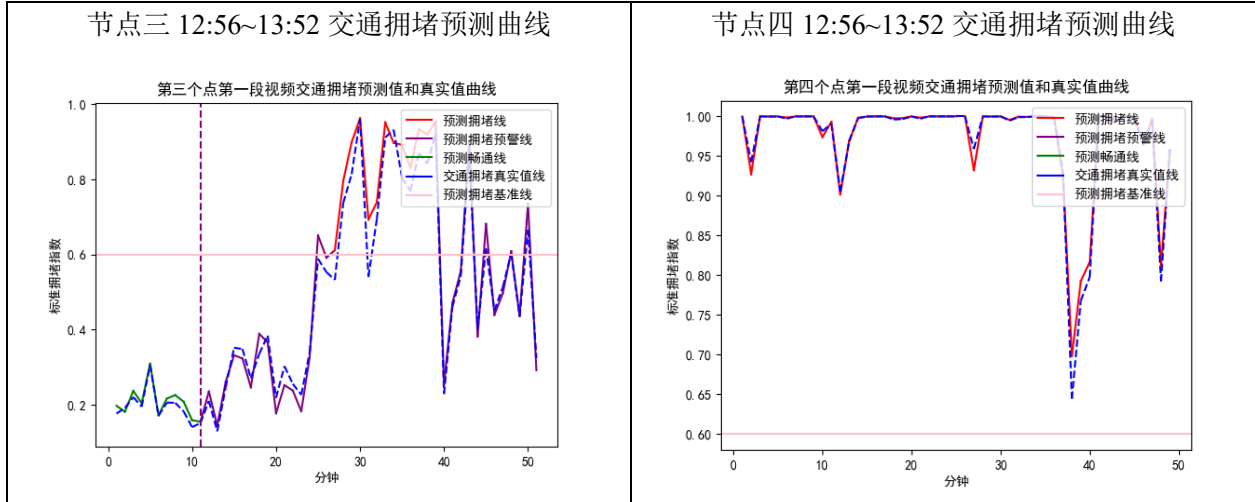
从这四个折线图可以看出，模型在预测交通流量和速度时表现出较好的准确度。无论是视频 3 还是视频 4，模型预测的流量和速度基本上与真实值保持较高的相似性。

尽管有一些小的偏差，尤其是在某些时间点上预测值稍高或稍低，但整体趋势仍然正确，这说明该预测模型已经较为成熟，可以用于未来的交通流量和速度预测场景中。

对于某些出现明显偏差的时刻，模型可能还需要进一步优化，以减少这些短时间段内的误差。总体来说，该模型具有较强的预测能力，尤其是在大趋势的把握上表现较好。

同时计算出了相应的拥堵指数与真实值相比的图像，如下表所示：

表 3.20 交通拥堵预测曲线



结果分析：

时间误差：根据描述，预测模型在 13:13 左右预计会出现交通拥堵，并在 13:23 左右发出拥堵信号，与真实的交通拥堵时间 13:20 仅有 3 分钟的误差，表明模型在时间预测上的精度较高。

拥堵持续时间：预测模型预计拥堵将持续 12 分钟，而实际的拥堵持续了 15 分钟，误差在 3 分钟以内，这也是一个较为精准的预测。

预测误差范围：根据描述，预测时间的误差控制在 20%以内，说明模型整体的预测准确度相对可靠，能够为交通管理提供有效的参考。

3.3 子问题（3）

3.3.1 问题分析

问题一的第三小问要求利用视频数据对所建立模型的有效性进行验证。本文采用了监控点 1 和监控点 2 的第一段视频数据进行模型的训练。随后利用训练好的模型对第二段监控视频进行预测，从而评估模型在实际应用中的有效性和泛化能力。通过将预测结果与计算得出的真实标准拥堵指数进行对比，进而验证在预测拥堵发生前约十分钟的时间段，模型是否可以实现有效的提前预警。

算法流程图如下：

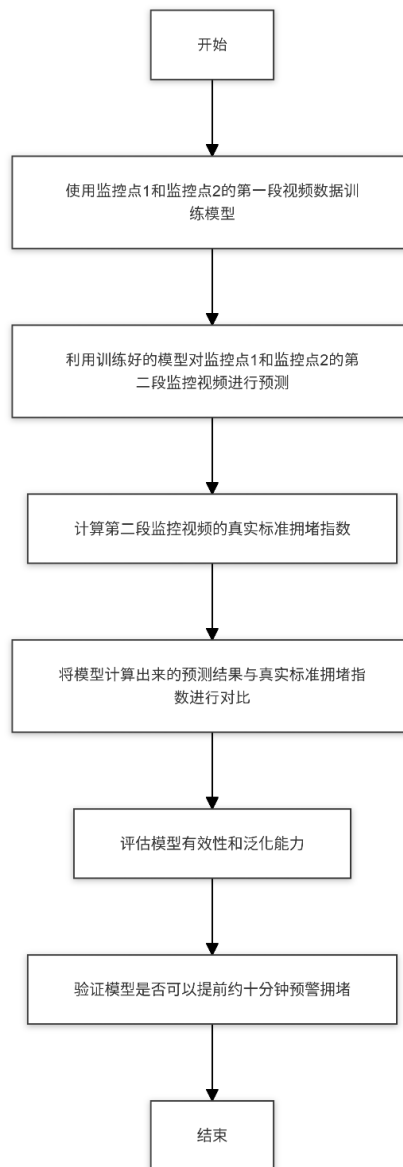


图 3.39 思路流程图

3.3.2 验证模型有效性

实时预警和验证：利用模型提供对该路段的实时拥堵预警，并结合实际视频数据对模型的有效性进行验证，确保模型能够准确预警潜在的拥堵现象。

对该模型进行了 100 轮的训练，设置学习率为 0.001。分别预测其每个点位的车流速度，以下是不同路段不同点位的训练与验证损失曲线、模型评价指标和预测结果曲线：

表 3.21 车流量和车流速度时序预测模型训练效果

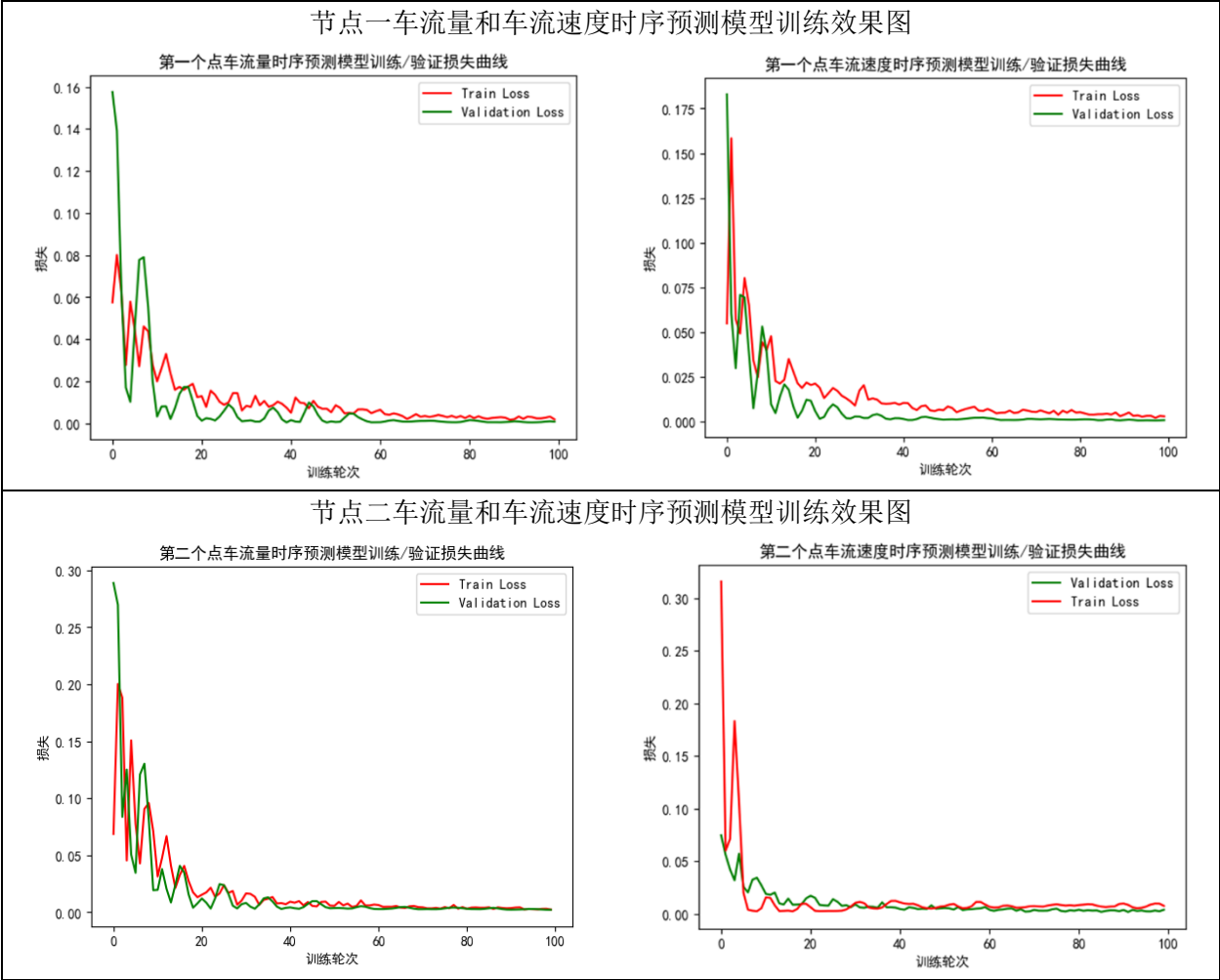
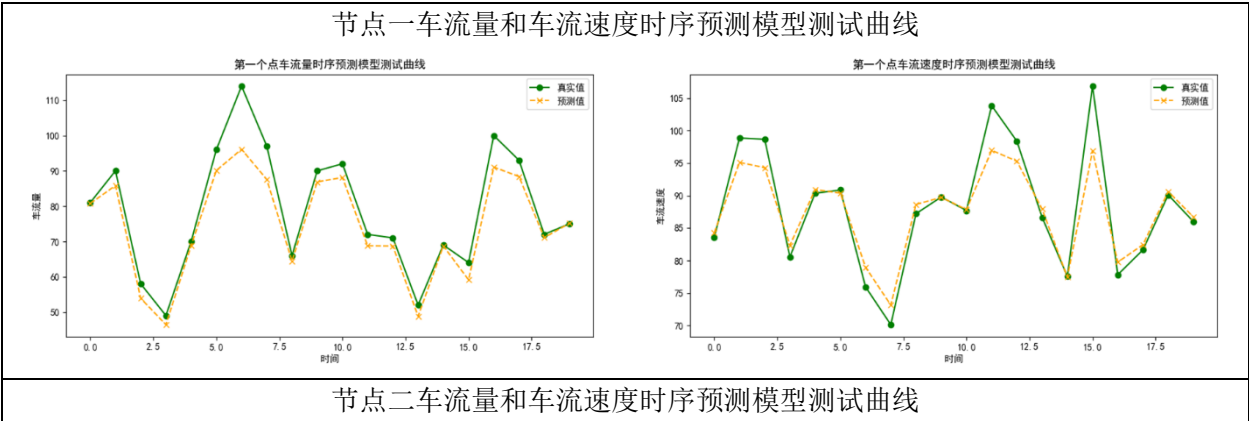
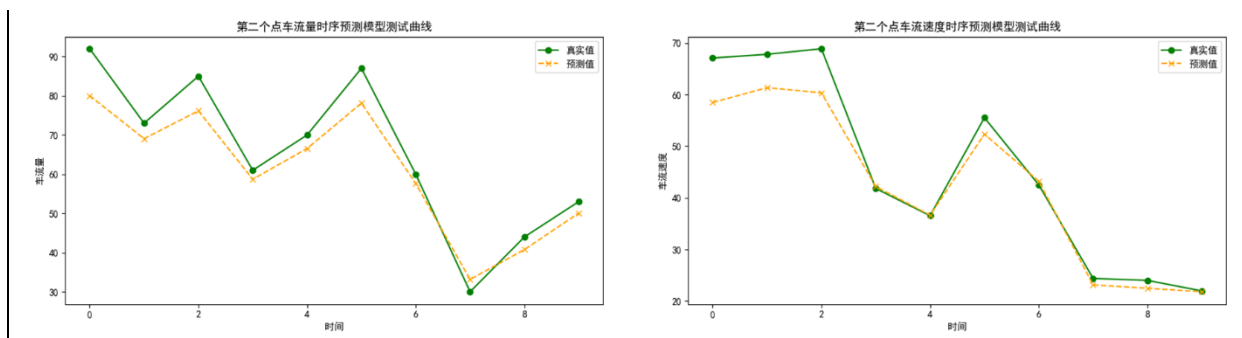


表 3.22 车流量和车流速度时序预测模型测试曲线





模型训练最终的评价指标如下表：

表 3.23 模型训练最终的评价指标

	MSE	RMSE	MAE	MAPE	R^2
节点一（视频一）	4.01118	2.00279	1.71149	2.04259	0.96516
节点一（视频二）	60.11071	7.75311	7.31109	6.31928	-0.81001
节点二（视频一）	8.58122	2.92937	2.36429	2.45081	0.96721
节点二（视频二）	20.40765	4.51748	3.09883	5.57180	0.93577

其中：

MSE 代表均方误差；RMSE 代表均方根误差；MAE 代表平均绝对误差；MAPE 代表平均绝对百分比误差； R^2 代表决定系数。

结果分析：

训练和损失曲线：

在训练初期（前 10 次迭代左右），损失快速下降，尤其是在初期的波动较大。这表明模型在初期快速学习并调整参数；经过 20 次迭代后，损失曲线趋于平稳，训练损失和验证损失都逐渐接近并稳定在较低值，说明模型已经较好地收敛；验证损失的变化趋势与训练损失接近，且在后期没有明显的过拟合现象，即模型泛化能力较强。。

预测结果曲线：

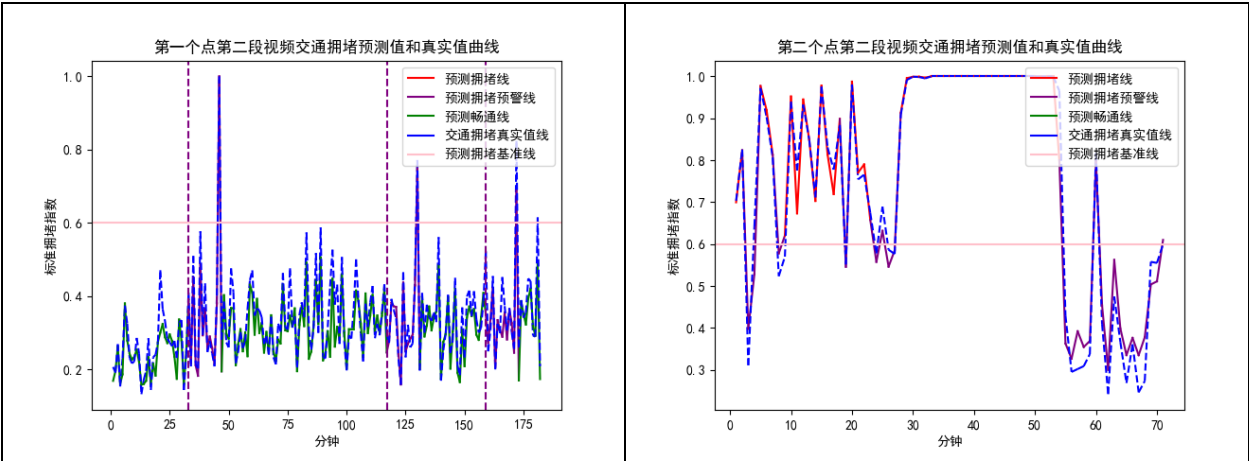
两个节点的视频数据测试显示，模型预测的车流量与车速曲线基本能够跟随实际值的波动，表明模型在大部分情况下都能较准确地反映真实的交通情况；但也存在部分节点的预测值与实际值有一定的偏差，尤其在数据波动较大的时段，模型预测值无法完全跟随实际值。这可能表明模型在面对大幅度波动的交通数据时，预测的精度略有下降。

模型评价指标：

MSE、RMSE、MAE、MAPE 等指标较低，表明模型的误差较小； R^2 接近 1，意味着模型对数据的拟合效果优异，能够充分解释数据的变动情况。

接下来对不同路段监控点的交通拥堵值进行预测，并于真实值进行对比：

表 3.24 交通拥堵预测值与真实值曲线



结果分析：

(1) 第一个图（第一个点第二段视频交通拥堵预测值和真实值曲线）：

拥堵指数波动较大：图中的交通拥堵指数（绿色为真实值，蓝色为预测值）在时间轴上波动频繁，说明该路段的交通流量存在较多的变化和拥堵点，特别是预测值与真实值的吻合度较高，虽然在一些时刻有少量偏差。

竖直线（预测拥堵点）：竖直线标示出拥堵预警的时刻。可以看到，在竖直线后，交通拥堵指数的峰值较高，接近 0.6 至 1 的范围，表明该点的拥堵程度较为严重。这些点与红色的“拥堵基准线”对比，可以明显看到拥堵预测点的出现是合理的，符合实际数据的变化趋势。拥堵预警线起到了预测拥堵时间点的作用，这些线表明在拥堵点之前 10 分钟左右预警功能已经触发，说明模型能够提前预判交通拥堵的发生并发出预警。

(2) 第二个图（第二个点第二段视频交通拥堵预测值和真实值曲线）：

拥堵指数整体较高：在第二张图中，交通拥堵指数在很大一部分时间段内维持在较高水平（接近 0.7-1），表明该路段的交通流量拥堵情况较为严重，且预测值和真实值的吻合度较高，特别是在高峰期，两条曲线基本重合，说明模型在这一部分的预测非常准确。

低拥堵段落的偏差：在后半段时间内，指数出现大幅下降，特别是 60 分钟到 70 分钟之间，真实值和预测值均有较明显的下滑，但仍然保持较为相似的趋势，说明模型在拥堵消退时的预测也较为准确。

四、 问题二模型建立与求解分析

4.1 问题分析

为了缓解高速公路上可能发生的交通拥堵，合理启用应急车道成为一种有效的临时措施。然而，在确保安全性和通行能力的前提下，科学合理地决定何时启用应急车道，需要建立系统决策的理论依据。该理论依据不仅应基于实时交通流数据，还需结合道路条件、历史数据和环境因素，以实现应急车道的合理使用，避免对整体交通造成不必要的干扰。应急车道的启用不仅需要缓解交通压力，同时也要确保道路安全。为此，理论依据考虑了以下几点：

(1) 通行能力评估：通过对车流量、车速等参数的实时监控，此理论依据可以判断在启用应急车道后对整体通行能力的提升效果，并在密度回落、标准拥堵指数降到中度拥堵以下、车速恢复至正常水平后建议关闭应急车道。

(2) 动态调整机制：此理论依据具有动态调整激波传播速度和密度临界值的功能。通过不断更新的交通流数据，此理论依据可以灵活调整应急车道的启用与关闭策略，确保其启用时间合理且有效，关闭时不会再次导致拥堵。

算法流程图如下：

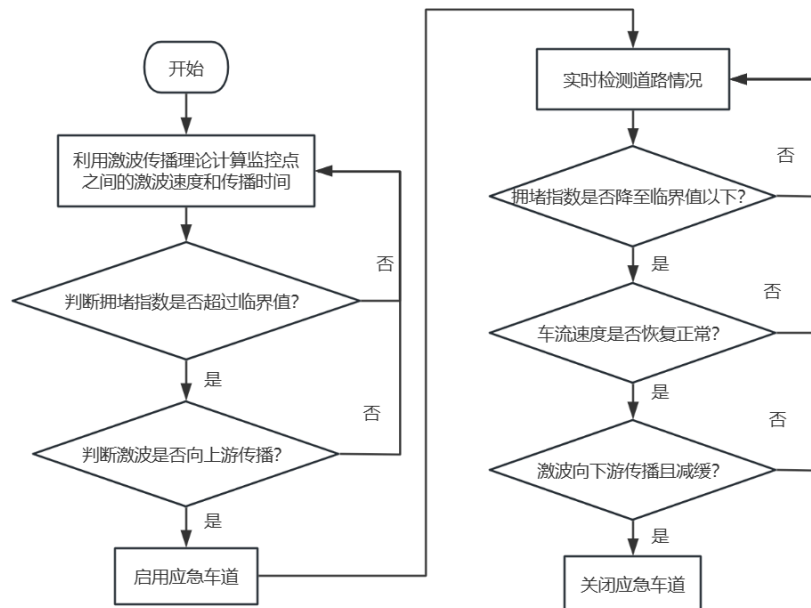


图 4.1 思路流程图

4.2 开启应急车道的理论条件



图 4.2 路段示意图

根据所提供的监控点布局，监控点 1 与监控点 2 的距离为 1 公里，监控点 2 与监控点 3 之间为 1 公里，而监控点 3 与监控点 4 之间的距离为 3 公里。车辆由监控点 1 向监控点 4 的方向行驶，形成了一个分段的交通流动系统。为了准确预测可能发生的交通拥堵，我们引入了之前提到的激波传播理论进行建模分析。

根据激波传播的基本原理，激波的速度 w 取决于流量 q 和密度 k 的变化。公式如下：

$$w_{ij} = \frac{q_j - q_i}{k_j - k_i} \quad (4-1)$$

其中：

- q_i, k_i 是监控点 i 的车流量和车流密度；
- q_j, k_j 是监控点 j 的车流量和车流密度；
- w_{ij} 表示激波在监控点 i 到监控点 j 之间的传播速度。

我们需要分别计算以下三个区域的激波速度：

- w_{12} : 从监控点 1 到监控点 2 的激波速度；
- w_{23} : 从监控点 2 到监控点 3 的激波速度；
- w_{34} : 从监控点 3 到监控点 4 的激波速度。

激波从监控点 i 传到监控点 j 的传播时间 t_{ij} 可以表示为：

$$t_{ij} = \frac{D_{ij}}{w_{ij}} \quad (4-2)$$

其中 D_{ij} 是监控点 i 和监控点 j 之间的距离。

我们要分别计算以下三个区域的时间：

- $t_{12} = \frac{D_{12}}{w_{12}}$: 从监控点 1 到监控点 2 的时间；
- $t_{23} = \frac{D_{23}}{w_{23}}$: 从监控点 2 到监控点 3 的时间；
- $t_{34} = \frac{D_{34}}{w_{34}}$: 从监控点 3 到监控点 4 的时间。

通过上述计算可以获得激波从监控点 1 传递到监控点 4 的总时间：

$$t_{\text{total}} = t_{12} + t_{23} + t_{34} \quad (4-3)$$

t_{total} 可以用于预判整个道路段的交通状况。当 t_{total} 显示拥堵波将在未来某个时间到达时，系统会提前进行预警，并结合以下标准决定是否启用应急车道。

在前文我们已经设定了标准拥堵指数 $\gamma_{\text{norm}}^p(t)$ ，我们设定临界标准交通拥堵指数 γ_{critical} 作为预判拥堵的关键指标，在本文中我们以 $\gamma_{\text{critical}} > 0.6$ 为判定的标准，即达到中度拥堵以上。该临界值根据历史交通数据、道路承载能力和环境因素确定。当监控点 2、监控点 3 或监控点 4 的车流密度满足：

$$\gamma_{\text{norm}} > \gamma_{\text{critical}}$$

则系统判定存在较高的拥堵风险。

结合激波传播的方向和速度，对交通状况进行深入分析：

- 当 $w_{ij} < 0$ ：激波向上游传播，表示上游可能出现拥堵，需要提前预警并采取分流措施；
- 当 $w_{ij} > 0$ ：激波向下游传播，虽然对上游影响较小，但下游区域可能面临更严重的拥堵，应密切关注。

通过对监控点 3 和监控点 4 的实时数据分析，可以判断拥堵波是否会从下游向上游传播。如果激波速度较大，且趋势显示上游区域将受到影响，应考虑在该区域提前启用应急车道，以缓解潜在的拥堵压力。

当满足以下条件时，应考虑启用应急车道：

- 总传播时间临界值：当 $t_{\text{total}} \leq T_{\text{threshold}}$ （如 10 分钟）时，表示拥堵波将在短时间内到达，需要提前应对；
- 车流密度超出临界值：监控点的车流拥堵指数从持续高于 γ_{critical} ；
- 激波向上游传播： $w_{ij} < 0$ ，且有加速趋势。

根据实时数据，动态调整激波传播速度 w_{ij} 和预警时间，及时更新应急车道的启用策略。

4.3 关闭应急车道的决策条件

为了确保应急车道的关闭不会导致交通再次出现拥堵或高负载情况，以下几个条件需要同时满足：

条件 1: 车流拥堵指数 $\gamma_{\text{norm}}^p(t)$ 回落到安全水平

当车流拥堵指数 $\gamma_{\text{norm}}^p(t)$ 降到临界密度 γ_{critical} 以下时，说明拥堵情况已经得到有效缓解，交通压力下降到可接受范围。临界密度可以通过历史数据和道路承载能力来设定。具体条件为：

$$\gamma_{\text{norm}}^p(t) < \gamma_{\text{critical}}$$

此时，车流密度的下降表明交通状况逐步恢复，车辆通过率较高，可以考虑关闭应急车道。

条件 2: 车速恢复至正常水平

车速的恢复是衡量交通状况改善的重要指标。若车速恢复至一定水平（如 $v(t) > v_{\text{normal}}$ ，可以通过历史平均速度设定该阈值），表明道路通行效率已经大幅提高，拥堵已经基本消除。具体条件为：

$$v(t) > v_{\text{normal}}$$

若车速持续超过设定的阈值，则说明应急车道的任务已完成，可以安全关闭。

条件 3: 激波向下游传播且减缓

通过激波模型可以判断拥堵波的消散情况。如果激波速度 w 为正（即 $w > 0$ ），表明拥堵波向下游传播，且速度逐渐减缓，意味着上游交通状况正在恢复，此时可以考虑关闭应急车道。具体条件为：

$$w_{ij} > 0 \text{ 且 } w_{ij} \text{ 持续减缓}$$

这意味着拥堵波已离开该路段，交通回归正常状态。

当同时满足以上三个条件时，应急车道可以安全关闭。这确保了在交通状况恢复的情况下及时释放应急车道，不对正常车道通行产生不必要的干扰。在关闭应急车道后，系统仍需对交通状况进行一段时间的持续监控，确保不会因关闭应急车道而出现新的拥堵。通过实时监控数据调整决策，持续评估交通流状况，以避免在交通恢复前过早关闭应急车道。如果再次发现流量或密度的异常波动，系统应有能力迅速重新启用应急车道。

五、 问题三模型建立与求解分析

5.1 问题分析

本研究基于实时监控数据，预测未来 t 时刻的交通流量和流速，并计算标准拥堵指数和拥堵波。我们利用标准交通拥堵指数 $\gamma_{norm}^p(t)$ 与推导出的拥堵波 w ，建立了一个联合算法，以控制是否开启应急车道，实现节点间路段的分段控制。

同时为验证算法的有效性，采用 Greenshields 模型进行建模，分析不同流量、速度和密度之间的关系。通过这一模型，我们量化了应急车道启用的影响，评估其对缓解交通拥堵的效果。最终结果表明，所设计的决策算法在实际应用中具有显著的有效性，为改善交通流提供了可行的解决方案。

算法流程图如下：

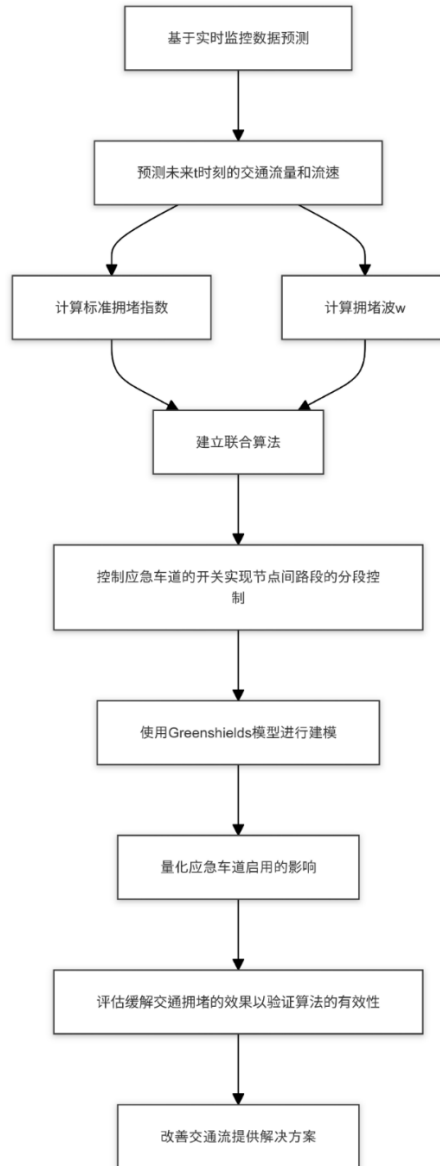


图 5.1 思路流程图

5.2 建立规则启用应急车道

在本节中我们将设计合理的算法实时决策是否启用应急车道，为了解决这个问题在前文我们已经分析交通数据，并且从四个观测点的监控数据中提取流量、密度和速度，以了解交通动态。同时建立拥堵预测模型：利用交通流理论中的基本关系，预测 3 号点和 4 号点之间路段的拥堵情况。并且构建了合理的启用高速公路应急车道模型。我们在总结了前面的算法以后提出了一个基于拥堵波和拥堵指数联合决策的算法模型，该算法模型的伪代码如下：

算法：交通拥堵决策模型

已知：当前的车流流量和车流速度，已知节点与节点之间的距离 D

算法流程：

```
while 预测的  $i, j$  点  $t$  分钟后的车流流量和车流速度，且  $j$  点在  $i$  点的下游：
| 已知车流流量和车流速度，根据公式 (3-62) 和 (3-63) 计算  $t$  时刻  $j$  点的标准拥堵指数  $\gamma_{norm}^j(t)$ 
| if  $\gamma_{norm}^j(t) > 0.6$ :
| | 利用公式 (4-1) 计算  $t$  时刻的拥堵波  $w$ 
| | 利用公式 (4-2) 计算拥堵波从  $j$  到达  $i$  点的时间  $t_{arrival}$ 
| | 预测  $i$  点  $t + t_{arrival}$  的车流量和车流速度并且计算此时的标准拥堵指数  $\gamma_{norm}^i(t)$ 
| | if  $\gamma_{norm}^i(t) > 0.6$  and  $w < 0$ :
| | | 预测点  $i$  的  $t + t_{arrival}$  之后直到  $\gamma_{norm}^i(t) < 0.4$  的时间  $\Delta_t$ 
| | | 预警： $t + t_{arrival}$  时刻后会发生拥堵，并且拥堵时间为  $\Delta_t$ 
| | | 在  $t$  时刻时打开  $i - j$  段应急车道
| | elif  $\gamma_{norm}^i(t) < 0.6$  and  $w > 0$ :
| | |  $i$  到  $j$  段不出现拥堵
| | | 关闭  $i - j$  段应急车道
| | else:
| | | 不采取行动
| else:
| | 利用公式 (4-1) 计算当前时刻的拥堵波  $w$ 
| | if  $w > 0$ :
| | | 利用公式 (4-2) 计算拥堵波从  $i$  到  $j$  点的时间  $t_{close}$ 
| | |  $i$  到  $j$  段在  $t_{close}$  后不出现拥堵
| | | 在  $t_{close}$  后关闭  $i - j$  段应急车道
| | else:
| | | 不采取行动
```

图 5.2 算法伪代码

伪代码中的， t 时间一般选 **10 分钟** 即预测 10 分钟后的节点的拥堵指数， $i、j$ 表示节点 1 到 4 中的任意相邻点，在开启应急车道的策略上我们采用节点 1 到 2 段，2 到 3 段，3 到 4 段分段管控，分段预测并且控制这三段路的应急车道的开启和关闭，具体流程如下表所示：

表 5.1 分段预测控制应急车道流程图

交通节点	控制路段	满足算法开启应急车道条件	满足算法关闭应急车道条件
节点 1、节点 2	1-2 之间的路段	开启 1-2 之间的路段	关闭 1-2 之间的路段
节点 2、节点 3	2-3 之间的路段	开启 2-3 之间的路段	关闭 2-3 之间的路段
节点 3、节点 4	3-4 之间的路段	开启 3-4 之间的路段	关闭 3-4 之间的路段

这种分段管理应急车道的好处在于能够根据每个路段的实时交通情况灵活调整应急车道的启用和关闭，从而提高交通效率。通过逐段控制，可以快速响应不同路段的拥堵情况，避免整体交通的滞碍。此外，分段预测还可以优化资源的利用，确保应急车道的使用更加精准和针对性，避免不必要的干预，从而提升整体道路的安全性和通行能力。综合来看，这种方法能够有效提升应急车道管理的灵活性和道路整体运行的稳定性。

5.3 量化启用应急车道模型

本文为了量化模型启用应急车道作用引入了 4 个变量分别为如下表所示：

表 5.2 模型变量解释

变量	名称	解释
n	车道数	道路的车道数量
$q_{\max}$	单车道容量	每条车道的最大通行能力
$Q_{\max}$	总容量	道路的总通行能力， $Q_{\max} = n \times q_{\max}$
v_f	自由流速度	表示每条道路在无干扰情况下的速度

5.3.1 Greenshields 模型建模

Greenshields 模型假设速度和密度之间呈线性关系：

$$v = v_f \left(1 - \frac{k}{k_j} \right) \quad (5-1)$$

其中 v_f 表示自由流速度（车辆在无干扰情况下的最高速度）。 k_j 表示堵塞密度（交通完全停止时的密度）。流量 q 可表示为：

$$q = k \times v = k \times v_f \left(1 - \frac{k}{k_j} \right) \quad (5-2)$$

这是一个关于密度 k 的二次函数，表示流量与密度之间的关系。在得到流量 q 的表示以后，利用该公式计算最大流量，将上式对密度 k 求导，找到最大流量对应的密度：

$$\frac{dq}{dk} = v_f \left(1 - 2 \frac{k}{k_j} \right) = 0 \quad (5-3)$$

解得 $k = \frac{k_j}{2}$ ，得出对应的最大流量和速度为：

$$q_{\max} = k_{\max} \times v_{\max} = \frac{k_j}{2} \times \frac{v_f}{2} = \frac{k_j v_f}{4} \quad (5-4)$$

5.3.2 模拟分析

假设条件：

- 自由流速度 $v_f = 100$ km/h。
- 交通完全停止时，车与车之间的间隔为 5m，则可求出堵塞密度 $k_j = 200$ veh/km (每车道)。
- 则可知单车道最大流量 $q_{\max} = \frac{k_j v_f}{4 \times 60} = \frac{250}{3}$ veh/min。

初始状态(两车道)：

- 车道数 $n_1 = 2$ 。
- 总容量 $Q_{\max} = n_1 \times q_{\max} = \frac{500}{3}$ veh/min。
- 假设交通需求量 $q_d = \frac{500}{3}$ veh/min, 即道路满载。

此时：

- 速度 $v_1 = \frac{v_f}{2} = 50$ km/h。
- 密度(每车道) $k_1 = \frac{k_j}{2} = 100$ veh/km。
- 总密度 $K_1 = n_1 \times k_1 = 200$ veh/km。

改造后状态(三车道)：

- 车道数 $n_2 = 3$ 。
- 总容量 $Q_{\max} = n_2 \times q_{\max} = 250$ veh/min。

情况一：交通需求不变

交通需求量仍为 $q_d = \frac{500}{3}$ veh/min。得出：

- 速度提升至自由流速度 $v_2 = v_f = 100$ km/h。
- 密度(每车道) $k_2 = \frac{q_d \times 60}{n_2 v_f} = \frac{10,000}{3 \times 100} \approx 33.33$ veh/km。
- 总密度 $K_2 = n_2 \times k_2 = 100$ veh/km。

变化分析：

- 速度提高了一倍，从 50 km/h 提升到 100 km/h。

- 总密度减少了一半,从 200 veh/km 降至 100 veh/km。
- 流量保持不变,为 $q_d = \frac{500}{3}$ veh/min。

情况二: 交通需求增加

假设交通需求增加至新容量,即 $q_d = 250$ veh/min。得出:

- 速度再次下降至 $v_2 = \frac{v_f}{k_l^2} = 50$ km/ha
- 密度(每车道) $k_2 = \frac{k_f^2}{2} = 100$ veh/km。
- 总密度 $K_2 = n_2 \times k_2 = 300$ veh/km。

变化分析:

- 流量增加了 50%,从 $\frac{500}{3}$ veh/min 增加到 250 veh/min。
- 速度保持在 50 km/h, 不变。
- 总密度增加了 50%,从 200 veh/km 增加到 300 veh/km。

5.3.3 量化结果分析

在对以上模型进行模拟分析以后,得出如下结论,总容量增加:车道增加使得道路的总容量按比例提升 (50%)。

速度变化:

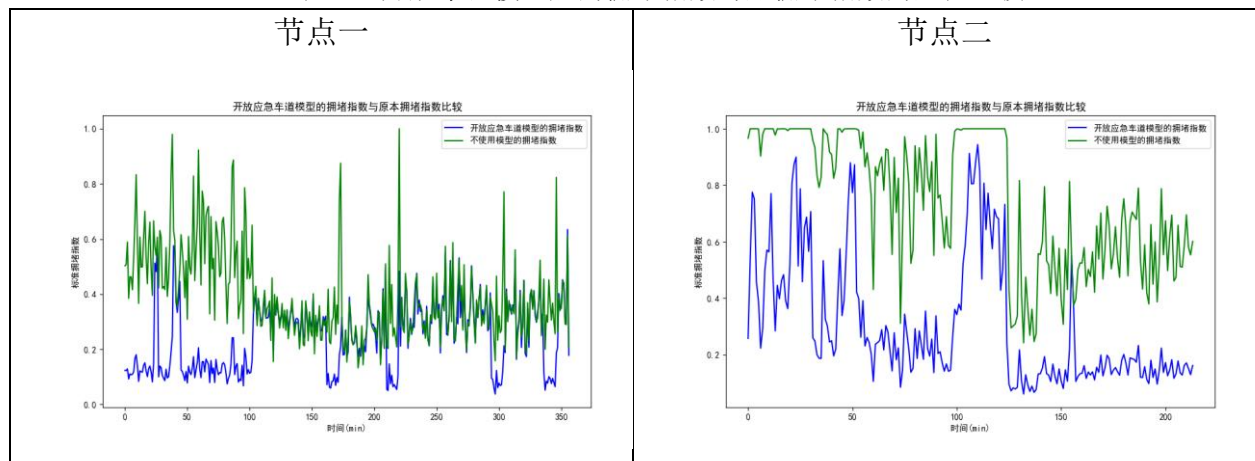
1. 如果交通需求不变,道路从拥堵状态转为畅通,平均速度提高(可能达到自由流速度)。
2. 如果交通需求增加至新容量,平均速度可能保持不变或略有提高,但仍处于拥堵状态。

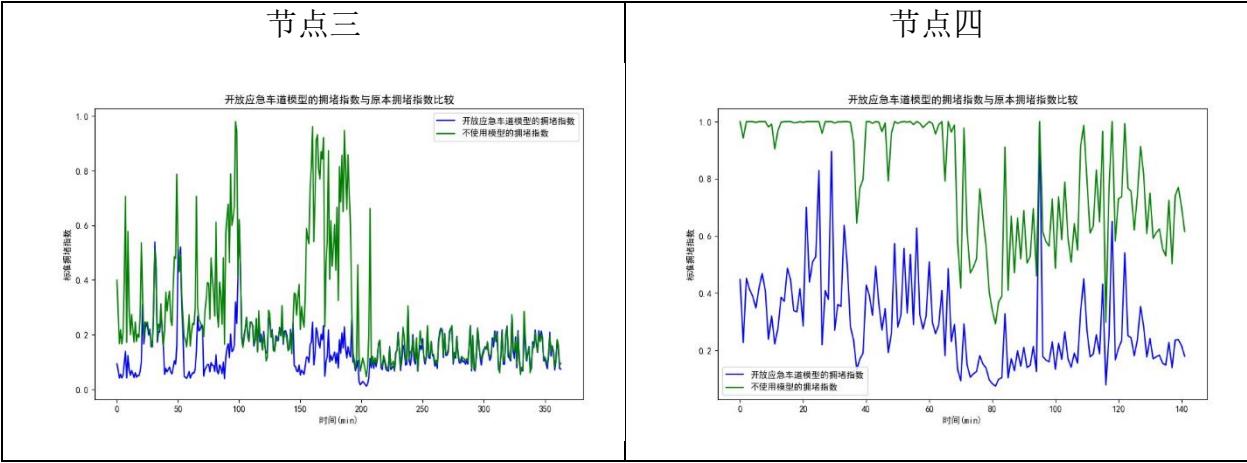
密度变化:

1. 在需求不变的情况下,车辆密度降低,行车舒适性提高。
2. 在需求增加的情况下,车辆密度可能增加,拥堵程度加剧。

同时为了验证理论的有效性,我们将以上理论带入交通预警开放车流模型中,同时假设总需求不变,得出在交通预警后开放应急车道 4 个节点的交通拥堵指数。此时,若总需求不变打开应急车道以后车辆速度将会增加为自由流速度以上,密度将会减小一半,以下为 4 个监控点在使用交通预警开放车流模型后的拥堵指数与原拥堵指数的比较图:

表 5.3 开放车流模型后的拥堵指数与原拥堵指数的量化比较





在得出开放应急车道模型后拥堵指数与原本拥堵指数比较图之后，我们对不启用模型中度拥堵以上的时间和启用模型后中度拥堵以上时间进行了统计，并且还计算了启用模型后开放应急车道的总时长，结果如下表：

表 5.4 模型启用应急车道量化结果表

观测点	观测时间	启用前中度拥堵以上时间（min）	启用后中度拥堵以上时间（min）	开放应急车道开启总时长（min）
节点一	11:41-16:14	35	1	60
节点二	11:52-15:28	143	29	150
节点三	11:35-16:08	28	1	55
节点四	12:56-15:20	110	7	130

对表格进行分析我们可以看出，应急车道在启用后显著减少了各观察点的中度拥堵时间，尤其是在节点二和四，拥堵时间下降了 80%以上。同时应急车道的利用率较高，在多个节点超过了 70%，显示了应急车道在开放期间对缓解交通拥堵具有良好的效果。

六、 问题四模型建立与求解分析

6.1 问题分析

优化监控点布置方案：为了提升第三个点到第四个点之间路段应急车道临时启用决策的科学性，同时考虑成本控制，给出最优的视频监控点布置方案。该方案应说明如何布置监控点，布置在何处，布置多少个，及其原因，以确保能够实时、准确地获取交通流信息并做出科学决策。

具体问题分析如下：

1. 空间覆盖不足：现有四个监控点布置在较大间距下，无法全面覆盖 5000 米的路段，尤其是在第三点到第四点之间，监控盲区较多。监控盲区导致对路况的实时监测不全面，无法及时发现和应对突发交通事件，降低了应急响应的效率。
2. 缺乏针对性数据：现有监控系统主要关注主车道的交通流量，未充分考虑应急车道的临时启用情况。在需要临时启用应急车道时，缺乏对应急车道使用情况及相关交通参数的实时监控，难以做出科学决策，影响道路通行效率和应急响应能力。
3. 数据滞后性：监控点间距较大，导致数据采集存在时间和空间上的滞后。无法实时捕捉路段内的交通流变化，尤其是在发生突发事件时，预警系统反应迟缓，影响交通管理的及时性和准确性。
4. 设备技术滞后：现有监控设备技术水平有限，数据处理能力不足，无法充分利用大数据和人工智能技术进行交通流分析和预测。限制了交通管理系统的智能化水平，无法实现对复杂交通状况的精准分析和预判

6.2 问题求解

1. 优化监控点布置的原则如下：

(1) 提高监控精度：在关键位置增设监控点，缩短监控点间距，提升对交通流变化的敏感度。

(2) 控制建设成本：在满足监控需求的前提下，尽量减少新增设备数量，充分利用现有资源。

(3) 重点监测区域：针对容易发生拥堵或交通流变化剧烈的区域，进行重点监控。

2. 监控点布置方案：

(1) 增加监控点数量：

- 在第三点到第四点之间新增 2-3 个监控点，将原本 5000 米的距离细分为更小的监控区间（如每 1000-1500 米一个监控点）。
- 关键位置布点：在容易产生瓶颈的区域，如上下坡、弯道、匝道入口处增设监控点。

(2) 利用移动监控设备：

- 无人机巡逻：在特定时段使用无人机对该路段进行巡逻，获取实时交通数据。
- 移动摄像头：安装在巡逻车辆上的摄像头，实时传输路况信息。

(3) 升级现有设备：

- 高精度传感器：在现有监控点添加高精度的交通流量和速度传感器，提升数据质量。

-
- 智能分析系统：引入人工智能和大数据分析，对监控视频进行实时处理，预测交通流趋势。

(4) 数据整合与共享

- 集中管理平台：将新增监控点的数据与现有系统进行整合，建立统一的交通管理平台，实现数据的集中管理、共享与分析。
- 实时数据传输：确保各监控点的数据能够实时传输到中心平台，保证数据的时效性和准确性。

3. 方案的合理性分析：

(1) 提升数据精度和实时性：

- 密集布点：更密集的监控点使得交通流数据的空间分辨率提高，能够更及时地发现拥堵苗头。
- 移动设备补充：无人机和移动摄像头的灵活部署，弥补固定监控点的局限，提供更加全面和灵活的监控能力。

(2) 支持科学决策：

- 实时数据支持：通过高精度和实时的数据采集，交通管理部门能够更准确地判断是否需要临时启用应急车道，优化交通流。
- 智能分析：人工智能和大数据分析系统能够对大量实时数据进行快速处理和分析，提供科学的决策支持，提升决策的前瞻性和准确性。

(3) 成本控制：

- 有限新增：新增监控点数量控制在 2-3 个之间，避免大规模投资，合理控制建设成本。
- 资源优化利用：通过升级现有设备和引入移动监控设备，充分利用现有资源，节约总体成本。

(4) 安全性考虑：

- 应急监控：在应急车道启用期间，新增监控点能够实时监测应急车道的使用情况，确保救援车辆的优先通行，提升整体道路安全性。

(5) 数据共享与整合：

- 集中管理：将所有监控点的数据集中管理，避免信息孤岛，提升数据的综合利用效率。
- 协同分析：不同监控点的数据可以进行协同分析，实现对整体交通状况的全面掌握和管理

(6) 未来扩展性：

- 技术升级：监控系统的升级为未来引入更多先进技术（如物联网、5G 通信等）奠定了基础，具备良好的扩展性。
- 智能交通管理：通过不断优化和升级监控系统，逐步实现智能交通管理，提升道路通行效率和管理水平。

通过优化监控点的布置，增加关键位置的监控点数量，利用移动监控设备和升级现有设备，不仅能够提升监控系统的覆盖范围和数据精度，还能增强交通管理的实时性和科学性。在成本控制的前提下，实现了监控系统的高效运行和智能化升级，为应急车道的临时启用提供了坚实的数据支持和决策保障，显著提升了整体交通管理水平和道路安全性。

七、 模型评价与改进

7.1 基于 YOLO 和 DeepSORT 的目标跟踪模型

模型优点：

- (1) 高效检测：YOLO 具有优秀的实时目标检测能力，能够在一帧中同时检测多个对象，为多目标跟踪提供了基础。
- (2) 可靠的特征匹配：DeepSORT 通过引入深度学习特征提取器，增强了目标的外观特征，提高了目标重识别的准确性，使得跟踪在遮挡和目标变换时能保持较高的稳定性。
- (3) 平衡速度与精度：YOLO 和 DeepSORT 两者结合使得模型在保持较高的检测精度的同时，仍然能够实现较快的处理速度，适用于实时应用场景。

模型缺点：

- (1) 深度特征提取性能不足：DeepSORT 中的特征提取网络较浅，在提取目标特征时可能会导致提取的特征信息太少，从而导致重识别性能下降。
- (2) 损失函数优化不足：YOLOv5 的损失函数可能无法充分反映检测任务中的所有需求，可能导致某些目标的检测精度降低。
- (3) 数据增强方法有限：输入端的数据增强方法较为简单，缺乏对多样性和复杂环境的充分适应，可能导致模型过拟合某些特定场景。
- (4) 比例系数推导误差：在求解汽车实际速度的过程中，在确定像素距离与实际物理距离之间的比例系数 k 时采用了反推的方法，即根据高速上车辆的最高时速进行反推计算，这可能存在一定的误差。如果有更加详细的信息，我们可以对比例系数 k 进行更精确的推导。

改进措施：

- (1) 改进 DeepSORT 特征提取网络：增加特征提取模型的深度或使用更先进的网络架构（如 EfficientNet 等），以提升目标特征提取的性能，增强模型的重识别能力。
- (2) 优化损失函数：根据检测任务要求设计或改进 YOLOv5 的损失函数，例如引入 FocalLoss 等，以提高模型在复杂场景中的检测性能和鲁棒性。
- (3) 丰富数据增强方法：采用更为多样化的数据增强方法（如 Cutmix、Mixup、RandomErasing 等），以增强模型对不同环境和条件下目标的适应能力。
- (4) 精确推导比例系数 k ：收集更多关于车辆实际速度和对应像素距离的数据，采用统计分析方法和机器学习模型进行更为精确的比例系数推导，从而减少因误差带来的影响。
- (5) 多模态信息融合：考虑将更多的信息（如传感器数据、IMU 传感器、GPS 信息等）融入跟踪系统中，以提升模型对目标运动状态的理解和预测能力。
- (6) 模型集成及迁移学习：对于特定应用场景，可以考虑模型集成或迁移学习，利用在其他数据集上预训练的模型以提高检测和跟踪的准确性。

7.2 基于 Bi-GRU 的时序预测模型

模型优点：

- (1) 多尺度特征捕捉：SKNet 模块能够关注不同尺度的特征，使模型更好地识别对回潮设备状态预测至关重要的信息。

-
- (2) 长期依赖性: TCN 模块可以处理长期时间依赖性, 同时保持较低延迟, 适合时间序列分析。
 - (3) 双向信息流: BiGRU 通过正向和反向的信息流处理序列数据, 有助于捕捉到过去和未来的上下文信息, 提高预测的准确性。
 - (4) 残差连接: TCN 中的残差连接有助于缓解深层网络的训练困难, 提高模型的训练效率和性能。
 - (5) 融合特征: SKNet 和 TCN 的融合机制能够结合空间和时间特征, 提供更全面的输入信息。

模型缺点:

- (1) 复杂性: 模型结构较为复杂, 涉及多个模块, 可能导致实现和调试的难度加大。
- (2) 参数量大: 多个模块叠加可能会导致参数量的增加, 进而可能导致过拟合, 特别是在数据量不足的情况下。
- (3) 实时性问题: 尽管 TCN 具有较低的延迟, 但整体模型的复杂性可能影响实时预测的能力, 尤其是在大规模应用场景中。
- (4) 对输入数据质量依赖: 模型的表现对输入数据的质量和预处理结果非常敏感, 噪声或异常值可能会显著降低预测性能。

改进方法:

- (1) 模型简化: 尝试简化模型结构, 例如通过减少层数或使用更轻量级的替代模块, 以降低计算和实现难度。
- (2) 正则化技术: 引入正则化技术 (如 Dropout、L2 正则化) 以减轻过拟合, 并提高模型在小数据集上的泛化能力。
- (3) 动态融合机制: 考虑引入动态特征融合机制, 根据不同时间段的特征重要性动态调整融合方式, 提高模型的适应性。
- (4) 自适应学习率: 使用自适应学习率方法 (如 Adam、RMSprop) 来优化训练过程, 提高收敛速度和最终性能。
- (5) 集成学习: 可考虑使用集成学习的方法, 结合多个不同模型的预测结果, 以进一步提高性能稳健性。

参考文献

- [1] 崔妍,刘东.北京市朝阳路可变车道交通组织研究[J].道路交通与安全,2006,(09):21-24.
- [2] M. Aron, S. Cohen and R. Seidowsky, "Two French Hard-Shoulder Running operations: Some comments on effectiveness and safety," 13th International IEEE Conference on Intelligent Transportation Systems, Funchal, Portugal, 2010, pp. 230-236.
- [3] 郑淑鉴,杨敬锋.国内外交通拥堵评价指标计算方法研究[J].公路与汽运,2014,(01):57-61.
- [4] Horvat M, Gledec G. A comparative study of YOLOv5 models performance for image localization and classification[C]//In Central European Conference on Information and Intelligent Systems. Faculty of Organization and Informatics Varazdin, 2022: 349-356.
- [5] 彭雅坤, 曹伊宁, 刘晓群. 基于 YOLOv5s 的滑雪人员检测研究[J]. 长江信息通信, 2021, 34(08):24-26.
- [6] 马琳琳, 马建新, 韩佳芳, 等. 基于 YOLOv5s 目标检测算法的研究[J]. 电脑知识与技术, 2021, 17(23): 100-103.
- [7] 曾文鑫. 基于机器学习的车标图像识别研究[D]. 西安石油大学, 2021.
- [8] 梁格颖, 王文琪, 汪文, 等. 基于空洞卷积的语义图像分割算法研究[J]. 信息通信, 2019(06): 33-36.
- [9] 刘锟. 基于改进的 YOLOX 目标检测算法研究[D]. 南昌大学, 2022.
- [10] 舒军, 王祥, 舒心怡. 基于 YOLACT++的槟榔检测算法研究[J]. 湖北工业大学学报, 2022, 37(04):29-35.
- [11] 白玉, 马广焜, 彭新茗, 等. 基于 YOLO 框架的农田火源自动检测系统[J]. 现代计算机, 2022, 28(19): 33-38.
- [12] 徐诚, 殷楠, 段世红, 等. 基于奖励滤波信用分配的多智能体深度强化学习算法[J]. 计算机学报, 2022,45(11): 2306-2320.
- [13] 陈伯云, 陆安江, 周骅, 等. 激光点云中三维多目标在线跟踪方法[J]. 激光杂志, 2023, 44(04): 68-73.DOI: 10.14016/j.cnki.jgzz.2023.04.068.
- [14] 吴凯, 党鑫, 郭风祥, 等. 神经网络异构加速跟踪系统设计与实现[J]. 小型微型计算机系统, 2022,43(11): 2257-2262.
- [15] 马瑞, 蔡满春, 彭舒凡. 一种基于改进的 Xception 网络的深度伪造视频检测模型[J]. 信息安全, 2021,21(12): 109-117.

附录 python 程序

程序模块 1: YOLO-DeepSort 速度计算代码示例:

```
1. from collections import defaultdict
2. import cv2
3. import numpy as np
4. import csv
5.
6. from ultralytics.utils.checks import check_imshow
7. from ultralytics.utils.plotting import Annotator, colors
8.
9.
10. class SpeedEstimator:
11.
12.     def __init__(self):
13.
14.         self.im0 = None
15.         self.annotator = None
16.         self.view_img = False
17.
18.         self.reg_pts = [(20, 400), (1260, 400)]
19.         self.region_thickness = 3
20.
21.         # Speed line at x = 230
22.         self.speed_line_x = 1230
23.
24.         # Predict/track information
25.         self.cls = None
26.         self.names = None
27.         self.bboxes = None
28.         self.trk_ids = None
29.         self.line_thickness = 1
30.         self.trk_history = defaultdict(list)
31.
32.         # Speed estimator information
33.         self.dist_data = {}
34.         self.trk_previous_points = {}
35.         self.distance_meters = 50 # 假设区域对应实际距离为 10 米
36.         self.pixels_per_meter = None # 像素与米的转换比例
37.
38.         # CSV 文件初始化
39.         self.csv_file = open('vehicle_speeds.csv', 'w', newline='', encoding='utf-8')
40.         self.csv_writer = csv.writer(self.csv_file)
```

```

41.         self.csv_writer.writerow(['Frame Number', 'Vehicle ID', 'Speed at x=230 (km
/h)'])
42.
43.         # 用于记录哪些车辆已经记录过速度，防止重复记录
44.         self.logged_ids = set()
45.
46.         # 帧计数器
47.         self.frame_number = 0 # 初始化帧数
48.
49.         # Check if environment supports imshow
50.         self.env_check = check_imshow(warn=True)
51.
52.     def __del__(self):
53.         self.csv_file.close()
54.
55.     def set_args(
56.         self,
57.         reg_pts,
58.         names,
59.         view_img=False,
60.         line_thickness=2,
61.         region_thickness=5,
62.         spd1_dist_thresh=10,
63.         distance_meters=50,
64.     ):
65.
66.         if reg_pts is None:
67.             print("Region points not provided, using default values")
68.         else:
69.             self.reg_pts = reg_pts
70.             self.names = names
71.             self.view_img = view_img
72.             self.line_thickness = line_thickness
73.             self.region_thickness = region_thickness
74.             self.spd1_dist_thresh = spd1_dist_thresh
75.             self.distance_meters = distance_meters
76.
77.         # 计算像素与米的转换比例
78.         self.distance_pixels = abs(self.reg_pts[1][0] - self.reg_pts[0][0])
79.         if self.distance_pixels == 0:
80.             self.pixels_per_meter = 1 # 防止除以零
81.         else:
82.             self.pixels_per_meter = self.distance_pixels / self.distance_meters
83.

```

```

84.     def extract_tracks(self, tracks):
85.
86.         self.bboxes = tracks[0].bboxes.xyxy.cpu()
87.         self.cls = tracks[0].bboxes.cls.cpu().tolist()
88.         self.trk_ids = tracks[0].bboxes.id.int().cpu().tolist()
89.
90.     def store_track_info(self, track_id, box):
91.
92.         track = self.trk_history[track_id]
93.         bbox_center = ((box[0] + box[2]) / 2.0, (box[1] + box[3]) / 2.0)
94.         track.append(bbox_center)
95.
96.         if len(track) > 30:
97.             track.pop(0)
98.
99.         return track
100.
101.     def plot_box_and_track(self, track_id, box, cls, track):
102.
103.         speed = int(self.dist_data.get(track_id, 0))
104.         speed_label = f"{speed} km/h"
105.         bbox_color = colors(int(track_id)) if speed > 0 else (255, 0, 255)
106.
107.         self.annotator.box_label(box, speed_label, bbox_color)
108.
109.         trk_pts = np.array(track).astype(np.int32).reshape((-1, 1, 2))
110.         cv2.polylines(self.im0, [trk_pts], isClosed=False, color=(0, 255, 0), thickness=1)
111.         cv2.circle(self.im0, (int(track[-1][0]), int(track[-1][1])), 5, bbox_color, -1)
112.
113.     def calculate_speed(self, trk_id, track):
114.
115.         if len(track) >= 2:
116.             curr_pos = track[-1]
117.             prev_pos = track[-2]
118.
119.             # 初始化前一次的位置
120.             if trk_id not in self.trk_previous_points:
121.                 self.trk_previous_points[trk_id] = curr_pos
122.             return
123.
124.             time_difference = 0.4 # 固定的时间差 (秒)
125.

```



```

126.         dist_difference = np.linalg.norm(np.array(curr_pos) - np.array(self.tr
k_previous_points[trk_id]))
127.         speed_pixels_per_sec = dist_difference / time_difference
128.         speed_m_per_sec = speed_pixels_per_sec / self.pixels_per_meter
129.         speed_km_per_h = speed_m_per_sec * 3.6
130.
131.         self.dist_data[trk_id] = speed_km_per_h
132.
133.         # 更新前一次的位置
134.         self.trk_previous_points[trk_id] = curr_pos
135.
136.         # 检查车辆是否经过 x = 230, 并且未记录过速度
137.         if trk_id not in self.logged_ids:
138.             # 检测车辆是否经过 x = 230
139.             if (prev_pos[0] < self.speed_line_x <= curr_pos[0]) or (prev_pos[0
] > self.speed_line_x >= curr_pos[0]):
140.                 # 将帧数、车辆 ID 和速度写入 CSV 文件
141.                 self.csv_writer.writerow([self.frame_number, trk_id, speed_km_
per_h])
142.                 self.logged_ids.add(trk_id)
143.                 print(f"Frame {self.frame_number}: Vehicle {trk_id} speed at x
=230: {speed_km_per_h:.2f} km/h")
144.
145.     def estimate_speed(self, im0, tracks, region_color=(255, 0, 0)):
146.
147.         self.im0 = im0
148.         self.frame_number += 1 # 增加帧数计数器
149.
150.         if tracks[0].boxes.id is None:
151.             if self.view_img and self.env_check:
152.                 self.display_frames()
153.             return im0
154.         self.extract_tracks(tracks)
155.
156.         self.annotator = Annotator(self.im0, line_width=self.line_thickness)
157.         # 绘制速度计算区域
158.         self.annotator.draw_region(reg_pts=self.reg_pts, color=region_color, thick
ness=self.region_thickness)
159.         # 绘制 x = 230 的线
160.         cv2.line(
161.             self.im0,
162.             (int(self.speed_line_x), 0),
163.             (int(self.speed_line_x), self.im0.shape[0]),
164.             (0, 255, 255),

```

```

165.         self.region_thickness,
166.     )
167.
168.     for box, trk_id, cls in zip(self.bboxes, self.trk_ids, self.cls):
169.         track = self.store_track_info(trk_id, box)
170.
171.         self.calculate_speed(trk_id, track)
172.         self.plot_box_and_track(trk_id, box, cls, track)
173.
174.         if self.view_img and self.env_check:
175.             self.display_frames()
176.
177.     return im0
178.
179.     def display_frames(self):
180.         cv2.imshow("Ultralytics Speed Estimation", self.im0)
181.         if cv2.waitKey(1) & 0xFF == ord("q"):
182.             return
183.
184.
185. if __name__ == "__main__":
186.     SpeedEstimator()

```

程序模块 2: TCNAt-BiGRU 时间预测代码

```

1. import pandas as pd
2. import numpy as np
3. from sklearn.model_selection import train_test_split
4. from sklearn.preprocessing import MinMaxScaler
5. from keras.models import Model
6. from keras.layers import Dense, LSTM, Bidirectional, Input
7. from keras.layers import Attention
8. from tcn import TCN, tcn_full_summary
9. from keras.optimizers import Adam
10. import matplotlib.pyplot as plt
11. import matplotlib
12. from matplotlib import rcParams
13. # 数据预处理
14. # 读取 CSV 文件
15. data = pd.read_csv('C:/Users/cy/Desktop/dataprocess/datacomple2_3.csv', encoding='gbk')
16.
17. # 选取特征和目标变量
18. X = data.iloc[:72, [1, 2]].values
19. y = data.iloc[:72, 1].values
20.

```

```

21. # 特征缩放
22. scaler = MinMaxScaler(feature_range=(0, 1))
23. X = scaler.fit_transform(X)
24. y = y.reshape(-1, 1)
25. y = scaler.fit_transform(y)
26.
27. # 划分训练集和测试集
28. X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
29.
30. # 重构输入数据的维度，以符合 LSTM 的要求
31. X_train = X_train.reshape((X_train.shape[0], 1, X_train.shape[1]))
32. X_test = X_test.reshape((X_test.shape[0], 1, X_test.shape[1]))
33.
34. from keras.layers import Input, Dense, Bidirectional, GRU, Multiply, Lambda, Concatenate, Activation, LayerNormalization, Dropout
35. from keras.models import Model
36. from keras.optimizers import Adam
37. from keras import backend as K
38.
39. # 构建模型的函数
40. def build_model(input_shape, tcn_nb_filters, tcn_kernel_size, tcn_nb_stacks, tcn_dilations, gru_units, output_units, learning_rate):
41.     # SK-Net 注意力机制
42.     def sk_net_attention(tensor, channels):
43.         # 全局平均池化和最大池化
44.         avg_pool = Lambda(lambda x: K.mean(x, axis=1))(tensor)
45.         max_pool = Lambda(lambda x: K.max(x, axis=1))(tensor)
46.
47.         # 分别通过全连接层
48.         fc1_avg = Dense(channels // 16, activation='relu')(avg_pool)
49.         fc1_max = Dense(channels // 16, activation='relu')(max_pool)
50.
51.         # 合并特征
52.         fc = Concatenate(axis=-1)([fc1_avg, fc1_max])
53.         attention = Dense(channels, activation='sigmoid')(fc)
54.
55.         # 应用注意力机制
56.         x = Multiply()([tensor, attention])
57.         return x
58.
59. # 模型输入层
60. input_layer = Input(shape=(input_shape[1], input_shape[2]))
61.

```

```

62.     # TCN 层
63.     tcn_layer = TCN(nb_filters=tcn_nb_filters, kernel_size=tcn_kernel_size, nb_stac
ks=tcn_nb_stacks,
64.                     dilations=tcn_dilations, padding='causal', return_sequences=True)(i
nput_layer)
65.
66.     # 添加 Dropout 层防止过拟合
67.     dropout1 = Dropout(0.2)(tcn_layer)
68.
69.     # 双向 GRU 层
70.     bigru_layer = Bidirectional(GRU(gru_units, return_sequences=True))(dropout1)
71.     dropout2 = Dropout(0.2)(bigru_layer)
72.
73.     # 应用 SK-Net 注意力机制
74.     sknet_attention_layer = sk_net_attention(dropout2, gru_units * 2)
75.
76.     # 归一化层
77.     norm_layer = LayerNormalization()(sknet_attention_layer)
78.
79.     # 输出层
80.     output_layer = Dense(output_units, activation='sigmoid')(norm_layer)
81.
82.     # 构建并编译模型
83.     model = Model(inputs=input_layer, outputs=output_layer)
84.     model.compile(optimizer=Adam(learning_rate=learning_rate), loss='mean_squared_error
')
85.
86.     return model
87.
88. # 构建模型
89. input_shape = X_train.shape
90. model = build_model(input_shape, tcn_nb_filters=64, tcn_kernel_size=3, tcn_nb_stacks=1,
91.                     tcn_dilations=[1, 2, 4], gru_units=128, output_units=1, learning_ra
te=0.0025)
92.
93. # 训练模型并保存历史记录
94. history = model.fit(X_train, y_train, epochs=100, batch_size=128, validation_split=0.2,
verbose=1)
95.
96. # 保存模型
97. model.save('C:/Users/cy/Desktop/datacomple.h5')
98. # model.save('C:/Users/cy/Desktop/h5/datacomple4_flow.h5')
99.

```

```

100.
101. # 设置字体为 SimHei（黑体），以支持中文显示
102. plt.rcParams['font.sans-serif'] = ['SimHei'] # 用于显示中文
103. plt.rcParams['axes.unicode_minus'] = False # 用于正常显示负号
104.
105. # 绘制训练集和验证集的损失曲线
106. plt.plot(history.history['loss'], label='Train Loss', color='red') # 训练集损失红
色
107. plt.plot(history.history['val_loss'], label='Validation Loss', color='green') # 验
证集损失绿色
108. plt.title('第二个点车流量时序预测模型训练/验证损失曲线')
109. plt.ylabel('损失')
110. plt.xlabel('训练轮次')
111. plt.legend()
112. plt.show()
113.
114.
115. # 测试模型
116. y_pred = model.predict(X_test)
117. y_pred_reshaped = y_pred.reshape(y_pred.shape[0], -1)
118. y_pred = scaler.inverse_transform(y_pred_reshaped)
119. y_test = scaler.inverse_transform(y_test)
120.
121. # 计算评价指标
122. mse = np.mean((y_pred - y_test)**2)
123. rmse = np.sqrt(mse)
124. mae = np.mean(abs(y_pred - y_test))
125. mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
126. ss_res = np.sum((y_test - y_pred) ** 2)
127. ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
128. r2 = 1 - (ss_res / ss_tot)
129.
130. # 输出评价指标
131. print('均方误差 (MSE):', mse)
132. print('均方根误差 (RMSE):', rmse)
133. print('平均绝对误差 (MAE):', mae)
134. print('平均绝对百分比误差 (MAPE):', mape)
135. print('决定系数 (R2):', r2)
136.
137. # Predict for the entire dataset
138. X_reshaped = X.reshape((X.shape[0], 1, X.shape[1]))
139. y_pred_all = model.predict(X_reshaped)
140.
141. # Inverse transform the predictions and original values

```

```
142. y_pred_all_reshaped = y_pred_all.reshape(y_pred_all.shape[0], -1)
143. y_pred_all = scaler.inverse_transform(y_pred_all_reshaped)
144. y_all = scaler.inverse_transform(y)
145. # 计算评价指标
146. mse = np.mean((y_pred - y_test)**2)
147. rmse = np.sqrt(mse)
148. mae = np.mean(abs(y_pred - y_test))
149. mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
150. ss_res = np.sum((y_test - y_pred) ** 2)
151. ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
152. r2 = 1 - (ss_res / ss_tot)
153.
154. # 输出评价指标
155. print('均方误差 (MSE):', mse)
156. print('均方根误差 (RMSE):', rmse)
157. print('平均绝对误差 (MAE):', mae)
158. print('平均绝对百分比误差 (MAPE):', mape)
159. print('决定系数 (R2):', r2)
```